

Configuring Formal Methods for Industry: Inferring, Distributing, and Propagating JML Specifications

Brendan Edmonds

Bachelor of Information Technology

A thesis submitted for the degree of
Doctor of Philosophy at
The University of Queensland
in 2026

School of Electrical Engineering and Computer Science
UQ Cyber

Abstract

Formal methods — the use of precise, machine-checkable specifications to verify that software behaves as intended — have been available for decades and are demonstrably effective, yet their use remains confined to a few niche domains such as safety-critical and security-critical systems. This thesis asks why, and whether the situation can be changed. Its premise is that the binding constraint is not the verification technology, which is mature, but the manual cost of writing the specifications that technology requires as input: specifications are expensive to author by hand, the cost scales with the size of the codebase, and it is paid up front before any benefit is realised. The thesis investigates whether automatically inferring specifications from existing code — removing that cost — and making applications easier to verify as a result can broaden the adoption of formal methods beyond their current niche.

The work is built around *JML-Inferer*, a static analysis system developed for this thesis that infers Java Modeling Language (JML) contracts — preconditions, postconditions, loop invariants, and frame conditions — from Java source by pattern matching organised around the weakest-precondition and strongest-postcondition calculi, with every inferred clause discharged *a posteriori* through a formal verifier so that the heuristic front end’s unsoundness is filtered rather than shipped. The first study establishes that this removes the cost barrier in practice: across 312 methods of a real library, the engine infers specifications at over 90% precision against both a human and a formal oracle, and supplying them to a large-language-model test generator improves its fault-detection (mutation) score by 40.7 percentage points over a signature-only baseline, evidence that cheap, automatically inferred specifications carry real value.

The second and third studies develop a second, more consequential consequence of inferred specifications for adoption: that they make software’s behavioural *compatibility* analysable at the library boundary. Because a verifier checks a caller against its callees’ contracts, a specification carried with a compiled library lets a consumer verify client code against a dependency without that dependency’s source; the thesis contributes an embedding mechanism that carries inferred JML losslessly inside Java bytecode and OpenAPI descriptions, validated across more than twenty thousand methods at under 11% overhead. And because a caller’s specification is *derived from* its callees’ specifications, a change to a library propagates into the contracts of its callers: when a library’s inferred specifications

change, the compatibility of client code with the new version becomes a question that can be answered at the specification level rather than guessed. The thesis measures the mechanism this rests on — that caller specifications depend substantially on callee specifications — across five libraries and 21,052 methods, finding that a compositional analysis recovers 42,370 cross-method obligations a single-pass analysis cannot express, dominated by exactly the guard-conditional and dispatch-dependent shapes through which library changes propagate to callers.

Taken together, the studies argue that the adoption barrier formal methods have long faced is, in significant part, removable: specifications can be inferred accurately and cheaply, distributed with the artefacts that need them, and made to propagate across library boundaries so that verification and compatibility analysis become routine rather than niche.

Declaration by Author

This thesis is composed of my original work, and contains no material previously published or written by another person except where due reference has been made in the text. I have clearly stated the contribution by others to jointly-authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my higher degree by research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Statement of Parts of the Thesis Submitted for Another Degree

None.

Research Involving Human or Animal Subjects

No animal or human subjects were involved in this research.

Publications During Candidature

This thesis is presented as a traditional monograph: a single continuous argument, with one unified review of the literature and one account of the methods, into which the research has been integrated rather than assembled from separately published chapters. The material has nonetheless been developed through three manuscripts, listed below, whose results are incorporated and substantially expanded within the unified narrative. In each case the candidate (B. Edmonds) was responsible for the conception of the study, the design and implementation of the software artefacts, the design and execution of the experiments, the analysis of the results, and the writing; the associated supervisor (M. Utting) provided guidance on study design, framing, and presentation.

1. B. Edmonds and M. Utting. “Do Specifications Help Large Language Models Write Better Unit Tests? An Empirical Study with Inferred JML Contracts.” *Manuscript prepared for submission to the Journal of Software: Evolution and Process (JSEP)*.
2. B. Edmonds and M. Utting. “Embedding Inferred JML Contracts in Java Bytecode and OpenAPI Specifications.” *Manuscript prepared for submission to a software-engineering conference*.
3. B. Edmonds and M. Utting. “Compositional Propagation of Inferred Specifications and Its Use for Library Compatibility.” *Manuscript in preparation*.

The research reported in these manuscripts is distributed across the thesis according to the argument’s structure rather than confined to one chapter apiece: the inference engine and its accuracy underpin Chapters 3 and 4, the distribution mechanism Chapter 5, and the compositional propagation and its compatibility consequence Chapter 6.

Contributions by Others to the Thesis

No significant contributions were made by others beyond the supervisory guidance acknowledged above.

Acknowledgements

I am grateful to my principal advisor, Professor Mark Utting, whose work on specification-based testing and model-based testing shaped the questions this thesis asks, and whose insistence on the distinction between heuristic plausibility and formal soundness is woven through every chapter. I thank my associate advisor, Dr Guowei Yang, for guidance on the empirical methodology and for sharpening the framing of the research questions.

I thank the maintainers of OpenJML, JavaParser, and the ASM bytecode library, without whose open-source work the artefacts described here could not have been built, and the Apache Commons and Google Guava communities, whose code served as the proving ground for every empirical claim in this thesis.

Finally, I thank my family for their patience and support over the course of this candidature.

Contents

Abstract	i
Declaration by Author	iii
Publications During Candidature	v
Acknowledgements	vi
1 Introduction	1
1.1 Why Formal Methods Stay Niche	1
1.1.1 What Code Does Versus What It Should Do	1
1.2 The Specification Gap in Practice	3
1.3 Two Routes Out of the Niche	4
1.4 Thesis Statement	5
1.5 Research Questions	6
1.6 Contributions	7
1.7 Research Approach and Validation Strategy	8
1.8 A Note on Methodology and Stance	9
1.9 The Contributions in Relation to the Literature	9
1.10 Scope and Delimitations	10
1.11 Thesis Structure	11
2 Background and Related Work	13
2.1 Program Reasoning: Hoare Logic and Predicate Transformers	13
2.1.1 The Lineage of Program Proof	14
2.1.2 The Weakest Precondition	14
2.1.3 The Transformer Rules, Concretely	15

2.1.4	The Strongest Postcondition	15
2.1.5	Why a Complete Implementation Does Not Scale	16
2.1.6	Two Uses of a Specification: Verifying and Testing	17
2.2	Abstract Interpretation and Sound Approximation	17
2.3	The Java Modeling Language and Its Verifiers	18
2.3.1	Semantics That Matter for Inference	19
2.3.2	A Complete Contract, Annotated	19
2.3.3	Verification Tools	20
2.4	Design by Contract and the Idea of an Obligation	21
2.5	Specification Inference	22
2.5.1	Dynamic Invariant Detection	22
2.5.2	Why Static and Dynamic Inference Find Different Things	22
2.5.3	Verification-Driven Annotation Inference	23
2.5.4	Documentation-Based Extraction	24
2.5.5	Language-Model-Based Inference	25
2.5.6	A Framework for Comparing Inference Approaches	26
2.6	Language-Model Test Generation and the Oracle Problem	26
2.6.1	The Test Oracle, Defined	27
2.6.2	The Oracle Problem	27
2.6.3	Why Specifications Should Help	28
2.6.4	The Broader Landscape of Learned Code Models	28
2.7	Mutation Testing as Evaluation Methodology	29
2.8	SMT Solvers and the Limits of Automated Discharge	30
2.9	Runtime Verification and the Spectrum of Checking	31
2.10	Distributing Specifications Alongside Compiled Code	32
2.11	Compositional and Interprocedural Analysis	32
2.12	Specification-Based Testing and Test Generation	34
2.13	The Cost of Specification and the Adoption Barrier	35
2.14	Positioning of This Thesis	36
3	The JML-Inferrer System	37
3.1	Architecture	37
3.1.1	Design Philosophy and Engineering Constraints	38

3.1.2	Why Pattern Matching Rather Than Symbolic Execution	39
3.1.3	The Sub-Analyser Composition	40
3.2	The WP/SP Design Discipline	41
3.3	Precondition Inference	41
3.3.1	What the Discipline Excludes	42
3.3.2	A Worked Example	42
3.4	Postcondition Inference	43
3.4.1	Branch-Sensitive Postconditions	44
3.4.2	The Symbolic Executor and Path Conditions	44
3.4.3	Soundness Gating by Return Type	45
3.5	Loop Invariant Inference	45
3.5.1	Promotion of Invariants to Postconditions	47
3.6	Frame Conditions and Purity	47
3.7	Interprocedural Propagation	48
3.7.1	The Standard Library Specification Database	49
3.7.2	The Boundaries of Heuristic Inference	50
3.8	Formal Validation	50
3.8.1	Extending the Verifier	51
3.8.2	Strictness Configuration	51
3.8.3	The Two Layers Are Independent	51
3.9	Class Invariants and History Constraints	52
3.10	An End-to-End Example	53
3.11	Method Categorisation	54
3.11.1	How Inference Behaviour Varies by Category	55
3.12	Operation, Logging, and Diagnostics	56
3.13	Summary	56
4	Specifications for Language-Model Test Generation	58
4.1	The Oracle Problem, Concretely	58
4.2	Experimental Design	59
4.2.1	Subject Selection	60
4.2.2	Metrics	60
4.2.3	Statistical Analysis	61

4.2.4	The Experimental Infrastructure	61
4.2.5	Reproducibility and the Replication Package	62
4.3	Fidelity of the Inferred Specifications (RQ1)	63
4.3.1	Manual Validation Against Documented Intent	63
4.3.2	The Manual Validation Protocol	64
4.3.3	Formal Discharge Against the Implementation	65
4.3.4	The Comparison Baselines in Detail	66
4.3.5	Comparison With Other Inference Approaches	67
4.4	Value of Specifications for Test Generation (RQ2)	67
4.4.1	The Principal Finding	67
4.4.2	Why More Tests Are Not Better Tests	69
4.4.3	Specification Strength and Its Consequences	69
4.4.4	Where Specifications Help Most	70
4.5	A Closer Look at Individual Classes	71
4.6	Where the Mutants Are Killed	72
4.7	Discussion	73
4.7.1	What the Two Results Together Establish	73
4.7.2	The Three Tractability Limits	73
4.7.3	The Tractability Limits, With Examples	74
4.7.4	Implications	74
4.8	Qualitative Analysis of Generated Tests	76
4.9	Threats to Validity	77
4.10	The Generalisation to Service-Boundary Code	78
5	Distributing Inferred Specifications	79
5.1	The Distribution Gap	79
5.1.1	Why Existing Channels Fall Short	80
5.2	Format Design	80
5.2.1	The Annotation Type	81
5.2.2	Three Size Optimisations	81
5.2.3	Canonical Form	82
5.2.4	The OpenAPI Extension and the Sidecar	82
5.2.5	The OpenAPI Extension in Depth	83

5.3	Deployment Scenarios	84
5.4	Implementation	84
5.5	What Lossless Roundtrip Means, Precisely	86
5.6	Empirical Validation	86
5.6.1	Lossless Roundtrip (RQ3a)	87
5.6.2	Byte Overhead (RQ3b)	87
5.6.3	Format Optimisation	89
5.6.4	Throughput (RQ3c)	89
5.6.5	Throughput in Context (RQ3c)	90
5.6.6	Toolchain Compatibility (RQ3d)	90
5.7	The Class-File Format and Why the Mechanism Is Transparent	91
5.7.1	A Worked Embedding	92
5.8	Consuming Embedded Specifications	92
5.9	Discussion and Threats	93
5.9.1	What the Result Establishes	93
5.9.2	Format Evolution and Versioning	94
5.9.3	When to Embed and When Not To	95
5.9.4	Threats to Validity	95
6	Compositional Propagation and Library Compatibility	97
6.1	Why the Dependence of Caller on Callee Matters	97
6.2	Where Single-Pass Propagation Is Blind	98
6.3	The Refinement Pass	99
6.3.1	Driver	100
6.3.2	Per-Method Refinement	100
6.3.3	Why Postcondition Propagation Needs a Termination Gate	100
6.3.4	The Driver in Detail	101
6.3.5	Class-Hierarchy Analysis and Dispatch Precision	102
6.3.6	What the Pass Is and Is Not	102
6.4	The Soundness of Substitution	103
6.5	Empirical Measurement	104
6.5.1	Subjects and Method	104
6.5.2	The Call Graph and the Measurement Harness	105

6.5.3	Headline Result (RQ4)	105
6.5.4	Shape Distribution	106
6.5.5	Interpreting the Magnitude	107
6.5.6	Runtime Cost	108
6.6	The Character of Each Library	108
6.7	From Compositional Dependence to Compatibility Analysis	110
6.8	Preliminary Downstream Signal	111
6.9	Worked Examples of the Two Shapes	113
6.10	Positioning	114
6.11	Implications for Inferer Architecture	114
6.12	Threats to Validity	115
6.13	Summary	116
7	Discussion and Synthesis	118
7.1	The Adoption Argument	118
7.2	The Coherence of the Whole	119
7.3	Heuristic Inference, Formal Validation	120
7.4	What the Studies Jointly Establish About Value	121
7.5	The Economics of the Pipeline	122
7.6	Tensions and Design Trade-offs	123
7.7	The Language Model as Producer and Consumer	124
7.8	Toward Automated Specification Engineering	125
7.9	Combined Threats to Validity	126
7.10	Engineering Lessons	127
7.11	The Recurring Role of Negative Results	128
7.12	What Would Falsify the Thesis	129
7.13	Summary	130
8	Conclusion and Future Work	131
8.1	Summary of Contributions	131
8.2	The Empirical Findings at a Glance	132
8.3	Reflections on the Methodology	133
8.4	Limitations	134

8.5	Future Work	134
8.6	A Research Roadmap	137
8.7	Broader Implications	138
8.8	Closing Remarks	139

List of Figures

List of Tables

4.1	Distribution of the 312-method corpus by category. The <i>other</i> category contains zero methods because Apache Commons Lang includes no event-handler or callback methods.	61
4.2	Manual validation of inferred specifications against documented intent, over all 312 methods.	63
4.3	OpenJML ESC discharge over all inferred clauses. <i>Discharged</i> clauses are verified against the implementation; <i>flagged</i> clauses fail discharge and are surfaced for review; <i>unsupported</i> clauses involve JML constructs the verifier cannot yet handle.	65
4.4	Comparison with Jdoctor and language-model specification inference. . . .	67
4.5	Test-generation results by class: per-method average test counts over five runs, and mutation scores, for each condition. The lower panel reports overall quality metrics as the unweighted mean across all 312 methods. . . .	68
5.1	Embedder validation on real-world libraries. Method counts, lossless-roundtrip rate, and overhead are deterministic and reported once; throughput is the median across 39 runs. The fourth-row group reports the full inferer-to-embedder pipeline on real inferred specifications.	87
5.2	Spec-shape coverage on the 2,332 Commons Lang specifications embedded through the real-inference harness (representative; the other libraries give similar distributions). Every category roundtrips losslessly.	88
5.3	Format-optimisation impact on bytecode-size overhead. The lossless-roundtrip rate is 100% under both formats.	89
6.1	Effect of compositional refinement over the isolated single-pass baseline across five libraries. <i>Refined</i> counts methods whose spec gained at least one new clause; percentages are of methods in cache.	106

6.2	Syntactic shape of the newly added preconditions across the five libraries. Branch-conditional implications dominate on four of five; disjunctive clauses are present on every library.	107
-----	---	-----

Chapter 1

Introduction

1.1 Why Formal Methods Stay Niche

Formal methods — the use of precise, machine-checkable specifications to prove that software behaves as intended — have existed for half a century, rest on solid theory, and are demonstrably effective where they are applied. And yet, by any honest measure, they remain confined to a few niche corners of software engineering: safety-critical avionics and rail, security-critical kernels and cryptographic code, the occasional high-assurance compiler. The overwhelming majority of software — the libraries, the services, the applications that make up everyday practice — is built and shipped without a formal specification in sight. This is the puzzle the thesis begins from. The technology to verify software against a specification is mature and freely available; the benefits of doing so are not in dispute; and still the practice does not spread. Why?

The thesis’s answer, which it then sets out to act on, is that the binding constraint is not the verification technology but the *specifications it requires as input*. A verifier checks code against a specification, and someone must write that specification. Writing it by hand is expensive, the expense scales with the size of the codebase, and it is paid up front before any benefit is realised — a combination that makes formal verification economically rational only where the cost of failure is catastrophic, which is precisely the niche where it is used. If the specifications could be produced automatically, the economics would change, and the question of this thesis is whether automatic inference — and the capabilities inferred specifications enable — can move formal methods out of the niche. The remainder of this section develops why the specification is the bottleneck; the sections that follow set out the two ways the thesis argues automatic inference attacks it.

1.1.1 What Code Does Versus What It Should Do

Software engineering has, since its inception, been organised around a distinction that is easy to state and hard to operationalise: the difference between what a program *does* and

what it is *intended to do*. The gap between the two is where bugs live. A test that checks only what the code does will pass on buggy code; documentation that describes only what the code does drifts the moment the code changes; a developer reading an unfamiliar library learns what its methods do by reading their bodies, but learns what they are *for* only by inference, guesswork, or asking someone else. The intended behaviour of a method is a real and consequential property, and yet it is rarely written down in a form a machine can check.

The classical answer to this problem is the *formal specification* [35, 36, 40]: a precise, machine-checkable statement of a program’s intended behaviour, expressed as preconditions (what must hold when a method is called), postconditions (what the method guarantees on return), and invariants (what remains true throughout). A specification is the externalisation of intent. Where a test encodes intent implicitly and by example, a specification encodes it explicitly and universally; where documentation states intent in prose that no tool can verify, a specification states it in logic that a verifier can discharge. The benefits of having specifications are not in dispute. They solve the test-oracle problem by supplying an independent statement of correct behaviour against which generated tests can assert. They solve the documentation-drift problem by being checkable, and therefore falsifiable, against the code they describe. They make it possible to reason modularly about a system, replacing a called method’s body with its contract. They are, in short, the substrate on which formal verification, contract-based design, specification-based testing, and a great deal of static analysis are built.

The benefits are not in dispute; the cost is. Writing specifications by hand is expensive, and the expense is not incidental but structural. In large-scale formal-verification efforts the specification burden has been measured at roughly half of total project effort [2, 33, 39]. The cost is paid up front, before any verification benefit is realised, and it scales with the size of the codebase rather than with the difficulty of the bugs one hopes to catch. Surveys of why formal methods have not achieved broad industrial adoption return repeatedly to this point [25, 52]: the technology to *check* specifications has matured to the point of practicality, but the labour to *write* them has not been removed, and so the technology sits unused. The barrier is not the verifier. The barrier is the blank page.

This thesis takes as its premise that the blank page is the problem worth attacking, and that the way to attack it is to infer specifications automatically from the code that already exists. If a static analyser can read a method body and emit a plausible, formally checkable contract — a **requires** clause for each guard, an **ensures** clause for each return path, a loop invariant for each loop — then the up-front labour cost collapses, and the question changes from “can we afford to write specifications?” to “what becomes possible once we have them for free?” That second question is the one this thesis pursues.

1.2 The Specification Gap in Practice

The abstract argument above has concrete manifestations in everyday software practice, and three of them recur throughout this thesis as motivating scenarios.

The first is the unit-testing scenario. A developer tasked with testing a method faces the oracle problem in its purest form: the test framework will run any assertion the developer writes, but it cannot tell the developer what the assertion should be. In the absence of a statement of intent, the path of least resistance is to assert what the code currently produces — to run the method, observe the output, and bake that output into the test as the expected value. The resulting test is a regression test, useful for detecting future change but useless for detecting present error, because it was written by consulting the very implementation it purports to check. When the testing is delegated to a language model, the same dynamic plays out automatically and at scale: the model, given the body, asserts what the body does. A specification breaks this loop by supplying an independent statement of intent, and the first study of this thesis measures exactly how much that matters.

The second is the integration scenario. A developer adopting a third-party library receives it as a compiled artefact — a JAR of bytecode, a remote service behind an HTTP interface — and must learn how to call it correctly. The compiled artefact carries the method signatures, which give the types, and perhaps some documentation, which gives prose; neither gives a checkable statement of the preconditions the library’s methods impose or the postconditions they guarantee. The developer discovers the contract by trial, by reading whatever source is available, or by encountering the exceptions that a violated precondition throws at runtime. A specification carried inside the artefact would let tooling — an IDE, a verifier, a contract-checking test generator — enforce the contract at the call site, but only if the specification can travel with the compiled form, which is the problem the second study addresses.

The third is the reasoning scenario, increasingly mediated by language models. A model asked to use, extend, or generate code against a library reasons about that library from whatever it can see: the signatures, the documentation, and whatever it absorbed of the library during training. For a library that postdates the model’s training cut-off, or a private library it never saw, the model has only the signatures and the documentation, and it reasons about the library’s behaviour by guesswork. A machine-readable specification, supplied as context, replaces the guesswork with a checkable contract — and a model is, as the first study shows, an able consumer of such contracts. The same machinery that helps a model write better tests can in principle help it write better client code, which is one of the broader motivations for making inferred specifications both abundant and distributable.

The fourth, and for this thesis the most consequential, is the *library-evolution* scenario.

A library is a moving target: it is released, revised, and re-released, and each new version may change the behaviour its methods promise — tightening a precondition, weakening a guarantee, altering a corner case. A consumer who upgrades a dependency wants to know whether the new version is compatible with their code, and today this question is answered crudely. Semantic versioning conventions signal intent through a version number; signature compatibility is checked by the compiler; but *behavioural* compatibility — whether the new version still honours the contracts the old one did, and whether the consumer’s code still satisfies what the new version requires — is left to tests, to release notes, and to discovery in production. A specification at the library boundary changes this. Because a verifier checks a caller against its callees’ contracts, and because a caller’s own inferred specification is *derived from* those callee contracts, a change in a library’s specifications propagates into the specifications of the code that uses it. When the library’s inferred contracts change between versions, the impact on a consumer’s contracts becomes computable, and behavioural compatibility becomes a question answerable at the specification level rather than guessed. This scenario is the one the thesis’s second and third studies build toward, and it is the clearest instance of inferred specifications enabling something that manual specification, confined to the systems that can afford it, never made routine.

These scenarios share a structure. In each, a consumer — a developer, a test generator, a model, a team upgrading a dependency — needs to know what a piece of code is intended to do, and in each the intent is present in the code’s design but absent from the form in which the consumer encounters it.

1.3 Two Routes Out of the Niche

If the manual cost of specifications is what confines formal methods to the niche, then automatic inference attacks the confinement along two routes, and the body of this thesis is organised around them.

The first route is the obvious one: *remove the cost*. If a static analyser can read a method and emit a formally checkable contract, the up-front labour that made verification economically rational only for catastrophe-cost systems collapses, and the question changes from “can we afford to write specifications?” to “what becomes possible once we have them cheaply?”. This route raises two questions the thesis must answer before the second route is even worth pursuing. The first is whether inference is *accurate* enough: a specification inferred by heuristic pattern matching is not guaranteed sound, so how well do the inferred contracts match both documented developer intent and the verdict of a formal verifier? The second is whether inferred specifications are *worth having*: granting they can be produced cheaply, do they deliver measurable value to a consumer, or is a cheap specification a worthless one? The first study (Chapters 3 and 4) answers both, using a large-language-model test generator — the most demanding contemporary consumer

of code-level intent, and one that suffers acutely from the oracle problem [34,46] — as the instrument that measures the value.

The second route is the one that makes inferred specifications more than a cheaper version of what experts already write, and it is where the thesis’s distinctive contribution lies: *make specifications a property of the library boundary, so that verification and compatibility analysis become routine across the dependencies that make up real software.* This route depends on two facts about how verification and inference compose. First, a verifier checks a caller against its callees’ *contracts*, not their bodies, so if a dependency’s contract is available a consumer can verify client code against it without the dependency’s source — which requires that the contract travel with the compiled library. Second, and more consequentially, a caller’s own inferred specification is *derived from* its callees’ specifications, so a change to a library propagates into the contracts of the code that uses it: when a library’s inferred specifications change between versions, the effect on a consumer’s contracts — and hence the behavioural compatibility of the upgrade — becomes computable. The second and third studies (Chapters 5 and 6) establish the two facts this route rests on: that inferred specifications can be carried losslessly with compiled libraries and service interfaces, and that caller specifications depend substantially and measurably on callee specifications, through exactly the cross-method obligations that a single-pass analysis cannot see and a compositional one recovers.

The two routes are not independent. The cost route produces the specifications; the boundary route makes them useful across the dependency graph that real software is built from. And the boundary route is what answers the adoption puzzle most directly: a capability that did not exist before — behavioural compatibility analysis at the library level, available without a dependency’s source — is the kind of new value that moves a technology out of its niche, because it offers practitioners something they want and could not previously have, rather than a cheaper version of something most of them never used.

1.4 Thesis Statement

The central claim of this thesis can be stated in one sentence:

The adoption of formal methods is constrained chiefly by the manual cost of the specifications they require, and that constraint is substantially removable: heuristic static inference, validated against a formal oracle, can produce Java Modeling Language specifications accurately and cheaply; those specifications can be carried with the compiled libraries and services that consume them; and because a caller’s specification is derived from its callees’, specifications at the library boundary make verification and behavioural-compatibility analysis — capabilities formerly confined to systems that could afford hand-written specifications — available across ordinary software.

The claim has two halves, corresponding to the two routes of the previous section. The first half — that inference removes the cost barrier — is the necessary groundwork: it must be true, and demonstrably so, before the second half is worth arguing. The second half — that library-boundary specifications enable verification across dependencies and, through the dependence of caller specifications on callee specifications, behavioural-compatibility analysis — is the half that bears on adoption most directly, because it offers a capability manual specification never made routine rather than a cheaper version of one it did. The thesis defends the first half with a fidelity measurement and a controlled value experiment, and it establishes the mechanisms the second half rests on — lossless distribution, and the substantial dependence of caller specifications on callee specifications — by measurement on real libraries, positioning the full compatibility application as the capability these mechanisms enable.

The claim is deliberately empirical. It does not assert that heuristic inference is sound in the sense a logician would demand; it asserts that inference plus *post-hoc* verification yields specifications whose accuracy can be measured, whose downstream value can be demonstrated, and whose cross-library propagation can be quantified. Each part is defended with quantitative evidence on real-world code rather than with appeal to construction.

1.5 Research Questions

The thesis is structured around four research questions. The first two ask whether inference removes the cost barrier; the second two establish the mechanisms by which library-boundary specifications enable verification and compatibility analysis.

RQ1 (Inference accuracy). How accurately can JML preconditions, postconditions, and loop invariants be inferred from Java source by weakest-precondition/strongest-postcondition-organised pattern matching — accurately enough to remove the manual authoring cost that is the principal barrier to adoption — measured both against human-judged developer intent and against formal discharge by an SMT-backed verifier?

RQ2 (Downstream value). Do cheaply inferred specifications deliver measurable value to a demanding consumer — specifically, supplied as context to a large-language-model test generator, do they improve the unit tests it produces relative to a signature-only baseline and a guidance-only control that isolates the contribution of specifications in particular — so that the cost reduction is worth having?

RQ3 (Library-boundary availability). Can inferred JML contracts be carried with compiled Java libraries and service-interface descriptions, reproduced losslessly by a consumer that has no access to the source, at an overhead and throughput com-

patible with routine workflows — so that a dependency’s contract is available where verification of client code against it must happen?

RQ4 (Compositional propagation). Because a caller’s specification is derived from its callees’, how substantially do library specifications propagate into the contracts of the code that uses them? Specifically, does a single-pass propagation capture this cross-method dependence, or does a compositional analysis recover a substantial and structurally characterisable body of caller obligations the single pass cannot express — the obligations through which a library change reaches its callers, and hence the basis for behavioural-compatibility analysis across versions?

1.6 Contributions

In answering these questions, the thesis makes the following contributions.

1. **JML-Inferer, a heuristic specification inference system organised by the WP/SP calculi.** The thesis presents the design and implementation of a static analyser that infers JML preconditions, postconditions, loop invariants, and frame conditions from Java source. The system’s distinguishing design choice is to use the weakest-precondition and strongest-postcondition calculi not as algorithms to execute but as a *discipline* that fixes which syntactic patterns the engine recognises and what clause each produces. Every emitted clause is discharged through OpenJML’s Extended Static Checker, recovering soundness *a posteriori* that the heuristic front end does not provide by construction. (Chapter 3.)
2. **An empirical answer, with effect size, to whether specifications improve LLM test generation.** A controlled experiment on 312 methods of Apache Commons Lang, holding the model fixed and varying only its input across four conditions, shows that specification-guided generation produces 272% more tests and a mutation score 40.7 percentage points higher than signature-only generation. A guidance-only control rules out the alternative hypothesis that any prompt enrichment, not specifications in particular, would suffice. (Chapter 4.)
3. **A toolchain-transparent mechanism for distributing inferred specifications.** An ASM-based embedder writes JML contracts into Java class-file attributes via a runtime-retained annotation, and a parallel OpenAPI extension family carries them across REST service boundaries. Evaluated on four corpora exceeding twenty thousand methods, the embedder achieves a 100% lossless-roundtrip rate at a bytecode-size overhead below 11%, with throughput high enough for per-pull-request workflows. (Chapter 5.)
4. **A measurement of how strongly caller specifications depend on callee specifications, and its consequence for compatibility.** A top-down weakest-

precondition refinement pass, measured across five open-source libraries and 21,052 methods, shows that a caller’s specification depends substantially on its callees’: it recovers 42,370 cross-method obligations that single-pass propagation cannot express, dominated by the guard-conditional and dispatch-dependent shapes through which a library change reaches the code that calls it. This dependence is the mechanism that makes specifications at the library boundary a basis for behavioural-compatibility analysis — when a library’s inferred contracts change, the impact on its callers’ contracts is computable through exactly these obligations — positioning version-level compatibility checking as the capability the mechanism enables. (Chapter 6.)

5. **An adoption argument grounded in measurement.** The thesis connects the four results into a single argument about why formal methods stay niche and what would change it: the manual cost of specifications is the binding constraint, inference removes it, and library-boundary specifications offer a capability — verification across dependencies and behavioural-compatibility analysis without a dependency’s source — that manual specification never made routine. (Chapters 1, 7, and 8.)
6. **Replication artefacts.** Each study is accompanied by a replication package containing source code, datasets, inferred specifications, generated test suites, and analysis scripts, supporting independent verification of every empirical claim.

1.7 Research Approach and Validation Strategy

The four research questions are answered by four empirical studies, and the overall research approach is worth setting out, because the studies share a methodological shape that is itself part of the thesis’s argument. Each study follows the same arc: it identifies a question that can be answered by measurement on real code, it builds or extends the software artefact needed to take the measurement, it runs the measurement on production libraries rather than constructed examples, and it reports the result with its uncertainty and its threats. The approach is empirical software engineering in the standard sense, and its commitments — controlled comparison, statistical treatment of stochastic components, explicit threats to validity, replication packages — are maintained throughout.

The validation strategy varies with the kind of claim. For the fidelity claim (RQ1), the validation is dual: a human oracle (two raters comparing inferred clauses against documented intent) and a machine oracle (a verifier discharging clauses against the implementation), reported separately because they measure different properties. For the value claim (RQ2), the validation is a controlled experiment with a baseline and a control condition, analysed statistically with effect sizes and corrected significance thresholds, because the claim is causal — that specifications *cause* better tests — and a causal claim requires a controlled comparison. For the reach claim (RQ3), the validation is a deterministic measurement of losslessness, overhead, and throughput, because the claim is about a de-

terministic mechanism whose properties can be measured exactly rather than estimated statistically. For the completeness claim (RQ4), the validation is again a deterministic measurement — counting the clauses a second pass adds — because the claim is structural, supplemented by a preliminary downstream probe that connects the structural result back to the value metric of RQ2.

The variation in validation strategy is principled: a causal claim gets a controlled experiment, a deterministic-mechanism claim gets an exact measurement, a structural claim gets a count. Matching the validation to the claim is what allows each study to make a defensible assertion of the right kind — the value study asserts a causal effect with a confidence interval, the reach study asserts an exact losslessness rate, and neither overclaims the other’s kind of certainty. This matching, and the consistent reporting of threats that bound each claim’s scope, is the methodological backbone the four studies share.

1.8 A Note on Methodology and Stance

Two methodological commitments run through the thesis and are worth stating at the outset. The first is the systematic separation of *heuristic plausibility* from *formal soundness*. The inference engine is heuristic: it matches patterns, and a pattern can fire where it should not. Rather than claim soundness the engine cannot deliver, the thesis treats inference and verification as two independent layers. Inference proposes; verification disposes. A clause that an OpenJML discharge rejects is flagged, not silently retained. This stance — infer aggressively, verify conservatively — recurs in every chapter and is, in a sense, the methodological thesis underlying the technical one.

The second commitment is to measurement on real code at scale, in preference to argument from construction. Where a claim can be made by counting — how many clauses discharge, how many tests a model generates, how many bytes an embedding costs, how many clauses a second pass adds — the thesis makes it by counting, on production libraries (Apache Commons Lang, Commons IO, Commons Math, jOOL, Vavr, Google Guava) rather than on toy examples. The cost of this commitment is that the conclusions are quantitative and corpus-bound rather than universal; the benefit is that they are defensible.

1.9 The Contributions in Relation to the Literature

The contributions can be located against the existing literature, which sharpens what each adds. The inference engine joins a tradition that includes dynamic detection (Daikon), verification-driven template inference (Houdini), documentation extraction (Jdoctor), and recent language-model approaches, and it occupies a position none of these does: purely

static, so it needs no test suite; grounded in code structure, so it needs no documentation; deterministic, so it is consistent across runs; and validated against a formal oracle, so its heuristic output is filtered for soundness. The engine does not dominate the alternatives on every axis — documentation extraction recovers exceptional behaviour the engine misses, and dynamic detection recovers numeric invariants the engine cannot see — but it occupies the static-structural-verified corner that the others leave open, and the thesis quantifies the complementarity rather than claiming dominance.

The test-generation result joins the literature on language-model test generation and the oracle problem, where prior work has documented the problem and learned oracles from corpora. The contribution is to show, with a controlled experiment and an effect size, that grounded specifications supplied as context address the oracle problem from a different direction — not by learning oracles but by inferring specifications — and that the effect is large and specific to specifications rather than to prompt enrichment in general. The guidance control is the methodological contribution: it isolates the specification as the active ingredient, which prior work supplying various context to models has not always done.

The distribution mechanism addresses a gap the inference and verification literature has largely left implicit: that a specification computed from source is unavailable to consumers who lack the source. The contribution is a toolchain-transparent mechanism for closing the gap, validated empirically, which to the author’s knowledge no prior work provides for JML at the bytecode level. The compositional refinement joins the interprocedural-analysis literature, and its contribution is a measurement — of how substantially a caller’s specification depends on its callees’ — that the literature has not previously taken on real code at this scale, together with the observation that this dependence is what makes library-boundary specifications a basis for behavioural-compatibility analysis. In each case the contribution is positioned not as a wholesale replacement of prior work but as the filling of a specific, identifiable gap, and the thesis’s value is the demonstration that the gap-fillings together remove a real barrier to the adoption of formal methods and open a capability — compatibility analysis across dependencies — that the manual-specification regime never made routine.

1.10 Scope and Delimitations

The thesis’s claims are bounded, and stating the bounds at the outset prevents the results from being read as more than they are. The work is confined to Java: the inference engine, the verifier integration, and the distribution mechanism are all Java-specific, and although the underlying ideas are not intrinsically Java-bound, the thesis does not establish that they transfer to other languages. The evaluation corpora are open-source library code — utility classes, value types, collections, numerical and functional libraries — and the thesis does not establish its claims on application code, framework code, concurrent code, or

proprietary systems, each of which has structural features the corpora lack. The language-model experiments use a single model family and defend the ordering of conditions rather than the absolute magnitudes, which the thesis does not establish are model-independent.

The thesis is also delimited in what it attempts. It does not aim to automate verification — it automates the specification step that verification requires as input, leaving the verification itself to existing tools. It does not aim to produce complete specifications — it produces the specifications that local structural inference can reach, and it measures the gap to documented intent rather than closing it. It does not aim to replace the consumers it serves — it supplies specifications to test generators and verifiers rather than competing with them. And it does not aim to settle the design question of whether a second compositional pass should always run — it measures the gap the single pass leaves and provides the information a deployer needs to decide, leaving the decision to the deployment context. These delimitations are not weaknesses to be apologised for but the boundaries that make the claims precise: the thesis establishes specific results on specific corpora with specific tools, and the value of the results is in their defensibility within those bounds, not in an overreach beyond them.

1.11 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 surveys the background and related work spanning the four research questions: the program-reasoning calculi (Hoare logic, weakest preconditions, strongest postconditions) and the Java Modeling Language that operationalises them; the landscape of specification inference, from dynamic detection (Daikon) through documentation-based extraction (Jdoctor) to recent language-model approaches; the literature on language-model test generation and the oracle problem; mutation testing as an evaluation methodology; the verification tools that serve as the formal oracle; and the channels by which specifications might be distributed.

Chapter 3 presents JML-Inferrer: its architecture, its WP/SP-organised inference of preconditions, postconditions, loop invariants, and interprocedural summaries, its OpenJML-backed validation layer, and its method categorisation scheme.

Chapter 4 reports the controlled experiment on specification-guided LLM test generation, including the manual and formal validation of the inferred specifications that the experiment depends on.

Chapter 5 develops the specification-distribution mechanism — the class-file embedding and the OpenAPI extension — and reports its empirical validation across four corpora.

Chapter 6 measures how substantially a caller’s specification depends on its callees’, via a top-down compositional analysis evaluated on five libraries, and develops the conse-

quence of that dependence — that a library change propagates into its callers’ contracts — as the basis for behavioural-compatibility analysis at the library boundary. It also reports a preliminary downstream signal from re-running the test-generation experiment with the more thoroughly propagated specifications.

Chapter 7 synthesises the four studies around the adoption question, drawing out the cross-cutting themes — the cost barrier and its removal, the library-boundary capabilities inferred specifications enable, the recurring heuristic-versus-formal tension, and the combined threats to validity — and **Chapter 8** concludes and sets out directions for future work, the foremost of which is the full version-to-version compatibility study the compositional mechanism makes possible.

The chapters are meant to be read in sequence, because the argument is cumulative: inference produces the specifications (Chapter 3), the test-generation study shows they are worth having (Chapter 4), distribution carries them to the library boundary (Chapter 5), and the compositional study shows how they propagate across that boundary and what that propagation enables (Chapter 6), before Chapter 7 draws the threads into the adoption argument. Each chapter builds on the background and methods established once in Chapters 2 and 3 rather than restating them, and each hands off explicitly to the next. What unifies them is the single question the thesis pursues throughout: why formal methods stay niche, and whether removing the specification-authoring cost and making specifications a property of the library boundary can change that.

Chapter 2

Background and Related Work

This chapter establishes the conceptual and technical foundations on which the thesis builds, and situates its contributions within the existing literature. It is organised to follow the logical dependencies of the work rather than chronology. Section 2.1 reviews the program-reasoning calculi — Hoare logic and the weakest-precondition and strongest-postcondition transformers — that supply the design discipline for the inference engine. Section 2.3 introduces the Java Modeling Language, the concrete notation in which every specification in this thesis is expressed, and the verification tools that check it. Section 2.5 surveys the landscape of specification inference. Section 2.6 reviews language-model test generation and the oracle problem that motivates the central experiment. Section 2.7 discusses mutation testing as the evaluation methodology. Section 2.10 reviews the existing channels for distributing specifications alongside compiled code. Section 2.11 reviews compositional and interprocedural analysis. Section 2.14 draws the strands together and states precisely where this thesis sits.

2.1 Program Reasoning: Hoare Logic and Predicate Transformers

The formal study of program correctness begins with Floyd’s assertions on flowcharts and Hoare’s axiomatic basis for computer programming [26]. Hoare’s central object is the triple $\{P\} S \{Q\}$, read: if the predicate P holds before the statement S executes, and S terminates, then the predicate Q holds afterward. P is the precondition, Q the postcondition, and the triple is the atom of program specification. The Hoare calculus supplies inference rules for composing triples — a rule for sequential composition, a rule for conditionals, a rule for loops governed by an invariant, and a rule of consequence that allows preconditions to be strengthened and postconditions weakened. The loop rule is the one that matters most for the inference problem this thesis attacks, because it is the rule that requires an artefact no syntactic procedure can supply for free: the loop invariant,

a predicate that holds before the loop, is preserved by each iteration, and combines with the loop’s exit condition to establish the desired postcondition.

2.1.1 The Lineage of Program Proof

The triple is the culmination of a line of thought worth tracing, because it locates the inference problem within a long-standing programme. Turing had already observed, in the earliest days of stored-program computing, that one might attach assertions to points in a program and reason about their preservation. Floyd formalised this for flowcharts, attaching predicates to the edges of a program’s control-flow graph and giving rules by which a predicate on one edge implies a predicate on the next, so that a proof of correctness becomes a consistent labelling of the graph. Hoare [26] recast Floyd’s edge-labelling into the syntactic, compositional form of the triple, replacing reasoning about a graph with reasoning about program text by structural rules — a move that made proofs follow the program’s syntax and so made them mechanisable.

The significance of this lineage for the present work is that it establishes preconditions and postconditions not as ad hoc annotations but as the natural vocabulary of program correctness, the predicates that a correctness proof must manipulate. An engine that infers preconditions and postconditions is inferring exactly the objects that fifty years of verification theory have identified as the load-bearing ones, which is why the inferred specifications connect directly to the verification machinery: they are in the language the machinery already speaks. The inference engine does not invent a new kind of specification; it recovers, by pattern matching, the same pre- and postconditions that a Floyd-Hoare proof would require as input, which is what allows OpenJML — a direct descendant of the Floyd-Hoare programme — to consume them.

2.1.2 The Weakest Precondition

Dijkstra recast Hoare’s calculus in a constructive form with the weakest-precondition (WP) predicate transformer [14, 15]. For a statement S and a postcondition Q , the predicate $WP(S, Q)$ is the weakest condition on the entry state such that every terminating execution of S establishes Q . “Weakest” is precise: $WP(S, Q)$ is implied by every other precondition that would also guarantee Q , so it characterises exactly the set of starting states from which S is guaranteed to achieve Q . The transformer is defined by structural recursion on the program: the weakest precondition of an assignment $x := e$ with respect to Q is Q with e substituted for x ; the weakest precondition of a sequence $S_1; S_2$ is $WP(S_1, WP(S_2, Q))$; the weakest precondition of a conditional distributes over its branches under the guard. The WP transformer is the engine of *backward* reasoning: it pushes a desired postcondition backward through the program text to discover what must have held at entry.

The practical significance of WP for this thesis is that it gives a precise account of

where preconditions *come from*. A method body that begins `if (x == null) throw ...` has, as the weakest precondition of its normal-return path, a condition that includes `x != null` — the throw is reachable exactly when the precondition is violated, and the weakest condition that avoids it is the negation of the guard. The guard-and-throw idiom, ubiquitous in defensive Java code, is a WP-derived precondition written in executable form. An inference engine that recognises the idiom and emits the negated guard as a **requires** clause is, in effect, computing one step of a weakest-precondition analysis by pattern recognition rather than by symbolic substitution.

2.1.3 The Transformer Rules, Concretely

The WP transformer is defined by structural recursion, and writing out the principal rules makes concrete both what a complete implementation would have to do and which steps the pattern-matching engine approximates. For an assignment, $WP(x := e, Q) = Q[e/x]$: the weakest precondition is the postcondition with the assigned expression substituted for the variable. For sequential composition, $WP(S_1; S_2, Q) = WP(S_1, WP(S_2, Q))$: one reasons backward through the second statement and then through the first. For a conditional, $WP(\text{if } C \text{ then } S_1 \text{ else } S_2, Q) = (C \Rightarrow WP(S_1, Q)) \wedge (\neg C \Rightarrow WP(S_2, Q))$: each branch must establish Q under its guard. For the abort statement that models a thrown exception, $WP(\text{abort}, Q) = \text{false}$: no precondition makes an aborting statement establish a normal-return postcondition, which is exactly why a guarded throw imposes a precondition on the caller — the weakest precondition of the path that avoids the throw is the negation of the guard.

The loop rule is where the recursion does not bottom out syntactically. For $WP(\text{while } C \text{ do } S, Q)$ one must exhibit a loop invariant I such that I holds on entry, $I \wedge C$ implies $WP(S, I)$ (the invariant is preserved), and $I \wedge \neg C$ implies Q (the invariant plus loop exit establishes the postcondition); and for total correctness one must additionally exhibit a variant that decreases on each iteration. No finite syntactic procedure produces I in general, because invariant inference is undecidable. This single gap — the loop invariant — is the reason a complete WP transformer cannot be a pure syntactic rewrite, and it is the reason Chapter 3 devotes a battery of heuristics specifically to loops while handling the other constructs by direct pattern correspondence. The transformer rules above are precisely the steps the engine emulates by pattern matching: the assignment rule underlies postcondition inference from return statements, the conditional rule underlies branch-sensitive postconditions, and the abort rule underlies guard-derived preconditions.

2.1.4 The Strongest Postcondition

The dual transformer reasons forward. For a precondition P and a statement S , the strongest postcondition $SP(S, P)$ is the strongest predicate that holds on the exit state given that P held on entry [13]. Where WP transforms a postcondition backward, SP

transforms a precondition forward, and “strongest” is again precise: SP characterises exactly the set of states reachable by executing S from a state satisfying P . For an assignment $x := e$, the strongest postcondition introduces an existential over the prior value of x , capturing the convention that a postcondition about an assigned variable refers to its new value while other mentions refer to the old. This is the formal root of the `\old` construct that appears throughout JML postconditions: a postcondition that says a counter has been incremented must refer both to the counter’s new value and to its value in the pre-state, and SP makes the distinction explicit.

For the inference engine, SP gives the account of where postconditions come from. A method whose body ends `return this.value` has, as a strongest-postcondition consequence, the clause `\result == this.value`; a method that assigns `this.count = this.count + 1` has the postcondition `this.count == \old(this.count) + 1`. Branch structure complicates the forward rule: the textbook SP of a conditional combines the two branches by disjunction, but doing so discards the link between each return value and the path condition that reaches it. The inference engine described in Chapter 3 departs from the textbook rule deliberately at this point, recording per-path guarded postconditions $C \Rightarrow \text{post}_1$ and $\neg C \Rightarrow \text{post}_2$ rather than the collapsed disjunction, because the guarded form is both more informative to a downstream consumer and more readable.

2.1.5 Why a Complete Implementation Does Not Scale

A complete WP or SP analysis of arbitrary Java is not a practical target, and the reason is worth stating because it motivates the central design decision of this thesis. Computing $\text{WP}(S, Q)$ symbolically requires a decision procedure for the assertion logic and a precise operational semantics for every language construct S might contain: integer arithmetic with overflow, heap mutation with aliasing, dynamic dispatch, exceptions, reflection. The loop rule requires invariants that no purely syntactic procedure can derive in general — invariant inference is itself undecidable. Tools that perform the full computation, such as the verification-condition generator Boogie [4] and the deductive verifiers built on it, are engineering achievements of considerable depth, but they target *verification* of human-supplied specifications and accept the attendant cost: they need the specifications, including loop invariants, written for them. An inference engine cannot assume that input, because producing it is precisely the problem.

The resolution adopted in this thesis, developed in detail in Chapter 3, is to treat the WP and SP calculi as a *design discipline* rather than as algorithms to execute. The calculi determine which abstract-syntax-tree patterns the engine looks for, what JML clause each pattern yields, and how clauses propagate across method boundaries; they do not require the engine to compute a symbolic transformer. This trades formal completeness for scalability, a trade-off with ample precedent in industrial static analysis [8], and recovers soundness after the fact by discharging each emitted clause through a verifier.

2.1.6 Two Uses of a Specification: Verifying and Testing

A specification can be put to two distinct uses, and both matter to this thesis because the inferred specifications serve both. The first use is verification: a specification is the property a verifier proves the code satisfies, the obligation the code must discharge. The second use is testing: a specification is the oracle against which generated tests assert, the statement of intent that tells a test whether the observed behaviour is correct. The two uses make different demands on a specification. Verification demands soundness above all — an unsound clause causes the verifier to prove a false property or report a spurious failure — and tolerates incompleteness, since a partial specification simply proves a partial property. Testing demands relevance and richness — a clause that distinguishes correct from incorrect behaviour — and tolerates a degree of unsoundness, since a test generated from a slightly wrong oracle merely fails and is discarded rather than corrupting a proof.

This difference in demands shapes how the thesis treats its inferred specifications for each consumer. For the verification consumer, the validation layer is essential: only discharged clauses are sound enough to verify against, and the flagged and unsupported clauses must be excluded or handled with care. For the testing consumer, the validation layer is helpful but not strictly necessary: the test-generation experiment of Chapter 4 supplies the model even the unsupported clauses, because their content is useful oracle information whether or not the verifier can discharge them, and a test generated from an imperfect clause that happens to be wrong simply does not compile or fails and is filtered. The same inferred specification thus serves two consumers with different tolerances, and the thesis’s two-layer architecture — inference plus validation — supplies what each needs: the validation gate for the consumer that requires soundness, the full inferred set for the consumer that benefits from richness and tolerates the occasional imperfection. Understanding that a specification has these two uses, with their different demands, is what justifies retaining the unsupported clauses rather than discarding them: they are useless to one consumer and useful to another, and the thesis serves both.

2.2 Abstract Interpretation and Sound Approximation

Between the exact predicate transformers of Section 2.1 and the heuristic pattern matching of this thesis lies a body of theory that explains why a deliberately approximate analysis can still be principled: abstract interpretation [13]. Abstract interpretation provides a general framework for sound, terminating approximation of a program’s concrete semantics. The concrete semantics — the exact set of states a program can reach — is in general uncomputable, so an analysis works instead in an *abstract domain*, a lattice of approximate descriptions related to the concrete states by a Galois connection. The analysis computes a fixpoint in the abstract domain, and the soundness theorem guarantees that the abstract result over-approximates the concrete one: every concrete behaviour is captured by the

abstract description, though the abstract description may admit behaviours the concrete program does not exhibit.

The framework matters to this thesis in two ways, one direct and one by contrast. The direct relevance is that any specification-inference pass can be located within it. A precondition is a description of the entry states under which a method behaves correctly; a postcondition is a description of the exit states it can reach. An analysis that infers these is computing an approximation of a fixpoint, and the single-pass interprocedural propagation of Chapter 3 is a particular, deliberately weak, approximation — one that handles unconditional callee obligations soundly but approximates the guarded and polymorphic cases by omission. The compositional refinement of Chapter 6 narrows the approximation, recovering obligations the weaker pass dropped, and it can be understood precisely as moving to a more precise point in the abstract domain — trading analysis cost for tighter descriptions, the central trade-off abstract interpretation formalises.

The relevance by contrast concerns soundness direction. Abstract interpretation is classically *over*-approximate: it never misses a concrete behaviour, at the cost of admitting spurious ones, which is the right bias for verification (a sound verifier must never miss a bug). The heuristic engine of this thesis is deliberately *not* sound in this sense: its pattern matching may both miss genuine clauses (under-approximation, costing recall) and emit clauses that do not hold (the unsoundness the validation layer catches). The engine is, in the abstract-interpretation vocabulary, neither a sound over-approximation nor a sound under-approximation but a heuristic estimate, and its principled status comes not from a soundness theorem but from the *post-hoc* verification that filters its output. This is the precise sense in which the thesis departs from the abstract-interpretation tradition: it gives up the soundness-by-construction that the framework provides, in exchange for the scalability that exact or sound-approximate analysis of arbitrary Java cannot deliver, and it recovers a weaker but sufficient guarantee — that every *retained* clause has been checked — by discharging through a verifier. Infer [8] makes a comparable trade in the memory-safety setting, and the lineage of scalable-but-unsound industrial analysis is the practical tradition this thesis joins, with the verifier-backed filtering as its distinguishing safeguard.

2.3 The Java Modeling Language and Its Verifiers

The Java Modeling Language (JML) [35, 36] is a behavioural interface specification language for Java. It expresses specifications as structured comments — `//@` line comments or `/*@ ... */` block comments — that are invisible to the Java compiler but meaningful to JML-aware tools. The core constructs are **requires** (precondition), **ensures** (postcondition), **assignable** (the frame condition, listing the locations a method may modify), **signals** (exceptional postconditions), **invariant** (class invariants that hold in every visible state), and **loop_invariant** (predicates attached to loops). Within clause expressions,

JML extends Java with specification-only constructs: `\result` for a method’s return value, `\old(e)` for the pre-state value of an expression, the quantifiers `\forall` and `\exists`, the generalised quantifiers `\sum`, `\product`, and `\num_of`, and the frame keywords `\nothing` and `\everything`.

JML descends from the Design by Contract methodology of Eiffel [40], which made pre- and postconditions first-class language constructs and articulated the discipline of treating a method’s contract as an obligation on both caller and implementer. JML adapts the methodology to Java and to a richer logic, supporting both runtime assertion checking and static verification. Its lineage as a research lingua franca is long: it has been the target notation for a generation of verification and testing tools, which is why it is the natural output format for an inference engine intended to feed those tools.

2.3.1 Semantics That Matter for Inference

Two semantic features of JML recur throughout this thesis and are worth isolating. First, reference equality versus value equality: the JML `==` operator is reference equality, exactly as in Java, so a postcondition relating two strings must use `.equals` rather than `==` to express value equality. An inference engine that emits `\result == "false"` for a method returning a string literal produces a clause that is true only by the accident of string interning and false in general; the engine must instead emit a value-equality clause. Second, the treatment of integer arithmetic: JML can be configured to interpret specification arithmetic either with Java’s bounded two’s-complement semantics or with unbounded mathematical integers. A clause such as `\result >= 0` for `x * x` is true under mathematical-integer semantics but false under two’s-complement, because the product can overflow to a negative value. An inference engine that does not gate such clauses by return type emits unsound specifications; the engine in this thesis gates them.

2.3.2 A Complete Contract, Annotated

A short example fixes the notation and shows what an inferred specification looks like in the form a verifier checks. Consider a method that returns the larger of two integers:

```
//@ requires true;
//@ ensures \result >= a && \result >= b;
//@ ensures \result == a || \result == b;
//@ assignable \nothing;
public static int max(int a, int b) {
    return (a >= b) ? a : b;
}
```

The contract has four clauses. The `requires true` states that the method imposes no precondition — it is total, accepting all inputs. The two `ensures` clauses together pin down the result: it is at least as large as both arguments, and it is equal to one of them. Neither

clause alone is a complete specification — the first is satisfied by any sufficiently large value, the second by either argument regardless of size — and it is their conjunction that captures “the maximum”. This is a recurring feature of postcondition inference: a single returned relationship is often under-determined, and a faithful specification requires several clauses that jointly constrain the result, which is part of why postcondition inference is harder than precondition inference. The `assignable \nothing` is the frame condition, stating that the method modifies no heap location; it is the default the engine emits for methods that appear pure, and its ubiquity is exploited by the distribution format of Chapter 5.

A verifier checking this contract assumes the precondition, symbolically executes the body, and attempts to discharge each postcondition. For `max` the discharge is immediate: the ternary expression yields `a` when `a >= b` and `b` otherwise, and a case split on the guard establishes both `ensures` clauses under the operational integer semantics. The example is deliberately trivial; the point is that the inferred contract is a precise, machine-checkable object, and that the inference engine’s job is to produce exactly such objects from the body alone, without the annotations being present in the source.

2.3.3 Verification Tools

The verifier used throughout this thesis as the formal oracle is OpenJML [11], which implements both runtime assertion checking and, in its Extended Static Checker (ESC) mode, deductive verification by translation to the SMT-LIB logic and discharge through an SMT solver (Z3 by default in this work). OpenJML’s ESC checks each method modularly: it assumes the method’s precondition, symbolically executes the body using the contracts of called methods in place of their bodies, and asserts the postcondition, emitting a verification condition that the solver attempts to discharge. A clause that the solver proves is *discharged*; a clause it refutes or cannot prove within a time budget is reported. This modular, contract-based checking is exactly the consumer that an inference engine should target, because it converts an inferred clause into a binary, machine-checkable verdict.

The deductive-verification tradition includes the KeY system [1], which verifies JML-annotated Java through an interactive theorem prover based on a dynamic logic for Java, and the broader family of verification-condition generators descending from ESC/Java [21] and Boogie [4]. These tools share a stance — they *consume* specifications and check them — that stands in deliberate contrast to the stance of this thesis, which *produces* the specifications they consume. The two are complementary halves of a pipeline: an inference engine that emits JML and a verifier that checks it together close the loop from unspecified code to verified contract without a human writing the contract in between.

The KeY system [1] illustrates the consumer end of this pipeline in its most demanding form. KeY verifies JML-annotated Java through an interactive theorem prover based on a dynamic logic for Java, and its interactivity is significant: for the hardest properties, a human guides the proof, supplying the insight the automated tactics lack. KeY’s power

exceeds OpenJML’s automated discharge — it can prove properties no SMT solver discharges automatically — but at the cost of the human guidance that automation avoids. The relevance to this thesis is that an inference engine could supply KeY’s input as readily as OpenJML’s: the inferred JML is in the language both consume, and a workflow that inferred contracts and verified the hard ones interactively in KeY would lower the entry cost of even the most demanding verification. The thesis uses OpenJML’s automated checker as its oracle because automation is what scales to twenty thousand methods, but the inferred specifications are not tied to that choice; they are JML, and any JML verifier — automated or interactive — can consume them. This independence of the inference from the particular verifier is part of why the thesis frames its contribution as supplying the specifications the verification ecosystem requires, rather than as a contribution to any one verifier.

2.4 Design by Contract and the Idea of an Obligation

The notation of Section 2.3 carries a methodology, and the methodology is worth stating because it shapes what a specification is *for*. Design by Contract, introduced with the Eiffel language [40], frames a method’s specification as a contract between two parties: the caller and the implementer. The precondition is the caller’s obligation — the caller must establish it before calling — and in return the implementer guarantees the postcondition. The metaphor of mutual obligation has a precise consequence: a precondition is not a check the method performs but a promise the caller makes, and a method is entitled to assume its precondition holds rather than to defend against its violation. This is why a guard-and-throw idiom is a degenerate contract: a method that throws on null input is defending against a precondition violation, and the inferred **requires** clause makes explicit the obligation the defensive check implies.

The contract metaphor also explains the asymmetry between precondition and postcondition strength that recurs in the inference results. Strengthening a precondition narrows the contract — it demands more of the caller — while strengthening a postcondition widens it, promising more to the caller. An inference engine biased toward precision should therefore be cautious about preconditions, since an over-strong precondition wrongly forbids valid calls, and cautious in a different direction about postconditions, since an over-strong postcondition wrongly claims behaviour the method does not guarantee. The engine’s behaviour reflects this: it emits a precondition only from an explicit aborting guard, the strongest evidence that the caller really is obligated, and it emits postconditions from observed return and assignment structure, the strongest evidence of what the method really guarantees. The contract framing, inherited from Eiffel and carried into JML, is thus not merely notation but the conceptual basis for the engine’s conservative inference policy.

The methodology’s relevance extends to the distribution problem of Chapter 5. A contract is an interface property — it describes the boundary between caller and implementer

— and an interface property naturally belongs with the artefact that exposes the interface, not with the source that implements it. The intuition that a contract should travel with the compiled library, recoverable by any caller, is a direct consequence of the Design by Contract view that the contract is the caller’s business as much as the implementer’s. Embedding inferred contracts in bytecode is, in this light, the natural completion of the methodology: it puts the caller’s half of the contract where the caller can see it.

2.5 Specification Inference

Automatically recovering specifications from code is a well-established problem with several distinct methodological traditions. This section reviews them in the order of their relationship to the present work.

2.5.1 Dynamic Invariant Detection

Daikon [19] pioneered the dynamic detection of likely program invariants. Daikon instruments a program, runs its test suite, records the values of variables at procedure entry and exit, and reports the relationships — equalities, inequalities, linear relationships, containment properties — that held over every observed execution. The inferred invariants are “likely” rather than guaranteed: they are generalisations from observed traces, and their quality is bounded by the coverage of the test suite that drives them. An invariant Daikon reports may be an artefact of an incomplete suite, true of every test but false in general; conversely, a genuine invariant exercised by no test will not be reported. Daikon’s strengths and limitations are both consequences of its dynamism: it can discover numeric relationships that no syntactic analysis would guess, but it requires an executable program with a test suite, and it offers no soundness guarantee. The contrast with the present work is sharp on both axes. The engine in this thesis is purely static — it requires no test suite and no execution — and it pursues soundness through *post-hoc* verification rather than abandoning it; the cost is that heuristic pattern matching will miss the subtle numeric relationships that dynamic observation surfaces.

2.5.2 Why Static and Dynamic Inference Find Different Things

The contrast between Daikon’s dynamic detection and the static inference of this thesis is not merely methodological; the two approaches find systematically different specifications, and understanding why clarifies what each is for. Dynamic detection generalises from observed values: it reports a relationship if it held on every execution the test suite drove. It therefore excels at numeric and relational invariants that are true but not syntactically apparent — that one field is always twice another, that a list is always sorted, that a return value always falls in an observed range — because these emerge from the values regardless of how the code is written. Its blind spot is the relationship that no test exercised:

a precondition guarding a code path the suite never took will not appear, because no observation refutes its absence.

Static structural inference has the complementary profile. It excels at specifications that are apparent in the code’s structure — the guard that yields a precondition, the return that yields a postcondition — regardless of whether any test exercises them, because it reads the code rather than its executions. Its blind spot is the relationship that holds semantically but is not structurally apparent: the numeric invariant that emerges only from the values, the algorithmic postcondition that characterises what a loop computes rather than how. The two approaches are thus not competitors on a single axis but explorers of different regions of the specification space, and the recall figures of Chapter 4 — high for guard-derived preconditions, lower for complex postconditions — trace exactly the boundary of the static region. A hybrid that ran both and took the union would cover more than either, which is a direction the thesis notes among its future work; the present engine is purely static because the static region contains the guard-and-return specifications that are both the most common and the most reliably inferred, and because static inference requires neither a test suite nor an execution, which broadens its applicability to any compilable code.

2.5.3 Verification-Driven Annotation Inference

Houdini [20] occupies a position between inference and verification. It conjectures a large set of candidate annotations from a fixed template vocabulary — for each field, candidate clauses such as `f != null`, `f >= 0`, and so on — and then repeatedly invokes ESC/Java to prune the candidates that cannot be verified, iterating until a mutually consistent set remains. Houdini is, in spirit, the closest precursor to the infer-then-verify architecture of this thesis: it too treats inference as proposal and verification as disposal. The difference is in the proposal step. Houdini guesses from templates and relies entirely on the verifier to filter; the present work replaces the template guess with WP/SP-organised structural pattern matching grounded in the code’s actual control and data flow, and extends the verification oracle from ESC/Java to OpenJML with quantifier support. The structural grounding raises the precision of the proposals, so the verifier filters less and the surviving set is larger.

The cost structure of the two approaches differs in a way that matters at scale. Houdini’s reliance on the verifier to filter means it pays a verification cost proportional to the number of candidate annotations it conjectures, and its template vocabulary must be kept small to bound that cost — a larger vocabulary conjectures more candidates, each requiring verifier calls to prune. The structural engine of this thesis inverts this: its pattern matching proposes a smaller, more precise set of candidates, so the verification cost is proportional to the emitted clauses rather than to a template-vocabulary-sized candidate space, and the verifier serves as a soundness filter on a population that is already mostly

sound rather than as the primary pruning mechanism on a population that is mostly noise. The two architectures share the propose-then-verify shape but differ in where the intelligence sits: in Houdini it is in the verifier doing the pruning, in the present work it is in the pattern matcher doing the proposing, with the verifier demoted from primary filter to safety net. The demotion is what makes the approach scale to twenty thousand methods, because the verifier is the expensive component and the structural engine asks less of it.

2.5.4 Documentation-Based Extraction

A complementary tradition extracts specifications from natural-language documentation. Toradocu [24] and its successor Jdoctor [5] apply natural-language processing to Javadoc comments to recover exception preconditions — the conditions under which a method documents that it throws. Pandita et al. [43] infer resource specifications from API documentation in a similar spirit. These approaches are effective exactly where documentation is thorough and silent where it is absent, which makes them complementary to code-structural inference: a method with a documented `@throws` clause but no defensive guard yields a specification to Jdoctor and nothing to a structural engine, while a method with a defensive guard but no documentation yields the reverse. Chapter 4 quantifies this complementarity directly, reporting the clauses each approach captures that the other misses.

The mechanism of documentation extraction is worth understanding because its strengths and failure modes follow from it. Jdoctor parses a Javadoc comment’s prose, identifies the noun phrases and conditions it describes, and matches them against the method’s parameters and types to construct a formal condition — a documented “throws `IllegalArgumentException` if `x` is negative” becomes the precondition `x >= 0` guarding the normal path. The matching is the hard part, because natural language is ambiguous and Javadoc prose is irregular, and the approach’s precision depends on the documentation being both present and phrased in patterns the parser recognises. Where documentation is absent the approach yields nothing, which bounds its coverage to documented methods; where documentation is present but idiosyncratically phrased, the parser may extract the wrong condition, which bounds its precision. The structural engine of this thesis has the opposite profile — it ignores documentation entirely and reads the code — so the two fail on disjoint inputs, the documentation extractor on undocumented methods and the structural engine on behaviours not visible in the code’s structure, which is the basis for the complementarity the later chapter measures.

PreInfer [3] occupies a middle ground between dynamic and documentation-based inference, using symbolic analysis of the path conditions collected from failing tests to infer preconditions for C#. It requires an execution infrastructure, like dynamic detection, but reasons symbolically about the paths rather than generalising from concrete values, which lets it infer preconditions that distinguish failing from passing inputs. Its dependence on

an execution infrastructure and a corpus of failing tests is the requirement the purely static engine of this thesis avoids, at the cost of the path-sensitive precision PreInfer’s symbolic analysis achieves; the trade is the recurring one between the reach of dynamic or symbolic execution and the universality of static structural inference.

2.5.5 Language-Model-Based Inference

The most recent tradition prompts a large language model to emit specifications directly. SpecGen [38] and the approach of Endres et al. [18] prompt a model to produce formal contracts from code; Richter and Wehrheim [45] extend this to full precondition/postcondition pairs and report the false-alarm rates a verifier produces on the results; Teuber and Becker [47], closest in target language to the present work, generate JML for Java deductive verification. The language-model approach has the appeal of requiring no hand-built pattern repertoire and the liability of being ungrounded: the model may hallucinate clauses that are plausible in surface form but false against the code, and its outputs vary across runs at nonzero temperature. Chapter 4 reports a direct comparison, finding that language-model inference attains recall comparable to the structural engine but substantially lower precision and far lower run-to-run consistency, because its proposals are not anchored in concrete code structure. The finding motivates a methodological caution that the thesis maintains throughout: a language model is an excellent *consumer* of grounded specifications (Chapter 4) and a less reliable *producer* of them.

The mechanisms these systems use illuminate why. SpecGen and the approach of Endres et al. prompt a model with a method and ask it to emit a contract, sometimes iterating with verifier feedback to repair clauses that fail. The iteration helps — it filters some hallucinations — but it is bounded by the model’s ability to repair toward a correct clause rather than toward a different plausible one, and by the verifier’s ability to discharge the candidate at all. Richter and Wehrheim’s emphasis on verifier false-alarm rates is telling: a model-produced specification that does not match the implementation causes the verifier to report a spurious failure, and the rate of such false alarms is a direct measure of the model’s ungroundedness. The structural engine avoids the false-alarm problem at its source, because its clauses are read from the code’s structure rather than predicted, so a clause it emits about a guard is a clause the guard actually imposes. The contrast is not that models cannot produce useful specifications — the iterative approaches show they can produce some — but that the rate of ungrounded clauses they produce is high enough to require verifier filtering as a primary mechanism rather than a safety net, which is the same cost structure that distinguishes Houdini from the present engine, here arising from the model’s prediction rather than from a template vocabulary.

2.5.6 A Framework for Comparing Inference Approaches

The four traditions just reviewed — dynamic, verification-driven, documentation-based, and learned — can be organised along three axes that together locate the present work and explain its complementarities. The first axis is the *source of evidence*: dynamic detection draws on execution traces, documentation extraction on natural-language text, language-model inference on a learned prior over code, and the present engine on the code’s syntactic structure. The second axis is the *soundness guarantee*: dynamic invariants are likely but not guaranteed, language-model clauses are unguaranteed, documentation clauses are as reliable as the documentation, and the present engine’s clauses are heuristic but filtered through a verifier. The third axis is the *input requirement*: dynamic detection needs a test suite and an executable program, documentation extraction needs documentation, language-model inference needs a model and tolerates none of the determinism the others offer, and the present engine needs only compilable source.

Placed in this framework, the present engine occupies the cell defined by structural evidence, verifier-filtered soundness, and a minimal input requirement of compilable source alone. No other tradition occupies it: dynamic detection trades the test-suite requirement for the ability to find non-structural invariants, documentation extraction trades the documentation requirement for access to intent the code does not express, and language-model inference trades determinism and precision for freedom from a hand-built pattern repertoire. The framework makes the complementarities precise — each tradition’s strength is another’s blind spot, located on a different axis — and it explains why the thesis frames its comparisons as complementary rather than competitive: the engine dominates no tradition on every axis, but it is the only one in its cell, and its cell is the one that requires least of the user (no tests, no documentation) while offering most by way of soundness (verifier filtering) on the evidence most reliably available (the code itself).

2.6 Language-Model Test Generation and the Oracle Problem

Large language models have rapidly become a practical means of generating unit tests [9, 10, 37, 46, 50], and a substantial literature has accumulated, recently surveyed by Zhang et al. [51] across 115 publications. Given a method signature a modern model produces compilable, plausibly named tests in seconds; given the method body it produces tests that exercise the visible control-flow paths. Foundational contributions include AthenaTest [48], which adapted transformer architectures to test generation with focal context; CodaMOSA [37], which combined search-based generation with model sampling to escape coverage plateaus; ChatTESTER [50]; CodeT [9]; and the empirical evaluation of Schäfer et al. [46]. Kang, Yoon, and Yoo [31] showed that models can be effective few-shot testers when prompted with bug reports.

2.6.1 The Test Oracle, Defined

The oracle problem has a precise statement that clarifies why specifications address it. A test oracle is a procedure that decides whether a program’s behaviour on a given input is correct. Every test has one, if only implicitly: the assertion at the end of a test is the oracle, deciding correctness by checking the observed output against an expected one. The oracle problem is the difficulty of obtaining the expected output — of knowing, for a given input, what correct behaviour is. The problem is fundamental because correctness is defined relative to intent, and intent is not present in the implementation; an implementation defines what the program *does*, and an oracle requires what it *should* do, which is a separate piece of information.

There are several sources from which an oracle can be drawn, and they form a hierarchy of strength. The strongest is a formal specification, which states the intended behaviour directly and against which any output can be checked. Weaker is a reference implementation, which gives correct outputs but only for inputs one can run it on, and only if it is itself correct. Weaker still is a regression oracle, which asserts that behaviour does not change — useful for detecting regressions but useless for detecting present bugs, since it takes the current behaviour as correct by definition. Weakest is an implicit oracle, which checks only that the program does not crash or violate a language-level invariant — the “does not throw” assertion that a model falls back to when it has nothing better. The hierarchy explains the thesis’s central hypothesis precisely: a specification sits at the top of the hierarchy, and supplying one to a test generator moves its oracles up from the regression or implicit level, where a model without a specification is stranded, to the specification level, where genuine bugs become detectable. The mutation-score metric measures exactly this movement, because a mutant is a seeded bug and only a specification-level oracle reliably catches it.

2.6.2 The Oracle Problem

The persistent and well-documented weakness of language-model test generation is the oracle problem [30, 34, 46]. A unit test has two parts: a stimulus that drives the code into some state, and an oracle that asserts the state is correct. A model can synthesise stimuli readily, but the oracle requires a statement of *intended* behaviour that the code itself does not contain. Lacking such a statement, the model infers the oracle from the implementation — and so asserts what the code does rather than what it should do. Konstantinou et al. [34] demonstrate this empirically: model-generated oracles encode the program’s actual behaviour, faithfully reproducing its bugs as expected results, and when given only a signature the model falls back to vacuous assertions such as “does not throw”. Molinelli et al. [41] evaluate 13,866 model-generated oracles on 135 Java projects and report a mean mutation score of 43% against 45% for human-written oracles, a calibration point for the magnitude of the problem.

A line of work attacks the oracle problem by learning oracles from corpora. Dinella et al. [16] trained a neural model jointly to produce exceptional and assertion oracles; Hossain and Dwyer [27] showed that large general-purpose models produce stronger and more diverse oracles than the original neural baseline, detecting an order of magnitude more bugs. Foster et al. [22] report industrial deployment of mutation-guided model test generation at Meta. The approach of this thesis is orthogonal and complementary: rather than learning oracles from a corpus of tests, it supplies the model with an externally grounded specification derived from program structure. The two could in principle be ensembled — a learned oracle and an inferred specification are independent sources of intended behaviour — but the experiment in Chapter 4 isolates the contribution of the inferred specification alone.

2.6.3 Why Specifications Should Help

The hypothesis that inferred specifications improve model test generation rests on an information argument. A method signature carries types but no behaviour. A method body carries behaviour but no statement of intent — and, crucially, it carries the test answer directly, which is why supplying the body is an upper bound rather than a realistic deployment condition. A specification occupies the gap: it carries behavioural intent that the signature lacks, in a form an order of magnitude more compact than the body, and it carries that intent as discrete clauses a model can target one at a time. The experiment in Chapter 4 tests this hypothesis with a four-condition design — signature only, signature with edge-case guidance, signature with specification, full source — in which the guidance condition is included precisely to rule out the alternative explanation that any prompt enrichment, not specifications in particular, would account for the effect.

2.6.4 The Broader Landscape of Learned Code Models

The language-model test generators reviewed above sit within a wider movement to apply learned models to program analysis tasks, and locating the thesis within it clarifies which parts of that movement it embraces and which it holds at arm’s length. Models trained on code have been applied to code completion, program repair, type inference, and code summarisation, and the survey of Jiang et al. [30] maps the breadth of code generation alone. The common thread is that a model learns statistical regularities of code from a large corpus and applies them to a task by prediction. For tasks where a plausible prediction is a useful output — completing a line, suggesting a fix for review — the approach is transformative, because plausibility is most of what is wanted.

For tasks where the output must be *correct* rather than merely plausible, the picture is more nuanced, and specification inference is squarely such a task. A specification that is plausible but false is worse than no specification, because a downstream verifier or test generator will trust it and be misled. This is the basis for the thesis’s division of labour:

it uses a learned model where plausibility suffices (generating tests, which a test runner then checks) and a grounded static analysis where correctness is required (producing the specifications that the model and the verifier consume). The division is not a claim that learned models cannot produce correct specifications — the specification-inference work of SpecGen and its successors [18, 38, 45, 47] shows they can produce some — but a claim that, on the precision and consistency dimensions a downstream verifier cares about, a grounded analysis is the more reliable producer, as the comparison in Chapter 4 quantifies.

The neural oracle-generation line — TOGA [16] and TOGLL [27] — occupies an interesting middle position. It learns oracles, which are a form of specification, from a corpus of tests rather than from code structure, and it succeeds because an oracle need only be a useful discriminator, not a verified contract. The contrast with the present work is the source of grounding: the neural oracle is grounded in a corpus of human-written tests, while the inferred specification is grounded in the code’s own structure. The two groundings are complementary — they fail on different inputs, the corpus-grounded oracle on code unlike its training set and the structure-grounded specification on code whose intent is not visible in its structure — and an ensemble of the two is a natural direction the thesis notes but does not pursue.

2.7 Mutation Testing as Evaluation Methodology

Measuring whether one set of tests is *better* than another requires a metric more discriminating than code coverage, because coverage rewards tests that execute code without checking its behaviour. Mutation testing [44] supplies the discriminating metric. A mutation tool systematically introduces small faults — a flipped conditional, a swapped arithmetic operator, a replaced return value — into the program under test, producing a population of *mutants*, each differing from the original by one seeded fault. A test suite is run against each mutant; a mutant that causes some test to fail is *killed*, and a mutant that no test detects *survives*. The mutation score, the fraction of mutants killed, measures the suite’s fault-detection power directly, and it penalises exactly the weakness of oracle-free tests: a test that executes code without asserting its behaviour will cover the mutated line but fail to kill the mutant. Wang et al. [49] make the point quantitatively, reporting that 100% statement coverage can coexist with a mutation score as low as 4%.

The tool used throughout this thesis is PIT [12], a mature mutation-testing system for the JVM that operates on bytecode and is widely adopted in both research and industry. Operating on bytecode rather than source gives PIT two advantages relevant here: it can mutate code for which source is unavailable, and it integrates with the standard JVM build without a separate compilation of mutated sources, which matters for the throughput of an experiment that mutates hundreds of methods. PIT’s default operator set is the field’s de facto standard, which makes results reported against it comparable across studies, and its automatic equivalent-mutant filtering reduces the most pernicious source of mutation-

score distortion without manual intervention.

Recent work has begun to study language models and mutation testing in combination, which situates this thesis’s use of the methodology. Wang et al. present an empirical study of models for mutation testing across hundreds of Java bugs, and a related line develops mutation-guided model test generation, reporting the disconnect between coverage and mutation score that motivates using the latter as the quality metric. Foster et al.’s industrial deployment of mutation-guided model test generation at scale demonstrates that the methodology is not merely an academic instrument but one that informs production test generation. The thesis adopts mutation score for the same reason these efforts do: it is the available metric that measures fault detection rather than mere execution, and it is therefore the metric that can distinguish the oracle-bearing tests specifications produce from the oracle-free tests a signature alone yields. Mutation testing has known limitations, principally the equivalent-mutant problem — a seeded mutant that is semantically identical to the original cannot be killed by any test and depresses the score spuriously — and the question of whether the operator set is representative of real faults. The experiments in this thesis mitigate the first with PIT’s automatic equivalent-mutant filtering supplemented by a sampling estimate of the residual rate, and address the second by using PIT’s default operator set, which is the field’s de facto standard. Test count is reported alongside mutation score as a complementary metric, and the relationship between the two — specifically, that more tests do not imply a higher mutation score — is itself one of the experimental findings.

2.8 SMT Solvers and the Limits of Automated Discharge

The verifier that serves as this thesis’s formal oracle does not prove verification conditions itself; it translates them to the logic of a satisfiability-modulo-theories (SMT) solver and asks the solver to discharge them. Understanding what the solver can and cannot do is essential to interpreting the discharge results of Chapter 4, because the clauses that fail to discharge fail not because they are false but because they exceed the solver’s reach.

An SMT solver decides the satisfiability of formulas in a combination of first-order theories — linear integer and real arithmetic, the theory of arrays, the theory of uninterpreted functions, bitvectors, and others — by combining a propositional satisfiability engine with decision procedures for each theory. The default solver in this work is Z3, a mature and widely used solver; the validation pipeline also bundles alternative versions and an alternative solver for cross-checking. The solver’s power is real but bounded, and the bounds fall in predictable places. Linear arithmetic is decidable and the solver handles it well; nonlinear integer arithmetic — multiplication of variables — is undecidable in general, and the solver’s heuristics for it are incomplete, which is the root of the recursive-multiplicative-growth limit that Chapter 4 documents. The theory of arrays supports reading and writing individual elements, but reasoning about a universally

quantified property preserved across an array update strains the solver, which is the root of the quantified-invariant-over-arrays limit. Quantifier instantiation, the mechanism by which the solver reasons about `\forall` and `\exists`, is heuristic and can fail to find the instantiation a proof needs, or can diverge searching for it.

These are not defects of the particular solver but properties of the underlying decision problems, and they bound what any verifier built on SMT can certify. The thesis’s response is twofold. First, it tunes the strictness configuration (Chapter 3) to the point that maximises genuine discharge without provoking timeouts, an empirical optimum that stricter settings overshoot. Second, it extends the verifier with recursive function definitions for the generalised quantifiers (Chapter 3), giving the solver a handle on a class of clauses it would otherwise refuse. Even so, a residual remains, and the thesis classifies it precisely rather than reporting an undifferentiated failure rate, because the classification distinguishes clauses that are false (which would indicate an inference bug) from clauses that are true but unprovable-at-budget (which indicate a solver limit). The distinction is what allows the discharge rate to be read as a measure of the solver’s reach rather than of the engine’s correctness.

2.9 Runtime Verification and the Spectrum of Checking

Static discharge is one end of a spectrum of ways to check a specification; the other is runtime verification, in which the specification is compiled into checks that execute alongside the program and signal a violation when one occurs. JML supports both: the same contract can be discharged statically by the extended checker or compiled into runtime assertions by the runtime-assertion-checking compiler. The two have complementary strengths. Static checking proves the specification holds on all executions but is bounded by the solver’s reach; runtime checking observes the specification on the executions that actually occur, catching violations the static checker could not prove absent but saying nothing about executions that did not happen.

The spectrum matters to the thesis in two ways. First, it clarifies that the unsupported clauses of Chapter 4 — those the static checker cannot discharge — are not worthless: a clause that cannot be proved statically can still be checked at runtime, and can still serve as oracle information to a test generator, which is why the thesis retains them rather than discarding them. Second, it bears on the distribution work of Chapter 5: the runtime-assertion channel reviewed in Section 2.10 is the runtime-verification end of the spectrum applied to distribution, and its limitation — that it carries only executable, behaviour-altering checks — is what motivates the structured, behaviour-preserving embedding the thesis develops instead. The embedded specification can feed either end of the spectrum: a consumer can discharge it statically or compile it to runtime checks, because it is carried as data rather than as either kind of check, which is a flexibility the runtime-assertion channel forecloses.

2.10 Distributing Specifications Alongside Compiled Code

A specification inferred from source is computed on a machine that has the source; the consumers who would benefit most often do not. Chapter 5 addresses this, and the relevant background is the set of existing channels for shipping specifications with compiled Java, each of which leaves a gap.

JML’s native channel is the *specification file* (`.jml` or `.spec` stub), a source-like file carrying specifications separately from the implementation. OpenJML reads these natively, which makes them the right format for a verifier, but they are a sidecar: they travel as separate files, are not present in the compiled JAR, and are invisible to any tool that does not specifically look for them. A second channel is *runtime assertion checking*, in which specifications are compiled into executable checks embedded in the bytecode; this carries the specification into the artefact but only in executable form, restricted to the properties that can be evaluated at runtime, and at the cost of altering the program’s behaviour. A third channel, for service interfaces, is the documentation field of an interface description such as OpenAPI, which carries prose a human can read but no structured contract a tool can consume.

The gap common to these channels is that none carries a *structured, toolchain-transparent* specification inside the standard compiled artefact — one that any class-file reader can recover without the source, that does not alter program behaviour, and that survives the ordinary build and distribution pipeline unmodified. Chapter 5 fills this gap with a runtime-retained Java annotation written into class-file attributes, and a parallel structured extension to OpenAPI for service boundaries. The relevant enabling technology is the ASM bytecode-engineering library, which permits class-file attributes to be added to an existing JAR in place, and the Java class-file format’s provision for `RuntimeVisibleAnnotations`, which makes an embedded annotation recoverable by reflection.

2.11 Compositional and Interprocedural Analysis

The fourth research question concerns the completeness of cross-method inference, and its background is the theory and practice of interprocedural analysis. The foundational framework is abstract interpretation [13], which provides the general account of a sound, terminating approximation of a program’s concrete semantics, within which any specification-inference pass can be understood as computing an approximation of a fixpoint. The single-pass interprocedural propagation used by the inference engine — analyse callees first, cache their contracts, substitute them at call sites — is the standard procedure-summary mechanism for modular analysis, and it is sound for the case it handles: a precondition a callee requires unconditionally must hold at the call site.

The single pass is, however, structurally blind to two classes of cross-method obligation,

and recognising this is the contribution of Chapter 6. The first is the *branch-conditional* obligation: a call reached only on one branch of a conditional contributes its precondition only on that path, so the caller’s obligation is $guard \Rightarrow precondition$ rather than the precondition unconditionally. Recovering this requires the caller’s control-flow context, which a single substitution at the call site does not have. The second is the *polymorphic-dispatch* obligation: a call through an interface or overridable method may dispatch to any of several implementations at runtime, so the sound caller precondition is the disjunction of the candidates’ preconditions, which a single nominal-callee resolution cannot produce. Recovering both requires walking the caller’s body in a weakest-precondition fashion, accumulating guard context and enumerating dispatch candidates — the second pass that Chapter 6 implements and measures.

The compositional-analysis literature offers points of comparison. Calcagno et al.’s Infer [8] computes genuinely compositional procedure summaries by bi-abduction in separation logic; it is compositional from the outset and needs no second pass, but it targets memory-safety properties rather than JML contracts and is not designed for downstream JML verification. The contrast clarifies the niche of the present work: a pass that *produces* JML, reasons compositionally by weakest-precondition lifting, and feeds an existing JML verifier, occupies a cell that the genuinely-compositional memory-safety analysers and the JML-consuming verifiers between them leave empty.

The cell can be located precisely along three orthogonal axes that recur when the compositional propagation of Chapter 6 is positioned against its neighbours: whether a technique *produces* JML, whether it *reasons compositionally* across procedure boundaries, and whether its cross-procedure reasoning is *checked against a solver*. The deductive verifiers KeY and OpenJML’s static checker reason compositionally and discharge against a solver but do not produce specifications — they consume hand-written ones. Daikon, Houdini, and the language-model inference approaches produce candidate specifications but do not lift a callee’s contract through the caller’s guards or emit a disjunction over dynamic-dispatch candidates. Infer reasons genuinely compositionally, by bi-abduction in separation logic, but targets memory-safety properties rather than functional JML contracts and is not designed for downstream JML verification. The verification-condition generators in the Boogie lineage perform the full weakest-precondition computation that a compositional inference pass only approximates, but they *consume* a specification — including loop invariants — rather than producing one. Houdini and the template-and-prune lineage are closest in spirit to the infer-then-verify architecture this thesis adopts, but they conjecture annotations per program element rather than lifting a callee’s contract into a caller, so they lack the cross-method propagation that is the subject of Chapter 6. The position the thesis’s compositional propagation occupies — producing JML, reasoning compositionally by weakest-precondition lifting, and feeding an existing JML verifier — is, to the best of the author’s knowledge, unoccupied by prior work; the novelty is the combination, not any single axis.

2.12 Specification-Based Testing and Test Generation

The consumers that an inference engine ultimately serves include the established family of automated test-generation tools, and understanding their relationship to specifications clarifies the role of inferred contracts. The search-based tool EvoSuite [23] generates JUnit test suites by evolutionary search over a coverage objective, producing suites that attain high structural coverage and, by default, regression oracles — that is, oracles that assert whatever the current implementation does, which detect future changes but not present bugs. The random tool Randoop [42] builds tests by feedback-directed random sequence generation, again strong on coverage and weak on oracles unless supplied with contracts. Korat [6] generates structurally complex test inputs from a specification of their validity, and is in this sense already a specification consumer. The common thread is that these tools solve the *stimulus* half of the testing problem — constructing inputs that exercise the code — comprehensively, while the *oracle* half remains dependent on a statement of intended behaviour that the tools themselves do not supply.

Design by Contract [40] and JML [35,36] address the oracle half directly, by embedding expected behaviour in specifications that a tool can turn into runtime checks or verification conditions. The JML-aware testing tools — JMLUnit and the contract-aware modes of the search-based and random generators — *consume* these specifications to produce tests with genuine oracles. The structural relationship that matters for this thesis is the direction of the dependency: every one of these tools requires specifications as input, and the manual cost of supplying that input is the same bottleneck the introduction identified. JML-Inferer supplies the upstream piece the established tools have always assumed, which is why the test-generation experiment of Chapter 4 is framed as supplying inferred specifications to a consumer rather than as competing with the consumer.

The contract-aware modes of these generators are the most direct illustration of the consumer relationship. EvoSuite, given a set of JML postconditions, can use them as assertion oracles rather than defaulting to regression oracles, turning a suite that detects change into one that detects fault; Randoop’s contract-checking mode filters generated sequences against supplied contracts. In both cases the specification converts a coverage-oriented generator into a fault-oriented one, and in both cases the specification is an input the tool does not produce. This is the upstream-downstream relationship the thesis’s first study exploits with a language model in place of a search-based or random generator: the generator is fluent at producing test stimuli and dependent on an external oracle, and the inferred specification supplies the oracle. The language model is, in this framing, simply the most capable contemporary instance of an oracle-dependent generator, and the finding that specifications help it generalises the long-standing observation that specifications help any oracle-dependent generator.

Industrial static analysers occupy a neighbouring niche. Facebook Infer [8] and Find-Bugs [28] demonstrate that static analysis scales to industrial codebases, but their objective

is bug detection rather than specification generation: they report suspected defects, not contracts that a verifier could discharge or a test generator could consume. The scalability lessons transfer — both tools establish that heuristic, deliberately unsound analysis can run on millions of lines and deliver value — but the output is a different artefact, and the inference problem this thesis attacks is the production of reusable contracts rather than the flagging of individual defects.

2.13 The Cost of Specification and the Adoption Barrier

The premise of the thesis — that the manual cost of writing specifications is the binding constraint on the adoption of formal methods — rests on a body of empirical evidence worth reviewing, because it is the evidence that makes inference worth attempting at all. The large-scale verification of the seL4 microkernel [2, 33] is the canonical data point: a fully verified operating-system kernel, in which the proof effort vastly exceeded the implementation effort, and in which the writing and maintenance of specifications and proof obligations dominated the cost. Matichuk et al. [39] quantified the relationship between proof size and effort in that project, establishing that the cost scales with the volume of specification rather than with the difficulty of the code, which is precisely the scaling behaviour that makes manual specification prohibitive for large codebases.

The adoption literature reaches the same conclusion from the opposite direction. Hall’s account of the seven myths of formal methods [25] catalogued the misconceptions that impeded adoption — that formal methods guarantee perfect software, that they require difficult mathematics, that they are only for safety-critical systems — and argued that the real obstacles were practical rather than conceptual. Among the practical obstacles, the labour of writing and maintaining specifications has proved the most durable: the misconceptions Hall identified have been answered by decades of tooling and experience reports, but the specification-writing cost has not, because no amount of better tooling for *checking* specifications reduces the cost of *producing* them. The subsequent decades of experience reports return repeatedly to this cost as the obstacle that keeps formal methods on the margins of practice, even as the checking technology matures. Zimmermann and Wehrheim [52] frame the problem as one of lightweight adoption: developers will use specifications if the cost of producing them is low enough to fit a normal development workflow, and will not if it is not. The inference approach of this thesis is a direct attack on that cost, and its viability is the reason the downstream-value question of Chapter 4 matters — if inferred specifications were cheap but useless, the cost reduction would be pointless; the thesis argues they are both cheap and useful.

2.14 Positioning of This Thesis

The strands above converge on a single observation. The technology to *check* specifications is mature (OpenJML, KeY, Boogie); the technology to generate tests, documentation, and other artefacts from specifications is mature (EvoSuite [23], Randoop [42], and the JML-aware tooling ecosystem); and the technology to *consume* specifications via language models is rapidly maturing. The binding constraint, across all of these, is the supply of specifications themselves, which the literature consistently identifies as a manual-labour bottleneck [25, 39, 52].

Existing inference approaches each address part of the supply problem and leave part open. Dynamic detection (Daikon) requires a test suite and offers no soundness guarantee. Documentation extraction (Jdoctor) requires documentation. Template-and-verify (Houdini) is grounded only in the verifier, not in code structure. Language-model inference is ungrounded and inconsistent. None of these, moreover, addresses what happens to a specification after it is inferred — whether it can be distributed to consumers without the source, or whether single-pass inference leaves recoverable information behind.

This thesis occupies the gaps. It contributes a code-structural inference engine, grounded in the WP/SP calculi and validated against a formal oracle, that requires neither a test suite nor documentation (Chapter 3); a demonstration, with effect size, that its output carries measurable value to the most demanding contemporary consumer (Chapter 4); a mechanism for distributing that output to consumers who lack the source (Chapter 5); and a measurement of the compositional information that the single-pass inference strategy leaves on the table, with a second pass that recovers it (Chapter 6). The chapters that follow develop each in turn.

The positioning can be summarised as a single observation about where the thesis sits relative to the two mature ends of the field. At one end is verification — the tools that check specifications, mature and powerful, waiting for input. At the other end is generation and consumption — the test generators, the models, the analysis tools, mature and capable, also waiting for input. The input both ends wait for is the specification, and the manual cost of producing it is the gap between the two mature ends, the missing middle that keeps the powerful checking and the capable consuming from connecting. This thesis is an attempt to fill that missing middle: to produce the specifications automatically, certify them with the verification end, deliver them to the consumption end, and refine them as analysis allows. The four studies are four aspects of filling the middle, and the thesis’s claim is that the middle can be filled well enough that the two mature ends connect through it on real code at real scale.

Chapter 3

The JML-Inferrer System

This chapter presents JML-Inferrer, the static-analysis system that produces every specification used in the remainder of the thesis. It is the load-bearing artefact of the work: the empirical studies of Chapters 4, 5, and 6 all consume its output, and the central methodological claim of the thesis — that heuristic inference validated against a formal oracle yields useful specifications — is a claim about this system. The chapter describes the system at the level of detail required to interpret the empirical results that follow: its architecture (Section 3.1), the weakest-precondition/strongest-postcondition design discipline that organises its inference (Section 3.2), the inference of each clause kind (Sections 3.3 through 3.5), the interprocedural propagation that handles method calls (Section 3.7), the formal validation layer that recovers soundness (Section 3.8), and the method categorisation scheme used to structure the evaluation (Section 3.11).

3.1 Architecture

JML-Inferrer is implemented in Java 21 and comprises approximately twenty thousand lines of code at the time of writing. It uses `JavaParser` [29] to construct abstract syntax trees from Java source and processes source trees recursively, inserting JML annotations in place. The pipeline has three phases: parsing, specification inference, and formatting. Parsing and formatting are standard — the first builds an AST from each compilation unit, the third serialises inferred specifications as JML comments and writes the annotated source back — and the analytical substance is entirely in the second phase, which this chapter describes. Average processing time is 88 milliseconds per file, a figure that places the system comfortably within the budget of a continuous-integration step, a pre-commit hook, or an interactive editor integration.

The inference phase is built on the visitor pattern over AST nodes. A coordinating visitor traverses each compilation unit and, for every method declaration that is eligible for processing, delegates to a specification inferrer that analyses the method body and re-

turns a structured specification object holding the inferred preconditions, postconditions, loop invariants, and frame conditions. The coordinating visitor then formats this object as JML and injects it as a structured comment attached to the method. The design is deliberately non-destructive: a method that already carries a specification — a hand-written `requires`, `ensures`, or `loop_invariant` — is skipped, so the engine never overwrites a human-supplied contract. The non-destructive principle reflects the engine’s intended role as an assistant rather than an authority: where a human has stated a contract, that contract is taken as ground truth and left untouched, and the engine fills only the gaps the human left. The principle also makes the engine safe to run repeatedly over an evolving codebase — a second run does not disturb the specifications the first produced unless the code changed — which is what allows the continuous-integration deployment the thesis envisions, where the engine runs on every change and refreshes only the specifications whose methods changed. The formatting phase preserves any existing Javadoc, appending the JML rather than replacing the documentation, and renders the inferred clauses legibly — on separate lines, simplified, in the conventional order of preconditions before postconditions before frame conditions — because a specification a reviewer cannot read is one a reviewer cannot check, and the review queue for flagged clauses depends on the emitted form being legible. Methods with no body (abstract methods, interface methods) and methods whose specifications are inferred elsewhere are handled specially, a point that becomes important for the distribution work of Chapter 5, where interface methods turn out to be a source of subtle bugs.

A design constraint maintained throughout the implementation is a file-size ceiling: no source file exceeds five hundred lines, and a class that grows beyond it is split into focused sub-analysers by logical grouping — precondition analysis, postcondition analysis, loop-invariant analysis, frame analysis, and so on, composed by delegation rather than gathered into a monolith. The constraint is a software-engineering discipline rather than an analytical one, but it matters for the maintainability of an engine whose heuristic repertoire grows over time, and it reflects the broader stance of the project that the inferrer is itself a piece of engineered software held to the standards it helps enforce on others.

3.1.1 Design Philosophy and Engineering Constraints

The engine is a research instrument, and its construction was governed by a philosophy worth stating because it shaped decisions throughout. The governing principle is that the correct solution is preferred to the easy one: when the engine produces invalid JML, the response is to fix the inference to produce valid JML, not to suppress the offending output. A pattern that cannot be expressed soundly in JML is replaced by the closest valid expression that captures part of its intent, and inference for a pattern is removed only as a last resort, when no valid JML can be produced for it at all. This principle is what keeps the engine’s output a set of genuine, checkable specifications rather than a mixture

of valid JML and natural-language approximations, and it is enforced by the requirement that every emitted clause be syntactically valid JML with no undefined variables and no expression that is false at the point it is asserted.

The engineering constraints follow from the engine’s status as evolving research software. The five-hundred-line file ceiling, mentioned above, forces the decomposition into focused sub-analysers. Every change to the inference logic is accompanied by verification tests that prove the inferred specifications remain valid, and a regression suite gates changes against the formal oracle, so that an improvement to one pattern cannot silently break another. The testing requirement is two-tiered, as Section 3.8 describes, and the preference is always for testing through the full inference pipeline rather than against hand-written JML, because the engine’s purpose is to *infer* specifications and a test that bypassed inference would not exercise that purpose. These constraints are not incidental; they are what allow a heuristic engine, whose pattern repertoire grows continually, to remain correct as it grows, and they are the software-engineering counterpart to the methodological discipline of validating every clause.

3.1.2 Why Pattern Matching Rather Than Symbolic Execution

The most consequential architectural decision is the deliberate avoidance of symbolic execution [7, 32] and full weakest-precondition computation. Both require an SMT decision procedure and a complete operational semantics for Java, and neither scales to arbitrary code: symbolic execution suffers path explosion on branch-heavy code and diverges on loops without a supplied bound, and a complete WP transformer needs the loop invariants that are themselves the hardest thing to infer. The engine instead performs pattern matching over AST nodes, recognising syntactic shapes that correspond to specification clauses and emitting the corresponding JML. This trades formal completeness for scalability and broad applicability — an established trade-off in industrial static analysis, where Infer and similar tools accept unsoundness in exchange for the ability to run on millions of lines [8].

The trade-off would be indefensible if it meant emitting unsound specifications with no recourse. The recourse is the validation layer of Section 3.8: the engine matches patterns aggressively and discharges the results conservatively, so that the unsoundness inherent in pattern matching is caught after the fact by a verifier rather than shipped to the consumer. This two-layer architecture — aggressive heuristic inference, conservative formal validation — is the system’s answer to the soundness objection, and it recurs as a theme throughout the thesis.

The trade-off has a precise cost and a precise benefit that are worth naming. The cost is recall: a pattern matcher recognises only the shapes it has been taught, so a precondition expressed by a guard idiom the engine does not recognise, or a postcondition that depends on a computation the patterns do not reach, is missed. The recall figures

of Chapter 4 measure this cost directly. The benefit is scale and applicability: the engine runs in milliseconds per file on any compilable Java, with no test suite, no documentation, and no SMT solver in the inference loop, which is what makes it practical to run over twenty-thousand-method libraries and to integrate into a build. A symbolic executor or a complete WP transformer would recover some of the missed recall but at a cost in scale that would foreclose the corpus-wide measurements the thesis depends on. The pattern-matching choice is thus not a compromise forced by implementation expedience but a deliberate position on the recall-versus-scale frontier, chosen because the thesis’s questions — does inference help, can it be distributed, what does a second pass add — are questions about scale, answerable only by a tool that runs at scale, and because the recall the choice sacrifices is partly recoverable by the complementary techniques (dynamic detection, documentation extraction) the thesis identifies as future ensemble partners.

3.1.3 The Sub-Analyser Composition

The five-hundred-line file ceiling forces a composition the inference engine would benefit from in any case: rather than a monolithic inferer, the engine is a collection of focused sub-analysers, each responsible for one clause kind or one pattern family, composed by delegation behind the coordinating visitor. A precondition analyser owns the guard-and-throw recognition; a postcondition analyser owns return and assignment structure; a loop-invariant analyser owns the four loop heuristics; a frame analyser owns purity and assignable inference; an overflow analyser owns the return-type gating of arithmetic clauses; an interprocedural analyser owns the call-site propagation; a class-invariant analyser owns field-level invariants; and a string analyser owns the value-equality subtleties of `String`-returning methods. The coordinating visitor invokes each in turn on a method and assembles their outputs into the method’s specification object.

The decomposition has a methodological payoff beyond maintainability. Because each sub-analyser is independently testable, the engine’s regression suite can target a specific analyser’s behaviour — asserting, for instance, that the precondition analyser emits exactly the expected `requires` clauses for a given guard shape — without running the whole pipeline. The suite is two-tiered: analysis tests call a sub-analyser directly and assert on its inferred clauses without invoking the verifier, while verification tests run the full pipeline through OpenJML and assert that the inferred specification discharges. The two tiers correspond to the two oracles of Section 3.8 — the analysis tier checks what the engine emits, the verification tier checks that what it emits is sound — and the decomposition into sub-analysers is what makes the analysis tier precise enough to localise a regression to a single pattern family. A change that improves one analyser can be verified not to regress another, which matters for an engine whose heuristic repertoire grows continually as new patterns are added.

The deduplication of clauses is handled at assembly: each clause-adding operation on

the specification object checks for an existing equal clause before adding, so that two sub-analysers that independently infer the same clause — the precondition analyser and the interprocedural analyser both concluding a parameter is non-null, say — do not produce a duplicate. This matters for the distribution work of Chapter 5, where duplicate clauses would inflate the embedded size, and for the readability of the emitted JML, where a duplicated `requires` clause would be noise.

3.2 The WP/SP Design Discipline

The engine performs no symbolic WP or SP computation, but the WP and SP calculi are not thereby irrelevant to it. They serve as a *design discipline*: they determine which AST patterns the engine recognises, what JML clause each pattern produces, and how clauses propagate across procedure boundaries. The discipline is what keeps the heuristic repertoire principled rather than ad hoc — every pattern the engine matches is justified by a correspondence to a step of WP or SP reasoning, and patterns that have no such correspondence are deliberately excluded.

The correspondence runs as follows. Preconditions are WP-derived: a guard-and-throw idiom corresponds to the weakest precondition of the normal-return path, and the engine emits the negated guard as a `requires` clause. Postconditions are SP-derived: a return statement and a field assignment each correspond to a strongest-postcondition consequence, and the engine emits the corresponding equality. Interprocedural propagation follows the textbook WP rule for procedure call: the callee’s contract is instantiated by formal-to-actual substitution at the call site. Loop invariants are the point at which syntactic WP/SP reasoning is incomplete, and the engine supplies heuristics in their place. The remainder of this chapter develops each of these correspondences in turn, and in doing so makes precise what the engine looks for and why.

3.3 Precondition Inference

Preconditions arise, under the WP discipline, from guards that abort the normal execution path. Consider a method body that begins with an early statement of the form `if (G) throw ...` or `if (G) return ....`. Taking the postcondition Q to be the post of the normal-return path, a weakest- precondition analysis propagates $\neg G$ backward to the method entry: the precondition is whatever must hold on entry to avoid the abort. The engine recognises throw-style guards, early-return guards, and validation-style guards (calls into validation utilities such as `Objects.requireNonNull` or a library’s `Validate` class), and emits the negation of each guard as a `requires` clause.

The most common shapes are worth enumerating because they account for the bulk of inferred preconditions. A null guard `if (p == null) throw ...` yields the precondition

`p != null`. A range guard `if (i < 0 || i >= n) throw ...` yields two conjuncts, `i >= 0` and `i < n`, because the disjunctive guard is violated when either conjunct fails and the weakest safe precondition requires both. A chained dereference such as `a.b.c` contributes its definedness conditions — the non-nullness of each receiver in the chain — which in WP terminology are the $\text{def}(e)$ side-conditions that must hold for the expression to evaluate without fault. Division contributes a non-zero-divisor precondition, and a division of the most-negative integer by minus one contributes an overflow-avoidance precondition, because that single case overflows under two’s-complement semantics.

3.3.1 What the Discipline Excludes

The WP discipline is as important for what it excludes as for what it includes, and the exclusions are where a naive pattern matcher would go wrong. A condition that controls *branching* rather than *aborting* — an `if/else` in which both branches return a value, or the condition of a ternary expression — yields no WP-derived precondition on the entry state, because neither branch aborts and so no entry condition is required to avoid an abort. A pattern matcher that emitted such a condition as a **requires** clause would misclassify internal control flow as a contract requirement, over-constraining the method and producing a clause that a verifier would reject. The engine therefore distinguishes aborting guards from branching conditions and emits preconditions only for the former. This distinction — between a guard that says “you may not call me unless G ” and a branch that says “if G then I do this, otherwise that” — is the single most error-prone judgment in precondition inference, and getting it right is the difference between a contract and a description of the implementation.

3.3.2 A Worked Example

The distinction is best seen on a concrete pair of methods. Consider first a defensive accessor:

```
public static int firstOf(int[] arr) {
    if (arr == null) {
        throw new NullPointerException("arr");
    }
    if (arr.length == 0) {
        throw new IllegalArgumentException("empty");
    }
    return arr[0];
}
```

Both guards abort, so both contribute WP-derived preconditions. The engine emits **requires** `arr != null` from the first guard and **requires** `arr.length > 0` from the second (the negation of `arr.length == 0`). The return statement contributes the post-condition **ensures** `\result == arr[0]`, and because the body reads but never writes,

the engine emits `assignable \nothing`. The inferred contract is exactly the contract a careful engineer would write, and every clause discharges against the body: the array access `arr[0]` is in bounds precisely because the two preconditions establish non-nullness and non-emptiness.

Now consider a superficially similar method whose leading condition does not abort:

```
public static String sign(int x) {
    if (x < 0) {
        return "negative";
    } else {
        return "non-negative";
    }
}
```

Here the condition `x < 0` controls branching, not aborting: neither arm throws, and both return a value. A pattern matcher that treated every leading `if` as a guard would emit the unsound and nonsensical precondition `requires x >= 0`, forbidding exactly the inputs the method is designed to handle. The engine instead recognises that both arms return and emits no precondition from this condition. It does, however, record the condition as a *path condition* for branch-sensitive postcondition inference, producing the two guarded clauses `ensures x < 0 ==> \result.equals("negative")` and `ensures x >= 0 ==> \result.equals("non-negative")`. The contrast between the two methods — identical in surface shape, opposite in the clauses they should yield — is the reason the aborting-versus-branching judgment is implemented as a distinct analysis step rather than folded into the guard matcher.

3.4 Postcondition Inference

Postconditions are forward-flow predicates derived, under the SP discipline, from return statements and field assignments. A return statement `return e` contributes the clause `\result == e`, the strongest-postcondition consequence relating the return value to the returned expression. A field assignment `this.f = e` contributes the clause `this.f == e`, with field references on the right-hand side wrapped in `\old(...)` to capture the SP convention that the assigned value is computed from the pre-state. A method that constructs and returns a new object contributes the postcondition `\result != null`, because object construction cannot return null.

The treatment of mutable parameters follows the standard SP convention. Java parameters can be reassigned within a method body, which would render a postcondition referring to a parameter ambiguous: does `x` in the postcondition mean the value passed in or the value after some internal reassignment? SP resolves this by modelling each parameter as a fresh entry-value variable and expressing postconditions in terms of that entry value. In emitted JML the convention is rendered by using the parameter name directly

— JML interprets a bare parameter name in a postcondition as its entry value — and `\old(...)` for fields, which are not frozen at entry the way parameters are.

3.4.1 Branch-Sensitive Postconditions

Branching receives finer-grained treatment than the textbook SP rule for conditionals prescribes. The textbook rule combines the two branches of an `if/else` by disjunction: $SP(S_1, P \wedge C) \vee SP(S_2, P \wedge \neg C)$. Emitting this disjunction directly would be correct but lossy: it discards the information that links each return value to the condition that produces it. A consumer of the specification — a test generator trying to construct an input that exercises a particular return path, or a verifier checking a particular branch — wants to know not merely that the result is one of two values but which condition yields which. The engine therefore records, for each return statement, the *path condition* that reaches it, and emits independent guarded clauses $C \Rightarrow \text{post}_1$ and $\neg C \Rightarrow \text{post}_2$ rather than the collapsed disjunction. Ternary expressions are decomposed identically, each arm becoming a guarded clause. A lightweight symbolic executor maintains the path condition as it walks the body, so that a return nested several conditionals deep carries the conjunction of the enclosing guards.

Path conditions are simplified algebraically before emission, so that the resulting JML is readable rather than a thicket of redundant conjuncts. The simplifier applies the obvious arithmetic identities: $x \leq 0 \wedge x < 0$ reduces to $x < 0$, and $x \leq 0 \wedge x \geq 0$ reduces to $x = 0$. The simplification is purely syntactic and conservative — it never strengthens or weakens a condition, only rewrites it to an equivalent shorter form — so it cannot introduce unsoundness, only improve legibility.

3.4.2 The Symbolic Executor and Path Conditions

The branch-sensitive postcondition inference of the previous subsection rests on a lightweight symbolic executor, and its design is worth describing because it is the component that makes guarded postconditions possible without incurring the cost of full symbolic execution. A full symbolic executor explores every path through a method, maintaining a symbolic state and a path condition for each, and suffers path explosion on branch-heavy code and divergence on loops. The engine's executor is deliberately partial: it tracks the path condition — the conjunction of branch guards under which a program point is reached — but does not maintain a full symbolic store, and it does not explore loop bodies symbolically, deferring loops to the invariant heuristics.

For postcondition inference this partial tracking is exactly enough. When the executor reaches a return statement, it has accumulated the conjunction of the guards on the path to that return, and it emits the guarded clause pairing that path condition with the returned expression. A return nested two conditionals deep carries the conjunction of

both enclosing guards; a return in the else-branch of a conditional carries the negation of that conditional’s guard. The executor handles both `if/else` statements and ternary expressions by the same mechanism, decomposing each into per-path returns rather than collapsing them into a disjunction, which is the choice that preserves the link between each return value and its enabling condition.

The cost of the partiality is that the executor cannot infer postconditions that depend on the symbolic values of intermediate computations — it tracks the path condition but not a full model of how each variable’s value evolves along the path. A method whose return value depends on a sequence of intermediate assignments, rather than directly on the path condition and the parameters, yields a weaker postcondition than a full symbolic executor would. This is a deliberate recall sacrifice for tractability, consistent with the engine’s overall stance: the partial executor handles the common case — a return whose value and enabling condition are both locally apparent — cheaply and correctly, and defers the hard case rather than paying the cost of full symbolic execution to capture it. The path conditions the executor produces are simplified algebraically before emission, as the previous subsection described, so that even a deeply nested return yields a readable guarded clause rather than an unsimplified thicket of conjuncts.

3.4.3 Soundness Gating by Return Type

A class of plausible-looking postconditions is unsound under Java’s two’s-complement integer arithmetic, and the engine gates these by return type. The clause `\result >= 0` is true for `Math.abs(x)` and for `x * x` only under mathematical-integer semantics; under two’s-complement, `Math.abs(Integer.MIN_VALUE)` is negative and `x * x` overflows to a negative value for large `x`. The engine emits the non-negativity postcondition only when the return type is floating point, where the concern does not arise, and suppresses it for `int` and `long`. This is a representative instance of a general principle: a postcondition that is true in the mathematical idealisation but false in the machine’s actual arithmetic must be gated by the conditions under which the idealisation holds, so that every emitted clause can withstand formal verification under the operational integer semantics the validation layer enforces.

3.5 Loop Invariant Inference

Loops are the point at which WP/SP reasoning is incomplete by syntax alone. The WP of a `while` loop with respect to a postcondition requires a loop invariant, and no purely syntactic procedure can derive invariants in general, since invariant inference is undecidable. The engine addresses the requirement through four complementary heuristics, applied in turn until one succeeds.

The first is *bound-based* invariant inference for counter loops. A loop of the form `for`

`(int i = a; i < b; i++)` maintains, throughout its execution, the bound $a \leq i \leq b$, and the engine emits a direction-aware bound: $i \geq a$ for an incrementing loop, $i \leq a$ for a decrementing one, together with the relationship to the upper bound. The direction-awareness matters, because an invariant that is true for an incrementing loop is false for a decrementing one, and a pattern matcher that ignored the step direction would emit an invariant that fails at loop entry or exit.

The second is *accumulator pattern detection*. A loop that maintains a running sum, product, maximum, or minimum — the canonical reduction patterns — admits an invariant relating the accumulator to the portion of the data processed so far. A maximum-finding loop, for instance, maintains that the accumulator is greater than or equal to every element seen so far and equal to some element seen so far; the engine emits both the universal and the existential clause, because the postcondition that the result is the maximum is not provable from either alone. These quantified invariants are the engine’s most ambitious output and the ones that most stress the downstream verifier, a point developed in Section 3.8.

The third is *monotonicity inference* from update expressions. A variable updated only by addition of a non-negative quantity is non-decreasing; a variable updated only by a guarded decrement that stops at zero stays non-negative. The engine walks the loop body’s update statements and emits monotonicity invariants where the update shape guarantees them.

Each of the first three heuristics corresponds to a recognisable family of loops, and the correspondence is what keeps them principled rather than ad hoc. The bound-based heuristic targets the counting loop, whose invariant is a relationship between the counter and its bounds; it is the most common loop shape and the most reliably handled. The accumulator heuristic targets the reduction loop, whose invariant relates a running result to the processed prefix; it is the most ambitious, producing the quantified invariants that the verifier extension exists to discharge. The monotonicity heuristic targets loops whose interest is the direction in which a variable moves rather than its exact value, and it produces the weaker but still useful invariants — non-decreasing, non-negative — that suffice when an exact relationship is out of reach. The three together cover the loops whose invariants are determined by their counter, their accumulation, or their update direction, which is the bulk of the loops in ordinary code.

The fourth, a fallback when the first three do not apply, is *bounded symbolic unrolling*: the loop is unrolled a small fixed number of times (three in the current configuration) and the resulting straight-line code analysed for an invariant that holds across the unrolled iterations. Unrolling is a fallback rather than a primary heuristic because it is unsound as a source of invariants — a predicate that holds for three iterations need not hold for all — so its output is treated as a candidate to be checked rather than as a trusted invariant, and the validation layer is what catches the candidates that fail to generalise. The fallback exists because a weak candidate invariant that the verifier can check is more useful than

no invariant at all: it gives the discharge step something to attempt, and a candidate that happens to generalise is recovered, while one that does not is flagged. When all four heuristics fail to produce a non-trivial invariant, the engine emits a conservative weak invariant in preference to nothing, so that the verification condition is at least partially discharged rather than left entirely open. Across the loops in the evaluation corpus, the four heuristics together produce a non-trivial invariant for roughly eighty-five percent of loops, applying uniformly to `for`, `while`, and `for-each` constructs.

3.5.1 Promotion of Invariants to Postconditions

A quantified loop invariant that remains valid at loop termination can be promoted to a method postcondition by substituting the loop counter with its exit value. A summation loop whose invariant states that the accumulator equals the sum of the first i elements yields, on termination with i equal to the array length, the postcondition that the result equals the sum of the whole array. This promotion is how established loop predicates flow into the strongest postcondition of the surrounding method, and it is what allows the engine to emit a meaningful postcondition for a method whose entire behaviour is a loop. The promotion is sound only when the invariant genuinely holds at exit, which the validation layer checks; a non-decreasing variant promoted as if it were monotonically decreasing is exactly the kind of error the discharge step catches.

3.6 Frame Conditions and Purity

A specification is incomplete without a frame condition — a statement of which locations a method may modify — because a postcondition that constrains the return value says nothing about whether the method also scribbled on the heap. In JML the frame condition is the `assignable` clause, listing the locations the method is permitted to modify; `assignable \nothing` declares the method pure (modifying no heap location), and `assignable \everything` is the conservative default that constrains nothing. The frame condition is essential to modular verification: when a verifier replaces a called method with its contract, it must know which locations the call may have changed, or it cannot preserve any fact about the unchanged ones.

The engine infers frame conditions by analysing the writes a method performs. A method whose body contains no field assignment, no array store, and no call to a method with a non-trivial frame is pure, and the engine emits `assignable \nothing`. A method that assigns to a field `this.f` has `f` in its assignable set; a method that stores into an array parameter has that array's elements in its set. The analysis is transitive through calls: a method that calls another inherits the callee's frame, so a method that is pure in its own body but calls a mutating method is not pure overall. The conservative default for a call into an unanalysed method is that it may modify anything, which is why an

otherwise-pure method that calls an unsummarised library method receives a conservative frame rather than `\nothing`.

The empirical regularity that almost every method in the evaluation corpus is pure — the corpus being utility and value-type code — has a consequence the distribution work of Chapter 5 exploits directly: since `assignable \nothing` is the overwhelming common case, the distribution format makes it the default and omits it from the wire representation, recovering it on read. The frame condition is thus a small clause with an outsized role: it is necessary for sound modular verification, it is trivial to infer for pure methods, and its ubiquity is the basis for a size optimisation downstream. The discharge results of Chapter 4 reflect its tractability — assignable clauses discharge at a higher rate than any other clause kind, because purity is the easiest property to establish against a body that visibly performs no writes.

3.7 Interprocedural Propagation

Methods call other methods, and a specification that ignored calls would miss the obligations they impose. The engine handles calls through procedure summaries, the standard mechanism for modular WP/SP analysis. It proceeds in two passes. The first analyses methods in dependency order — callees before callers, following a topological sort of the call graph — and caches the inferred specification of each method. The second pass propagates these cached specifications across call sites: when inference encounters a call `m(a1, ..., ak)`, it retrieves the cached pre- and postconditions of `m` and instantiates them by formal-to-actual substitution, exactly as the textbook WP rule for procedure call prescribes. A precondition that `m` requires of its j -th parameter becomes, at the call site, a precondition on the j -th actual argument.

Calls into the Java standard library are resolved against a curated specification database covering common operations on `String`, `Math`, `List`, `Map`, and the standard utility classes. The database is maintained by hand, which bounds its coverage but also bounds the risk of propagating an incorrect library summary; a curated entry is trusted, and a call into a library method with no entry is treated conservatively. Calls into other unannotated methods — application methods with no cached specification and no database entry — are treated as uninterpreted: the callee may throw, may modify any accessible state, and returns an unconstrained value. This conservative treatment is the limit case of the procedure-summary mechanism, and it is the reason a small number of methods in the evaluation corpus receive only an empty contract: they delegate entirely to a library method for which no summary is available, and the engine declines to invent one.

A small example fixes the mechanism. Suppose a method `checkedGet` calls a helper `elementAt(list, index)` whose inferred contract requires `index >= 0` and `index < list.size()`. When the engine analyses a caller `int second(List<Integer> xs) { return elementAt(xs,`

`1); }`, the interprocedural analyser retrieves the helper’s cached preconditions and substitutes the actual arguments — `xs` for the formal list and `1` for the formal index — to obtain `1 >= 0` (a tautology the simplifier drops) and `1 < xs.size()`. The latter becomes a precondition of `second`: the caller may invoke it only on a list of size at least two. The obligation, invisible in `second`’s own body, is recovered from the helper’s contract by substitution, which is exactly the textbook weakest-precondition rule for procedure call applied by the single pass. The example also shows the single pass at its competent best — an unconditional callee precondition lifted cleanly to the caller — which is the case the next chapter contrasts with the guarded and polymorphic cases the single pass cannot handle.

The single-pass character of this propagation — each callee’s contract read once and substituted at the call site without further transformation — is adequate for unconditional preconditions but blind to the branch-conditional and polymorphic-dispatch obligations that Chapter 6 addresses. That chapter’s contribution is precisely a measurement of what this single pass leaves behind, and a second pass that recovers it. The present chapter describes the single pass as the engine’s baseline interprocedural behaviour; the limitation is named here and quantified there.

3.7.1 The Standard Library Specification Database

The conservative treatment of unsummarised calls would cripple the engine on real code, because real code calls the standard library constantly — a method that calls `Math.abs`, `String.length`, or `List.get` would, under the uninterpreted-call policy, lose all information at the call site. The engine mitigates this with a curated database of specifications for common standard-library operations, covering `String`, `Math`, the collection interfaces `List` and `Map`, and the boxed-primitive utility classes. When the interprocedural analyser encounters a call into a library method with a database entry, it uses the curated specification in place of the conservative summary: a call to `Math.abs` on a floating-point argument contributes the non-negativity of the result, a call to `String.length` contributes a non-negative integer, a call to `List.get` contributes the in-bounds precondition on its index.

The database is curated by hand rather than inferred, which is a deliberate trade-off. Hand curation bounds the database’s coverage — only the entered methods are summarised — but it also bounds the risk: a curated entry is trusted, having been written and reviewed by a human, so propagating it cannot introduce the unsoundness an automatically inferred library summary might. The database is, in effect, a small set of axioms the engine takes on faith, and its size is the price of avoiding the alternative of inferring specifications for the standard library (which would require its source, often unavailable) or treating every library call conservatively (which would lose most cross-library information). The three uninferable methods in the evaluation corpus of Chapter 4 are precisely

those that delegate to a library method with no database entry: the engine declines to invent a summary and emits an empty contract, which is the correct conservative behaviour at the boundary of the database’s coverage.

3.7.2 The Boundaries of Heuristic Inference

It is as important to state what the engine does not infer as what it does, both for honesty and because the boundaries explain the recall figures of Chapter 4. The engine does not infer specifications that require reasoning about the global heap shape — properties relating objects reachable through long reference chains, which need a shape analysis the pattern matcher does not perform. It does not infer concurrency properties — the absence of data races, the correctness of a locking discipline — because these are not local syntactic properties and the engine reasons locally. It does not fully resolve the interaction of inheritance and specification: a method that overrides another should, under behavioural subtyping, honour the overridden method’s contract, and the engine’s handling of inherited specifications is partial. And it does not infer the complex algorithmic postconditions that characterise a method’s deep semantic purpose — that a sort produces a sorted permutation, that a search returns the correct index — when those postconditions are not visible in the local structure of returns and assignments.

These boundaries are the source of the recall gap: the postcondition recall of 78.1% against documented intent (Chapter 4) is the complement of the documented behaviours that lie outside what local pattern matching can reach. The boundaries are not failures of the implementation but consequences of the heuristic approach, and crossing some of them — the algorithmic postconditions in particular — would require the dynamic observation or the symbolic execution that the engine deliberately forgoes for scalability. Naming the boundaries precisely is what allows the recall figure to be read correctly: not as a measure of how often the engine is wrong, but as a measure of how much documented intent lies beyond the reach of local structural inference.

3.8 Formal Validation

Because the engine matches patterns rather than computing symbolic transformers, individual clauses are not guaranteed sound by construction. The WP/SP framing prescribes *what* the engine looks for; it does not guarantee that each emitted clause is provable against the actual code. Soundness is recovered *a posteriori* by discharging every emitted clause through OpenJML’s Extended Static Checker [11]. After inference, the verifier may be invoked on the annotated source; each clause is discharged independently, and clauses that fail verification are flagged for review rather than silently retained. The validation pipeline is distributed as a containerised toolchain to support reproducibility across operating systems, and it underpins a regression suite of several hundred end-to-end verification tests

that gate every change to the inference engine against the formal oracle.

3.8.1 Extending the Verifier

The discharge layer uses a forked OpenJML build that emits SMT-LIB `define-fun-rec` declarations for the `\sum`, `\product`, and `\num_of` generalised quantifiers. Upstream OpenJML reports these constructs as “not yet supported,” which would leave the loop-accumulator postconditions that the engine works hardest to produce undischARGEABLE — recorded as unverifiable rather than proved or refuted. The fork supplies recursive function definitions that give the solver a handle on these quantifiers, converting a category of “cannot check” into “checks” or “fails,” and it is bundled with the toolchain so that the validation pipeline remains reproducible. The extension is a contribution in its own right: it is the difference between an inference engine whose most ambitious output is unverifiable in principle and one whose output is verifiable in practice.

3.8.2 Strictness Configuration

The strictness of the verifier is fixed at a configuration that was settled empirically over the course of the project, because the choice materially affects both what discharges and how long discharge takes. Four flags are set. `--code-math=safe` interprets the executable code’s integer arithmetic with operational two’s-complement semantics and overflow checks, so that a clause must hold under the arithmetic the machine actually performs. `--spec-math=bigint` interprets specification arithmetic with unbounded mathematical integers, which is the right choice for the *intent* a specification expresses while `code-math=safe` guards the *implementation*. `--arithmetic-failure=hard` treats an arithmetic exception as a verification failure rather than a warning, so that overflow is caught rather than tolerated. `--nullable-by-default` treats unannotated reference types as potentially null, the conservative assumption. The combination of `code-math=safe` with `spec-math=bigint` was found to be the configuration that maximises genuine discharge without provoking solver timeouts; stricter combinations (`spec-math=safe`, feasibility checking on every path) were measured to cause the SMT solver to time out on the project’s preamble, discharging fewer clauses despite being nominally stronger. The default solver is Z3; alternative versions are bundled and selectable, and an alternative solver (cvc5) is available for cross-checking but is not used by default because it does not consistently terminate on the project’s preamble within the per-test timeout.

3.8.3 The Two Layers Are Independent

Manual validation and formal discharge are independent oracles, and the distinction is important enough to state explicitly. Manual validation asks: does the inferred clause agree with the method’s documented intent? Formal discharge asks: is the inferred clause

consistent with the method’s implementation? A clause can pass either without the other. A clause may match the Javadoc-documented intent perfectly yet fail discharge because of a subtle overflow boundary the documentation does not mention; conversely, a clause may discharge against the implementation yet be judged incomplete relative to the documented intent, because it captures only part of what the method promises. The two layers catch different errors — manual validation catches clauses that are formally consistent but miss the point, formal discharge catches clauses that look right to a human but conflict with the code’s actual arithmetic or aliasing behaviour — and running both reduces the risk that high precision under one masks errors the other would surface. This independence is exploited in the evaluation of Chapter 4, where the two are reported as separate measurements rather than conflated into a single accuracy figure.

3.9 Class Invariants and History Constraints

Beyond the per-method clauses, the engine infers class-level invariants — predicates that hold in every visible state of an object, before and after every public method. A class invariant captures a property of the object’s representation that the class maintains as a whole, rather than a property of a single method. The canonical case is a numeric field constrained to a range: a class with a `size` field that is incremented and decremented but never allowed to go negative has the invariant `size >= 0`, and the engine infers it by checking that every assignment, increment, and decrement of the field preserves the bound.

The inference is more delicate than it appears, because a field’s invariant must be preserved by *every* method, and a single method that violates it refutes it. The engine therefore examines all of a field’s mutations across the class before concluding an invariant holds. A decrement guarded by a positivity check — `if (size > 0) size--` — preserves `size >= 0`, and the engine recognises the guard; an unguarded decrement does not, and the engine must not emit the invariant, because a sequence of calls could drive the field negative. An early version of the engine emitted the invariant on the strength of the field’s declared initial value alone, ignoring the decrements, and the validation layer caught the resulting unsoundness — a concrete instance of the two-layer architecture catching an over-eager heuristic, and the reason the class-invariant analyser now walks every mutation rather than trusting the initialiser.

History constraints — the `\old` references in postconditions — are the per-method analogue of class invariants in their subtlety. A method that increments a counter has the postcondition `this.count == \old(this.count) + 1`, relating the new state to the old. The engine wraps a field reference on the right-hand side of an assignment in `\old` when the field is read before being written in the same expression, capturing the strongest-postcondition convention that the assigned value is computed from the pre-state. Getting the `\old` wrapping right is necessary for soundness: a postcondition that omitted it would

`assert this.count == this.count + 1`, a contradiction, and one that over-applied it would refer to pre-state values where the body uses post-state ones. The wrapping is therefore applied precisely, to field references whose value the postcondition draws from before the method’s writes, and the discharge results confirm the precision — the `\old`-bearing postconditions discharge at a rate consistent with the rest, indicating the wrapping is neither omitted nor overapplied.

3.10 An End-to-End Example

The individual analyses are best understood together, applied to a single method that exercises preconditions, a loop with an accumulator, a promoted postcondition, and the frame condition. Consider a method that sums the elements of an array:

```
public static long sum(int[] arr) {
    if (arr == null) {
        throw new NullPointerException("arr");
    }
    long total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i];
    }
    return total;
}
```

The engine processes this method as follows. The precondition analyser (Section 3.3) recognises the leading guard-and-throw idiom and emits `requires arr != null`, the negation of the aborting guard. No other guard is present, so no further precondition is emitted; in particular the loop condition `i < arr.length` is a branching condition internal to the loop, not an aborting guard, and contributes nothing to the precondition.

The loop-invariant analyser (Section 3.5) examines the `for` loop. The counter `i` is initialised to zero and incremented by one with an upper bound of `arr.length`, so the bound-based heuristic emits the direction-aware invariant `loop_invariant 0 <= i && i <= arr.length`. The body `total += arr[i]` matches the accumulator pattern: `total` is a running sum over the processed prefix of the array, so the accumulator heuristic emits the invariant `loop_invariant total == (\sum int k; 0 <= k && k < i; arr[k])`, relating the accumulator to the sum of the first `i` elements. This is the quantified invariant whose discharge motivates the `define-fun-rec` extension to OpenJML described in Section 3.8; without that extension the verifier would record it as unsupported.

The invariant-promotion step (Section 3.5) observes that the sum invariant remains valid at loop termination, where `i` equals `arr.length`, and substitutes the exit value of the counter to obtain the method postcondition `ensures \result == (\sum int k; 0 <= k && k < arr.length; arr[k])`. This is the strongest postcondition the engine can establish for the method, and it is exactly the specification a human would write: the

result is the sum of the whole array. The frame analyser observes that the body writes only to the local `total` and `i`, never to a heap location, and emits `assignable \nothing`.

The assembled contract is therefore:

```
/*@ requires arr != null;
   @ ensures \result ==
   @   (\sum int k; 0 <= k && k < arr.length; arr[k]);
   @ assignable \nothing;
public static long sum(int[] arr) { ... }
```

Every clause discharges. The precondition guards the array dereference inside the loop; the loop invariants are inductive (preserved by the body and established on entry) and combine with the exit condition to prove the postcondition; and the frame condition holds because the body touches no heap location. The example illustrates the division of labour the chapter has described: precondition inference from the guard, loop-invariant inference from the counter and accumulator, postcondition inference by promotion of the invariant, and frame inference from the absence of writes — each a separate analysis, justified by a distinct correspondence to the WP/SP calculi, and combining into a contract that the validation layer certifies. It also illustrates the return type’s role in soundness: the accumulator is declared `long`, and had it been declared `int` the engine would have to weigh the overflow risk of summing a large array, gating any non-negativity claim by the considerations of Section 3.4.

3.11 Method Categorisation

To enable systematic evaluation across diverse implementations, the engine assigns each method to one of six categories, extending the method-stereotype taxonomy of Dragan et al. [17]. The categories are *accessors and mutators* (methods that read or write a single field), *control flow* (methods organised around conditionals and loops), *factory and delegate* (methods that construct objects or forward to other methods), *utility* (static helper methods), *state modification* (methods that mutate object state under conditional logic), and *other* (event handlers and callbacks). When more than one category’s pattern matches a method, the most specific is chosen. The categorisation is not used by the inference engine itself — inference is category-blind — but it structures the evaluation, allowing the studies that follow to report where inferred specifications are most accurate and where they most improve downstream tasks. The taxonomy’s inter-rater reliability, established by two independent raters categorising the evaluation corpus, is high (Cohen’s $\kappa = 0.87$), which gives confidence that the per-category results of later chapters rest on a stable partition rather than on idiosyncratic labelling.

The categorisation also exposes a structural feature of the evaluation corpus that recurs in the threats-to-validity discussions of later chapters: the *other* category — event handlers, I/O operations, callbacks — is empty in the Apache Commons Lang corpus, be-

cause that library is a collection of pure utility and value types and contains no methods of that kind. Experience on broader Java code suggests this category is the hardest to specify heuristically, since its methods often have effects that escape the local syntactic patterns the engine recognises, and its absence from the controlled corpus is a limitation on the external validity of the results that the relevant chapters name explicitly.

3.11.1 How Inference Behaviour Varies by Category

Although inference is category-blind, the engine’s behaviour varies systematically across the categories, and understanding the variation explains the per-category accuracy results of Chapter 4. For accessors and mutators, inference is at its most reliable: a getter’s postcondition is the field it returns, a setter’s is the assignment it performs, and both are captured by the most direct postcondition patterns, which is why this category attains the highest precision. For control-flow methods, precondition inference dominates: their guards yield **requires** clauses reliably, and their branch structure yields guarded postconditions, so the engine produces strong specifications whenever the branches return values the patterns recognise. For utility methods, the picture is mixed: their input-validation guards yield good preconditions, but their computational postconditions — what the utility actually computes — often lie beyond the local patterns, which is why their postcondition recall trails their precondition recall.

For state-modification methods, inference is at its hardest, and the category’s lower specification strength reflects it. A method that mutates object state under conditional logic has a frame condition and a set of guarded state-change postconditions that the engine must assemble from the assignments scattered through its branches, and capturing the complete effect — every field that might change, under every condition — is the inference task most prone to incompleteness. The engine captures the assignments it can localise and emits guarded postconditions for them, but a method whose state change is distributed across many branches or mediated by helper calls yields a partial rather than a complete specification. For factory and delegate methods, inference depends heavily on interprocedural propagation: a factory’s postcondition is that it returns a non-null constructed object, captured directly, but a delegate’s behaviour is its callee’s behaviour, captured only if the callee has a cached or curated specification — which is why the three uninferable methods of the corpus are delegates to an unsummarised library method. The category variation in inference behaviour is thus the upstream cause of the category variation in both specification accuracy (Chapter 4) and test-generation benefit: the categories the engine specifies most completely are not always the categories where specifications help most, because the benefit depends also on the baseline, but the categories the engine specifies least completely — state modification, complex delegation — are consistently the ones where both accuracy and benefit are bounded by what local inference can reach.

3.12 Operation, Logging, and Diagnostics

A research instrument that is run thousands of times over evolving code needs to be observable, and the engine’s logging and diagnostics, while not an analytical contribution, are part of what made the empirical studies possible. The engine logs at two levels: a progress level reporting which files it processes and how many specifications it produces, suitable for a console during a run, and a detailed level recording, for each method, which patterns fired, which clauses were emitted, and which were suppressed by the soundness gates. The detailed log is what allows a maintainer to understand why a particular method received the specification it did — why a clause was emitted, or, more often usefully, why an expected clause was not — and it is the first recourse when a regression test reveals that a change to one analyser altered another’s output.

The diagnostic value of the logging is most apparent at the interprocedural and compositional boundaries, where a method’s specification depends on others’ and a surprising result is most likely to have a non-local cause. When a call site fails to propagate an expected precondition, the log records whether the callee was resolved, whether its specification was cached, whether the substitution succeeded, and whether the result was already present — the same diagnostic breadcrumbs that the compositional measurement of Chapter 6 aggregates into its call-site resolution statistics. The instrumentation that supports day-to-day maintenance is thus the same instrumentation that supports the measurement: a count of how often each step succeeds or fails per call site is both a debugging aid and a datum, and the compositional study’s report of arity-match rates is, in effect, the detailed log summarised over a corpus.

This observability reflects the engine’s status as evolving research software. A heuristic engine whose pattern repertoire grows continually cannot be understood by reading its code alone, because its behaviour on a given method is the combined effect of many patterns interacting; the log is how that combined effect is made legible. The investment in diagnostics is, in this sense, of a piece with the two-tiered testing and the file-size discipline: all three are the software-engineering practices that let a heuristic engine remain understandable and correct as it accumulates the patterns that broaden its reach.

3.13 Summary

JML-Inferer infers JML preconditions, postconditions, loop invariants, and frame conditions from Java source by pattern matching organised around the weakest-precondition and strongest-postcondition calculi. It avoids symbolic WP/SP computation, which does not scale, and uses the calculi instead as a design discipline that fixes its pattern repertoire and clause-generation rules. It propagates specifications across method calls by single-pass procedure summarisation. It recovers the soundness that pattern matching does not guarantee by discharging every emitted clause through a forked, quantifier- ex-

tended OpenJML, treating inference and verification as two independent layers in which the first proposes and the second disposes. The chapters that follow put this output to work: Chapter 4 measures its value to a language-model test generator and reports its accuracy under both the manual and the formal oracle; Chapter 5 carries it across the source-to-binary boundary; and Chapter 6 measures and recovers the compositional information that the single-pass propagation of Section 3.7 leaves behind.

Chapter 4

Specifications for Language-Model Test Generation

The previous chapter described how specifications are produced. This chapter asks whether having them is worth anything, and answers the question against the most demanding contemporary consumer of code-level information: the large language model used to generate unit tests. The chapter is organised as a single empirical study with two methodologically independent components. The first establishes the *fidelity* of the inferred specifications — how accurately they capture intended behaviour, measured against both human judgment and the formal verifier — and answers RQ1. The second establishes their *value* — whether supplying them to a language model measurably improves the tests it writes — and answers RQ2. The two share a corpus but are otherwise independent observations, and keeping them separate is essential to the argument: the fidelity result tells us the specifications are good, and the value result tells us they help, and neither subsumes the other.

4.1 The Oracle Problem, Concretely

A unit test has a stimulus and an oracle. The stimulus drives the code into some state; the oracle asserts the state is correct. A language model generates stimuli fluently — given a method it readily produces inputs that exercise its branches — but the oracle requires a statement of *intended* behaviour that the code itself does not contain. Lacking such a statement, the model does the only thing it can: it infers the oracle from the implementation, and so asserts what the code does rather than what it should do [34]. When the implementation is correct this is harmless; when it is buggy the model encodes the bug as the expected result, producing a test that passes on broken code and will never catch the bug. Given only a signature, with no body to infer from, the model falls back to vacuous assertions — that the method does not throw, that its result is not null — which compile and pass but distinguish almost nothing.

The hypothesis of this chapter is that an inferred specification, supplied to the model as context, breaks this failure mode by carrying the missing statement of intent. The specification says what the method should do independently of what it does; a precondition delimits the valid inputs, a postcondition states the expected output, and the model can assert against these rather than against the implementation. The hypothesis is not obvious a priori. The specification is itself derived from the code, so one might object that it carries no information the model could not extract from the code directly. The objection is answered by the experiment’s design: the specification is roughly an order of magnitude more compact than the body, it states intent the body only implies, and — as the results show — it shifts the model’s output in a direction that source-code access does not fully replicate at equivalent input size.

The objection deserves a fuller answer, because it touches the conceptual heart of the study. It is true that the specification is derived from the code, and that in an information-theoretic sense it contains nothing the code does not. But the model is not an information-theoretic ideal; it is a bounded reasoner that attends to what is salient in its input, and the same fact can be salient in one representation and buried in another. A postcondition stating that a method returns the maximum of its arguments makes the intent salient in five words; the same intent is present in the method body but buried in a ternary expression the model must recognise, interpret, and generalise into a statement of intent before it can test against it. The specification does the recognising, interpreting, and generalising in advance, and presents the result in the form the model can act on directly. The value is not new information in the Shannon sense but *distilled* information — the same intent, extracted from the implementation and stated explicitly — and the distillation is exactly what a model lacking a reliable specification-production capability cannot do for itself, as the producer-side comparison confirms. The study measures the value of the distillation, and that the distillation helps even though it adds no information the code lacked is the precise sense in which a specification is more than a summary of the code: it is the code’s intent made salient.

4.2 Experimental Design

The experiment holds the model fixed and varies only its input, isolating the contribution of specifications. The same 312-method corpus is used throughout, and the model is queried under four conditions.

P1 — Signature only. The model receives the method signature alone. This is the baseline: the floor of what a model can do with no behavioural information beyond types.

P2 — Signature with guidance. The model receives the signature together with explicit instruction to cover edge cases. This condition is the crucial control, included

to test the alternative hypothesis that any prompt enrichment, not specifications in particular, would account for an effect.

P3 — Signature with specification. The model receives the signature together with the inferred JML specification. This is the treatment condition.

P4 — Full source. The model receives the complete method body. This is the upper bound: with the body in hand the model has the test answer directly, so P4 bounds the achievable quality from above and is not a realistic deployment condition.

The model is Google’s Gemini 2.0, queried at temperature 0.3 to balance diversity against consistency, with five independent runs per method per condition to control for sampling non-determinism. Prompts are kept minimal and differ across conditions only in the inclusion of the specification (P3) or the source (P4) or the guidance instruction (P2); no other prompt engineering is applied, so that any difference in output is attributable to the input variation rather than to prompt craft. The choice of a single model family is a deliberate budget decision — the experiment requires several thousand generations and was repeated during pilot tuning, totalling on the order of twenty-five thousand model invocations — and the threats-to-validity discussion (Section 4.9) addresses the single-model limitation directly. The defended contribution is the *ordering* of the conditions, not the absolute scores, and the ordering is expected to be robust across models on the evidence of multi-model studies in the literature [41, 46, 50].

4.2.1 Subject Selection

The corpus is drawn from Apache Commons Lang 3.14, chosen for its diversity of method types, its thorough Javadoc documentation (which enables the manual fidelity validation), its maturity, its deterministic pure functions (well suited to mutation testing), and its precedent in the research literature [23, 42]. Eleven classes were selected to cover five of the six method categories of Section 3.11: `NumberUtils`, `BooleanUtils`, and `CharUtils` (utility); `MutableInt`, `MutableDouble`, and `MutableBoolean` (state modification); `Pair` and `MutablePair` (factory/delegate); `Validate` and `Range` (control flow); and the `Mutable` interface. All public, package-private, and protected methods of these classes were included, with no method excluded after the fact, yielding 312 methods. The distribution across categories is shown in Table 4.1. The *other* category is empty, because Commons Lang contains no event handlers or callbacks — a structural feature of the corpus whose consequence for external validity is taken up in Section 4.9.

4.2.2 Metrics

Test quality is measured by four quantities. *Test count* is the number of test methods generated, averaged over the five runs. *Compilation rate* is the fraction of generated tests

Table 4.1: Distribution of the 312-method corpus by category. The *other* category contains zero methods because Apache Commons Lang includes no event-handler or callback methods.

Category	Count	%
Utility Methods	89	28.5%
State Modification	78	25.0%
Accessors & Mutators	62	19.9%
Control Flow	48	15.4%
Factory & Delegate	35	11.2%
Other (callbacks, event handlers)	0	0.0%
Total	312	100%

that compile. *Pass rate* is the fraction of compiling tests that pass against the unmutated code. *Mutation score*, the headline metric, is the fraction of seeded mutants the suite kills, computed with PIT [12] using its default operator set and a per-test timeout of ten seconds. Equivalent mutants are filtered by PIT’s default heuristics, with the residual rate of trivially equivalent mutants estimated at under five percent via the sampling procedure of Papadakis et al. [44]. The mutation score is the metric of record because it measures fault-detection power directly and penalises exactly the oracle-free test that covers a line without checking its behaviour; test count is reported alongside as a complementary metric, and the relationship between the two is itself a finding.

4.2.3 Statistical Analysis

For each metric the analysis reports mean, standard deviation, median, interquartile range, and 95% bootstrap confidence intervals over ten thousand iterations. Phase comparisons use paired *t*-tests, with Wilcoxon signed-rank tests as a non-parametric corroboration where within-phase distributions depart from normality, and Cohen’s *d* as the effect-size measure. Because the design yields multiple pairwise contrasts — six pairs of conditions across four metrics, twenty-four tests in total — a Bonferroni correction is applied at family-wise $\alpha = 0.05$, giving a per-test threshold of approximately 0.0021. Reported significant differences survive this corrected threshold. A power analysis based on a thirty-method pilot establishes that the design detects an effect of 2.2 percentage points at 80% power under the corrected threshold; the observed headline effect of 40.7 percentage points yields a post-hoc power exceeding 0.999.

4.2.4 The Experimental Infrastructure

The experiment is mediated by a test-generation harness that warrants description, because its design determines what the measurement can and cannot capture. The harness extracts each method from the corpus, constructs the prompt for the condition under test

— signature alone, signature with guidance, signature with specification, or full source — queries the model, parses the generated test code from the response, writes it into a test project, and records the outcome. The parsing step is more delicate than it appears: a model’s response wraps the test code in prose and markdown fences, and the harness must extract the code robustly across the variations in how the model formats its output. A parser too strict discards valid code that happens to be formatted unexpectedly; a parser too lax admits prose as code and fails to compile.

The harness’s handling of the model’s non-determinism is to run each condition five times per method and record each run independently, so that the per-method outcome is a distribution rather than a point. The downstream analysis aggregates these distributions, and the separation of generation from analysis — the harness produces logs, the analysis scripts consume them — is what makes the analysis reproducible even though the generation is not. The harness also copies the generated tests into a project configured for mutation testing, so that the same artefacts the harness produced are the artefacts PIT measures, with no manual intervention between generation and measurement that could introduce inconsistency.

The infrastructure’s design reflects a lesson that recurs in the distribution work: the steps a harness performs silently are the steps most likely to corrupt a measurement. The code-extraction step, in particular, can silently discard valid tests if the model’s formatting drifts, depressing the test count for reasons unrelated to the condition under test. The harness logs the raw response alongside the extracted code precisely so that an unexpectedly low test count can be traced to either a genuine failure of the model to generate tests or an extraction failure on tests it did generate — a distinction that matters for interpreting the results, and one the downstream probe of Chapter 6 would later need when its extraction encountered exactly this kind of drift.

4.2.5 Reproducibility and the Replication Package

An experiment with several thousand stochastic model invocations raises an obvious reproducibility concern, and the design addresses it on three fronts. First, every model invocation is logged with its prompt, its raw response, and the parsed test code, so that the path from input to measured outcome is auditable for any of the thousands of generations. Second, the analysis is driven by scripts that take the logged outputs as input and produce the reported tables and statistics deterministically, so that the step from raw generations to headline numbers is itself reproducible even though the generations are not. Third, the replication package parameterises the model client, so that the experiment can be re-run against a different model by changing a configuration value, and the analysis scripts accept a model identifier as an input column, which is the structure a multi-model replication would use.

The non-determinism that remains — the model’s sampling at temperature 0.3 — is

handled statistically rather than eliminated. Five runs per method per condition give a per-method mean whose standard error is small relative to the between-condition effect, and the bootstrap confidence intervals quantify the residual uncertainty without assuming a parametric form for the sampling distribution. The pilot study established that increasing from five to ten runs reduced the standard error by under four percent, which did not justify doubling the model cost; five runs is the point at which additional sampling buys negligible precision. The reproducibility claim is therefore not that a re-run would produce identical tests — it would not, the model being stochastic — but that a re-run would produce the same statistical conclusions, and that the path from any given set of generations to those conclusions is fully scripted and auditable.

4.3 Fidelity of the Inferred Specifications (RQ1)

Before measuring whether specifications help, the study establishes that the inferred specifications are accurate, under two independent oracles.

4.3.1 Manual Validation Against Documented Intent

Two authors independently validated the inferred specifications for all 312 methods against the Javadoc documentation, the source code, and the standard- library specifications, labelling each clause as correct, incomplete, incorrect, or overconstrained, with disagreements resolved by discussion. Inter-rater reliability was high — Cohen’s $\kappa = 0.91$ for preconditions and 0.87 for postconditions — which gives confidence that the labels reflect a stable judgment rather than individual idiosyncrasy. The results appear in Table 4.2.

Table 4.2: Manual validation of inferred specifications against documented intent, over all 312 methods.

Classification	Precond.	Postcond.	Overall
Correct	294 (94.2%)	273 (87.6%)	567 (90.9%)
Incomplete	13 (4.2%)	29 (9.3%)	42 (6.7%)
Incorrect	4 (1.3%)	7 (2.2%)	11 (1.8%)
Overconstrained	1 (0.3%)	3 (1.0%)	4 (0.6%)
Total	312	312	624

Precision is 94.2% for preconditions and 87.6% for postconditions. The lower postcondition precision is attributable chiefly to incompleteness for methods with complex return logic, where the engine captures part but not all of the returned relationship. Recall, measured against documented behaviour, is 89.3% for preconditions and 78.1% for postconditions; the lower postcondition recall reflects the difficulty of inferring relationships involving collection properties or algorithmic correctness, which the local syntactic patterns do not reach. The asymmetry between precondition and postcondition accuracy is

a consistent theme: preconditions arise from explicit guards that the engine recognises reliably, whereas postconditions must capture the full effect of a method’s logic, which is harder to do from syntax alone.

4.3.2 The Manual Validation Protocol

The manual validation deserves a fuller account, because the precision and recall figures rest on it and a reader should know how the labels were produced. Two authors independently examined the inferred specification for each of the 312 methods, with three reference materials in view: the method’s Javadoc documentation, which states the documented intent; the method’s source, which states the actual behaviour; and the standard-library specifications, which fix the behaviour of any library methods the body calls. Each clause was assigned one of four labels. *Correct* meant the clause agrees with the documented intent and the implementation. *Incomplete* meant the clause is correct as far as it goes but omits part of the documented behaviour — the common case for postconditions of methods with complex return logic. *Incorrect* meant the clause asserts something false. *Overconstrained* meant the clause is true but stronger than warranted — a precondition forbidding inputs the method actually handles.

The four-way labelling is more informative than a binary correct/incorrect split, because it separates the two ways a clause can fall short of perfect: by claiming too little (incomplete) or by claiming too much (incorrect or overconstrained). The distinction maps onto the precision/recall decomposition: precision is the fraction of emitted clauses that are correct, penalised by the incorrect and overconstrained labels, while recall is the fraction of documented behaviour captured, penalised by the incomplete label and by behaviours for which no clause was emitted at all. The two failure modes have different consequences downstream — an incorrect clause misleads a consumer, while an incomplete one merely leaves value on the table — which is why the thesis reports them separately rather than collapsing them.

Disagreements between the two raters were resolved by discussion, and the inter-rater reliability before resolution — Cohen’s $\kappa = 0.91$ for preconditions and 0.87 for postconditions — is reported as evidence that the labelling was reproducible rather than idiosyncratic. The slightly lower agreement on postconditions reflects their greater subtlety: whether a postcondition is “complete” relative to documented intent is a judgment call on which careful raters can differ, while whether a precondition correctly negates a guard is more nearly mechanical. The high agreement on both, and the resolution of the residual disagreements by discussion, is what licenses treating the manual labels as a ground truth against which the formal discharge can be compared as an independent oracle.

4.3.3 Formal Discharge Against the Implementation

Manual validation establishes agreement with documented intent; OpenJML’s Extended Static Checker establishes formal consistency with the implementation. The two oracles are independent, as Section 3.8 argued, and a clause may satisfy either without the other. Table 4.3 reports the discharge outcome over all inferred clauses.

Table 4.3: OpenJML ESC discharge over all inferred clauses. *Discharged* clauses are verified against the implementation; *flagged* clauses fail discharge and are surfaced for review; *unsupported* clauses involve JML constructs the verifier cannot yet handle.

Clause type	Inferred	Discharged	Flagged	Unsupported
Preconditions	312	296 (94.9%)	11 (3.5%)	5 (1.6%)
Postconditions	312	268 (85.9%)	22 (7.1%)	22 (7.1%)
Loop invariants	53	43 (81.1%)	4 (7.5%)	6 (11.3%)
Assignable	308	308 (100.0%)	0 (0.0%)	0 (0.0%)
Overall	985	915 (92.9%)	37 (3.8%)	33 (3.4%)

The discharge rate exceeds the manual-validation precision in every category, which is initially surprising but follows from the independence of the oracles: the verifier accepts clauses the human raters judged incomplete, because its oracle is consistency with the implementation, not completeness against the documentation. The 37 flagged clauses — chiefly subtle overflow-boundary postconditions on signed-integer methods, plus one loop invariant where the heuristic produced a non-decreasing rather than a monotonically decreasing variant — would have been false positives had they been retained silently; they are routed to a review queue and excluded from the specifications supplied to the model in P3. The 33 unsupported clauses are dominated by nested-quantifier postconditions involving `\sum` under conditional path conditions; they are recorded as not formally verifiable but retained for the P3 prompts because their content is syntactically valid JML and useful as oracle information to the model, even though it is outside the verified-sound surface. The residual non-discharge, examined in detail in Section 4.7, falls into three categorically identifiable tractability limits of the SMT solver rather than into arbitrary failures of the engine.

Why Discharge Exceeds Manual Precision

The discharge rate exceeding the manual-validation precision in every category is the single most informative relationship between the two oracles, and it deserves a careful reading because it would be paradoxical if the oracles measured the same thing. They do not. Manual precision asks whether a clause matches the documented *intent*; discharge asks whether a clause is consistent with the *implementation*. A clause can be consistent with the implementation while falling short of the documented intent — this is exactly the *incomplete* label of the manual validation, a clause that is true but says less than the

documentation promises. Such a clause depresses the manual precision (it is not fully *correct* against intent) while passing discharge (it is *consistent* with the code), and the gap between the two rates is, to a first approximation, the population of such incomplete-but-consistent clauses.

The relationship has a practical consequence for how the two oracles should be used together. Discharge cannot replace manual validation, because a clause that discharges may still be incomplete relative to intent, and a consumer relying on completeness — a verifier proving a strong property, a test generator seeking thorough coverage — needs the manual oracle’s completeness judgment that discharge does not provide. Manual validation cannot replace discharge, because a clause the raters judge correct against intent may still conflict with the implementation’s actual arithmetic or aliasing in a corner the documentation does not mention — the subtle overflow boundary that discharge catches and human review misses. The two are genuinely independent, catching disjoint classes of error, and the thesis runs both precisely because neither subsumes the other. The discharge-exceeds-precision relationship is the empirical proof of the independence: were the oracles measuring the same property, their rates would track, and the systematic gap shows they do not.

4.3.4 The Comparison Baselines in Detail

The comparison of the engine against Jdoctor and against direct language-model inference (reported below) deserves a fuller account of how the baselines were constructed, because a comparison is only as informative as the fairness of its baselines. Jdoctor was run on the same 312-method corpus in its standard configuration, extracting exception preconditions from the methods’ Javadoc. Its coverage is bounded by what the documentation states: a method whose Javadoc documents a thrown exception yields a precondition, and a method whose documentation is silent yields nothing, which is why Jdoctor produces specifications for 45.2% of the corpus against the engine’s 99.0%. The comparison is not a competition the engine wins outright, because Jdoctor recovers documented exceptional behaviour the engine misses — 25 exception preconditions encoded in natural language that no structural pattern reaches. The honest reading is complementarity: the two tools draw on different sources, documentation and code structure, and a combined workflow captures the union.

The language-model inference baseline was constructed by prompting the same model used in the test-generation experiment to emit JML directly from each method’s source, under the same five-run protocol. The construction is deliberately favourable to the model: it sees the full source, the richest possible input, and is asked only to produce the specification. Even so, its precision falls short of the engine’s — 81.3% against 94.2% for preconditions — because it hallucinates clauses that are plausible against the method name but false against the body, and its run-to-run consistency is 72.4% against the static engine’s 100%, because its sampling is stochastic. The comparison isolates the value of

grounding: the model and the engine see the same code, but the engine’s output is determined by the code’s structure while the model’s is a prediction, and the prediction is less precise and less stable. This is the empirical basis for the thesis’s producer-consumer asymmetry, developed in the synthesis of Chapter 7: the model is the weaker producer here and the stronger consumer in the main experiment, and the two findings together motivate the architecture that uses each where it is strong.

4.3.5 Comparison With Other Inference Approaches

Table 4.4 compares the engine with Jdoctor and with direct language-model inference (the same Gemini model prompted to emit JML from source).

Table 4.4: Comparison with Jdoctor and language-model specification inference.

Metric	JML-Inferrer	Jdoctor	LLM (Gemini 2.0)
Methods with specs	309 (99.0%)	141 (45.2%)	312 (100%)
Precision (precond.)	94.2%	92%	81.3%
Recall (precond.)	89.3%	83%	88.7%
Precision (postcond.)	87.6%	—	74.8%
Recall (postcond.)	78.1%	—	82.1%
Consistency (5 runs)	100%	100%	72.4%

The engine produces non-trivial specifications for 99.0% of the corpus, against Jdoctor’s 45.2% — Jdoctor requires documented exceptions and is silent where they are absent. The two are complementary: Jdoctor captures 25 exception preconditions the engine misses, typically encoded in natural language, while the engine captures 37 the Jdoctor misses, undocumented in Javadoc. Against direct language-model inference, the engine attains markedly higher precision (94.2% versus 81.3%) and perfect run-to-run consistency (100% versus 72.4%), because its proposals are grounded in concrete code structure whereas the model may hallucinate plausible-but-false clauses and varies across runs. The comparison is the empirical basis for the methodological caution stated in Chapter 2: a language model is a strong consumer of grounded specifications and a weaker producer of them.

4.4 Value of Specifications for Test Generation (RQ2)

With the fidelity of the specifications established, the study turns to whether supplying them improves test generation. Table 4.5 reports the per-class test counts and mutation scores for each condition, with overall quality metrics in the lower panel.

4.4.1 The Principal Finding

Specification-guided generation (P3) produces 272% more tests than signature-only generation (P1) — 95% bootstrap confidence interval [245, 299], paired t -test $p < 0.001$, Cohen’s

Table 4.5: Test-generation results by class: per-method average test counts over five runs, and mutation scores, for each condition. The lower panel reports overall quality metrics as the unweighted mean across all 312 methods.

Class	P1: Sig		P2: Sig+Guide		P3: Sig+Spec		P4: Source	
	Tests	Mut.%	Tests	Mut.%	Tests	Mut.%	Tests	Mut.%
NumberUtils	12	28	48	78	41	68	65	92
BooleanUtils	10	31	45	82	38	71	62	95
CharUtils	9	30	41	80	35	70	58	93
MutableInt	10	26	40	89	36	69	59	100
MutableDouble	8	23	35	85	32	65	52	97
MutableBoolean	7	25	33	84	30	66	50	96
Pair	8	29	36	83	33	70	53	95
MutablePair	8	27	34	82	31	68	51	94
Validate	8	35	32	81	29	74	48	94
Range	6	22	28	76	25	62	42	91
Mean (per method)	9.0	27.5	38.0	81.8	33.5	68.2	54.7	94.8
<i>Test Quality Metrics (overall, 312 methods)</i>								
Compile Rate	89.2%		87.8%		91.5%		93.2%	
Pass Rate	94.1%		92.3%		96.8%		97.1%	
Valid Tests	83.9%		81.1%		88.6%		90.5%	

$d = 2.41$, confidence interval $[2.08, 2.79]$ — and achieves a mutation score 40.7 percentage points higher: 68.2% against 27.5%, paired t -test $p < 0.001$, Cohen’s $d = 3.07$, confidence interval $[2.71, 3.46]$. The effect is large by any conventional standard and survives the Bonferroni correction comfortably. Full source access (P4) attains the highest absolute mutation score, 94.8%, as expected when the model is handed the test answer directly, but the comparison that matters is not P3 against P4 in absolute terms; it is the comparison in *input density*. A JML contract typically occupies five to twelve lines of annotation, against the thirty to one hundred fifty lines of body that P4 supplies. Within that order-of-magnitude smaller input, P3 captures roughly sixty percent of the achievable gain — $(68.2 - 27.5)/(94.8 - 27.5)$ — and the residual gap to P4 is largely accounted for by the body containing the answer outright. P3 also attains higher compilation and pass rates than P1 or P2, evidence that specifications constrain the model toward syntactically and semantically valid test code rather than merely toward more of it.

The input-density argument deserves to be stated as a principle, because it is the form in which the result generalises beyond the particular numbers. The value of an input to a test generator is not its size but its information content about intended behaviour per unit of size. A signature is small and carries little such information; a body is large and carries much, but most of it is implementation detail rather than intent, and crucially it carries the test answer, which makes it an upper bound rather than a realistic input. A specification is small and carries information that is almost entirely about intent — it is the distillation of the body’s behaviour into the contract the body honours. The sixty percent of the achievable gain that P3 captures within an input an order of magnitude

smaller than P4’s is the quantitative expression of this density: the specification is the information-dense input, and its density is why it captures most of the benefit at a fraction of the size. This reframes the comparison with the source baseline: P4 wins on absolute score because it contains the answer, but it does so with an input that no deployment would use, and the practical comparison is between the signature a deployment always has and the specification a deployment can cheaply infer, where the specification’s density is decisive.

4.4.2 Why More Tests Are Not Better Tests

The most instructive comparison in the study is P2 against P3. The guidance condition P2 produces the *highest* per-method test count (38.0, against P3’s 33.5) yet a mutation score 13.6 percentage points *lower* (68.2% against 81.8% — the relationship inverts when measured by mutation rather than count). This is the result that the guidance control was designed to surface, and it refutes the alternative hypothesis that any prompt enrichment would explain the P3 effect: enrichment with edge-case guidance produces more tests but weaker ones. Inspection of representative P2 outputs identifies three mechanisms. First, the edge-case instruction prompts the model to enumerate near-duplicates — separate cases for 0, -0, `Integer.MIN_VALUE`, and `Integer.MIN_VALUE + 1` — that inflate the count without expanding the set of killed mutants. Second, in the absence of an oracle, edge-case tests default to weak assertions (`assertNotNull`, `assertDoesNotThrow`) that compile and pass but fail to distinguish mutants that alter return values. Third, P3 tests are concentrated on clauses — a test or two per inferred precondition and postcondition — producing higher mutant-kill density per test. The same pattern appears in the quality metrics: P2 has the lowest compilation and pass rates of the four conditions, consistent with the model generating more but less-constrained code. The finding is that *quantity of tests is a poor proxy for oracle quality*, and it is the strongest single argument in the study for specifications over generic prompting.

4.4.3 Specification Strength and Its Consequences

Underlying the per-category effect is a distribution of specification *strength* that the analysis tracks and that explains part of the variation. A specification is classified as *strong* when both its precondition and its postcondition are non-trivial, *partial* when exactly one side is non-trivial, and *weak* when neither is. The classification matters because a strong specification gives the model two sources of oracle information — what inputs are valid and what outputs to expect — while a partial one gives one and a weak one gives little more than the signature already did.

The strength distribution varies by category in a way that parallels the test-generation effect. Accessor and factory methods achieve the highest proportion of strong specifications, because their behaviour — read a field, construct an object — is fully captured by

a postcondition and any input constraint by a precondition. State-modification methods have the lowest, because capturing the complete frame condition for a method that mutates object state under conditional logic is the hardest inference task, and a method whose state change the engine captures only partially yields a partial rather than a strong specification. The categories with the strongest specifications are not, however, the categories with the largest test-generation gains, and the apparent tension resolves instructively: accessors have strong specifications but small gains because their signature-only baseline is already high, while control-flow methods have moderate specification strength but large gains because their signature-only baseline is low and the specification’s guarded postconditions are exactly what the baseline lacks. The gain is the product of specification strength and baseline weakness, not of strength alone, which is why the category with the most to gain — control flow, with weak baselines and informative guarded postconditions — shows the largest improvement despite not having the strongest specifications.

This decomposition refines the headline claim once more. Specifications help in proportion to the oracle information they add over the baseline, and that information is the product of how much the specification states (its strength) and how little the signature already implied (the baseline’s weakness). The categories sort themselves along this product, and the per-category results are its manifestation.

4.4.4 Where Specifications Help Most

The benefit of specifications varies systematically across method categories. Control-flow methods benefit most, with a mutation-score improvement of 47.1 percentage points from P1 to P3, because the specification delineates the expected behaviour of each branch and the model can target each guarded postcondition with a directed test. Utility methods improve by 40.6 points, their complex input constraints captured well by inferred preconditions; state-modification methods improve by 36.6 points, benefiting from postconditions that state their state changes. Accessors and mutators improve least, by around 19 points, not because specifications fail to help but because their signature-only baseline is already near the achievable ceiling — a getter is easy to test from its signature alone. Within the looping methods, those for which the engine inferred a non-trivial loop invariant attain a P3 mutation score of roughly 72% against roughly 54% for those left with only a weak invariant, a contrast that confirms the importance of the loop-invariant heuristics of Section 3.5 for downstream test quality.

The loop result deserves emphasis because it traces the value of one of the engine’s hardest inference tasks directly to a downstream outcome. A loop invariant is the single most difficult clause the engine infers, requiring the heuristic machinery of Section 3.5 where the other clause kinds need only pattern correspondence, and one might reasonably ask whether the effort is repaid. The eighteen-point gap between the well-invariant and weak-invariant looping methods answers that it is: a method whose loop the engine char-

acterises with a non-trivial invariant yields a specification whose promoted postcondition states what the loop computes, and a model given that postcondition tests the loop's result directly, while a method left with only a weak invariant yields a specification that says little about the loop's outcome and a model that tests it weakly. The downstream consequence of the loop-invariant heuristics is thus measurable in the mutation score, and it justifies the disproportionate machinery the engine devotes to loops: the hardest inference task produces, when it succeeds, the clauses that most improve the tests, and when it fails produces the methods where P3 most underperforms its potential. This is the clearest instance in the thesis of an inference capability's value being traced through to a downstream metric, and it is the reason loop-invariant strengthening is named among the most consequential directions for future work.

4.5 A Closer Look at Individual Classes

The per-class figures of Table 4.5 repay individual attention, because the classes differ in ways that explain the spread of the effect. The utility classes `NumberUtils`, `BooleanUtils`, and `CharUtils` show the largest absolute test counts and large mutation-score gains from P1 to P3. These classes are dense with conversion and parsing methods — turning strings into numbers, characters into integers, booleans into their string forms — and such methods have rich input-validity preconditions and precise output postconditions that the engine captures well. The inferred specification for a parser states which inputs are valid and what each valid input yields, and the model turns each clause into a targeted test, which is why these classes see both many tests and high mutation scores under P3.

The mutable wrapper classes `MutableInt`, `MutableDouble`, and `MutableBoolean` are interesting for the opposite reason: their methods are simple state mutations — increment, add, set — and their P1 mutation scores are already moderate because a getter or setter is easy to test from its signature. Their P3 gains are real but smaller, and concentrated in the methods whose postconditions relate the new state to the old via `\old` — an `addAndGet` whose contract states the result equals the old value plus the argument. The `\old`-bearing postconditions are exactly the clauses a signature cannot imply, and they are where the wrapper classes' P3 improvement concentrates. `MutableInt` reaching a 100% mutation score under the source condition P4 reflects the simplicity of its methods: with the body in hand the model tests them exhaustively.

The factory and value classes `Pair` and `MutablePair` test the engine's handling of construction and delegation. Their factory methods carry the postcondition that the result is non-null and that its components equal the arguments, and their accessors carry the postcondition that they return the stored component; the model turns these into construction-and-readback tests that kill the return-value mutants a signature-only suite would miss. The control-flow classes `Validate` and `Range` are where specifications help most in relative terms: `Validate`'s methods are pure guards whose entire behaviour is

a precondition-check-and-throw, so the inferred preconditions *are* the specification, and **Range**'s methods are dense with boundary conditions whose guarded postconditions give the model a test per boundary. These classes' large P1-to-P3 gains are the per-class manifestation of the category result that control-flow methods benefit most.

The reading reinforces the central finding from a different angle. Across the classes, the size of the P3 improvement tracks the gap between what a method's signature reveals and what its specification states: largest for the parsers and guards whose behaviour is invisible in their signatures, smallest for the accessors whose behaviour their signatures already imply. The specification's value is, consistently, the value of the information it adds over the signature, and that value varies by class exactly as the information gap does.

4.6 Where the Mutants Are Killed

The mutation score is an aggregate over a heterogeneous population of seeded faults, and decomposing it by mutation operator reveals *which kinds* of fault specifications help catch. The PIT default operator set seeds faults of several kinds: conditionals-boundary mutations replace a strict comparison with its non-strict variant; negate-conditionals mutations invert a boolean condition; math mutations swap a binary arithmetic operator; increments mutations flip an increment to a decrement; and a family of return-value mutations replace a method's return with a default, a negated, or a substituted value. Each kind stresses a different aspect of a test suite's oracle.

The operators that specifications help most are exactly those whose faults change *what a method returns* rather than *whether it runs*. A return-value mutation that makes a method return zero where it should return its computed result survives any test that does not assert the specific expected value — and the signature-only and guidance conditions, lacking an oracle, rarely assert the specific value, so they leave these mutants alive. The specification condition, carrying postconditions that state the expected result, generates tests that assert it, and kills the return-value mutants directly. The same holds for conditionals-boundary and negate-conditionals mutations on methods with guarded postconditions: the inferred guarded clauses tell the model which input should produce which result, so a mutation that shifts a branch boundary or inverts a guard produces an observably wrong result on a test the specification motivated. This is the operator-level mechanism behind the category result of Section 4.4: control-flow methods improve most because their faults are predominantly conditional-boundary and negate-conditional faults, which are exactly the faults the inferred guarded postconditions equip the model to catch.

The operators that specifications help least are those whose faults are caught by mere execution. A void-method-call mutation that removes a call with a side effect is often caught by any test that exercises the path, because the missing side effect changes ob-

servable state regardless of the oracle’s strength; specifications add little here, because the signature-only baseline already kills these mutants. The decomposition thus refines the headline result: specifications do not uniformly raise the mutation score, they raise it specifically on the return-value and conditional faults that an oracle is required to catch, and they leave the execution-detectable faults to the coverage that any condition achieves. The aggregate 40.7-point improvement is concentrated in the oracle-dependent operators, which is precisely where the oracle problem predicts it should be.

4.7 Discussion

4.7.1 What the Two Results Together Establish

The fidelity result and the value result are independent, and their conjunction is the chapter’s contribution. Fidelity alone would establish that the engine infers accurate specifications but not that anyone should care; value alone would establish that specifications help test generation but leave open whether *inferred* specifications, with their imperfections, help as much as hand-written ones would. Together they establish the full claim: the engine infers specifications that are accurate under both a human and a formal oracle, and supplying those specifications — imperfections included — to a language model improves its test generation by a large and statistically robust margin. The improvement is achieved with an input an order of magnitude more compact than the source and is not replicated by generic prompt enrichment, which isolates the specification as the active ingredient.

4.7.2 The Three Tractability Limits

The 7.1% of clauses that OpenJML does not discharge are not arbitrary failures; they fall into three categorically identifiable patterns that exceed the tractability budget of the solver at the project’s strictness configuration. The first is *recursive multiplicative growth*: the solver’s linear- arithmetic core cannot unfold the inductive product needed to bound a factorial, a Fibonacci recurrence, or a power function, so the engine emits a literal parameter bound (a small-input contract) that discharges while the unbounded form remains unprovable. The second is *quantified invariants over mutable arrays*: an in-place array scaling loop requires the solver to preserve a universally quantified element bound across an array store, and three formulations each exceeded the per-clause time budget. The third is *the for-each translation*: in a for-each accumulator the loop variable is bound from the array internally, but the relationship between the variable and the indexed element is not exposed to user JML, so the per-step overflow check cannot reference the precondition’s element bound. These are properties of the SMT solver and the JML translation at the present strictness, not of the engine’s heuristics; they delimit what verification technology can currently certify, not what the engine can emit, and naming them precisely is more useful than reporting an undifferentiated failure rate.

4.7.3 The Tractability Limits, With Examples

The three tractability limits warrant concrete illustration, because the distinction between a clause that is false and a clause that is true but unprovable is central to reading the discharge results correctly. The first limit, recursive multiplicative growth, is exemplified by a factorial method whose postcondition would relate the result to the product of the integers up to the argument. The solver’s linear-arithmetic core cannot unfold the inductive product — it would need to reason that the result of the recursive call, multiplied by the argument, satisfies a bound, and the multiplication of a variable by a variable is outside linear arithmetic. The engine’s response is to emit a literal parameter bound — the argument must be small enough that the factorial does not overflow — which discharges as a small-input contract while the unbounded relationship remains unprovable. The clause the engine emits is true and provable; the clause it would like to emit is true but beyond the solver.

The second limit, quantified invariants over mutable arrays, is exemplified by an in-place array scaling loop that multiplies each element by a factor. The postcondition would state that every element equals its old value times the factor, and the loop invariant would state that this holds for the prefix processed so far. Preserving the universally quantified invariant across the array store at each iteration requires the solver to reason that writing one element leaves the bound on the others intact — a property of the array theory that the solver supports in principle but that, combined with the quantifier, exceeds the time budget in practice. Three formulations were attempted, each timing out, which is the evidence that the limit is the solver’s and not a missing formulation the engine could supply.

The third limit, the for-each translation, is the most purely a verifier artefact. In a for-each loop over an array, the loop variable is bound internally from the array at the iteration index, but the relationship between the variable and the indexed element is not exposed to user-level JML — the variable simply appears, disconnected from the array in the specification namespace. A per-step overflow check on the accumulator therefore cannot reference the precondition’s bound on the array element, because it cannot name the array element the loop variable came from. This is a gap in the JML translation of the for-each construct, not in the engine’s inference or the solver’s power, and it is the case the thesis classifies as a structural limit of the toolchain rather than of the technique. The three together account for the bulk of the non-discharged postconditions, and naming them precisely converts an undifferentiated failure rate into a map of exactly where the verification frontier lies.

4.7.4 Implications

The results have practical implications beyond the test-generation setting. That the engine produces useful specifications for 99% of the corpus at 88 milliseconds per file makes

integration into continuous-integration pipelines, pre-commit hooks, and editor tooling practicable. A concrete workflow makes the implication tangible. A continuous-integration job runs the engine on every pull request, infers specifications for the changed methods, discharges them through the verifier, and reports the flagged clauses as review comments alongside the diff — so that a developer reviewing a change sees not only the code but the contracts the change implies and any clauses that failed to discharge against it. A clause that fails discharge on a changed method is a signal that the change may have violated an implicit contract, surfaced at exactly the moment a reviewer is looking at the code. The 88-millisecond-per-file throughput is what makes this viable: the inference disappears into the build, and the developer pays no perceptible cost for the contracts.

The editor-integration scenario is analogous at a finer grain. An editor that runs the engine as the developer types can display the inferred contract of the method under the cursor, turning the implicit behaviour the developer is writing into an explicit contract they can check against their intent — and flagging the moment the code and the inferred contract diverge from what the developer meant. These scenarios are not evaluated in this thesis, which measures the test-generation consumer specifically, but they follow from the same properties — accurate specifications produced fast — and they indicate that the test-generation result is one instance of a broader applicability the engine’s speed and accuracy enable. The complementarity with Jdoctor suggests a combined workflow that draws preconditions from both code structure and documentation. And the central finding — that compact behavioural specifications shift a language model’s output toward genuine behavioural checks — generalises in principle to any model and any JML-aware consumer, though confirmation across models and codebases remains future work and is named as a threat below.

A final implication concerns the relationship between this study and the rest of the thesis. The study measures value to one consumer, a language-model test generator, chosen because it is the most demanding and most contemporary consumer of code-level intent. But the value the study measures is not specific to that consumer; it is the value of the oracle information the specification carries, and any oracle-dependent consumer — a search-based test generator given the specification as assertion oracles, a verifier given it as the contract to check, a developer given it as documentation to read — draws on the same information. The language model is the study’s instrument for measuring that value precisely, because its output can be scored by mutation testing, but the value is a property of the specification, not of the model. This is why the distribution study of Chapter 5 matters: the value the model demonstrates is available to every consumer the distribution mechanism can reach, and the test-generation result is best read not as a fact about language models but as a measurement, via language models, of how much intent an inferred specification carries.

4.8 Qualitative Analysis of Generated Tests

The aggregate metrics establish that specifications help; inspecting the generated tests shows *how*. Consider a representative method, a numeric parser that returns a default on invalid input:

```
public static int toInt(String str, int defaultValue) {  
    if (str == null) { return defaultValue; }  
    try { return Integer.parseInt(str); }  
    catch (NumberFormatException e) { return defaultValue; }  
}
```

Under the signature-only condition (P1) the model, lacking any statement of intended behaviour, generates tests that exercise the obvious happy path and assert weakly. A typical P1 test passes "42" and asserts the result is 42, and a second passes `null` and asserts the result equals the default — both correct, but both reconstructible from the method name alone, and neither probing the catch path. The P1 suite covers the method's lines while leaving its behaviour largely unchecked, which is why its mutation score is low: a mutant that changed the default-return to return zero would survive every test that happened to pass zero as the default.

Under the guidance condition (P2) the model, instructed to cover edge cases, generates many more tests — empty string, whitespace, leading-plus, `Integer.MAX_VALUE`, overflow strings, embedded letters — but in the absence of an oracle it asserts that these “do not throw” or returns “some int”, because it has no statement of what each input *should* produce. The extra tests inflate the count and the coverage but kill few additional mutants, because their assertions do not distinguish a correct result from a plausible wrong one. This is the mechanism behind the headline P2-versus-P3 inversion: P2 generates the most tests and a lower mutation score, because quantity without an oracle is motion without progress.

Under the specification condition (P3) the model receives the inferred contract — the guarded postconditions stating that a null or unparseable string yields the default and a parseable string yields its integer value — and generates tests that assert against those postconditions directly. A P3 test passes "`notanumber`" and asserts the result equals the default *value supplied*, not merely that it does not throw; another passes a valid string and asserts the exact parsed result. These assertions kill exactly the mutants that P1 and P2 miss, because they encode what the method should do rather than what it happens to do. The specification's clause-by-clause structure also shapes the suite: the model produces roughly one test per guarded postcondition, yielding a suite that is smaller than P2's but denser in mutant-killing power.

The pattern generalises across categories. For control-flow methods the guarded postconditions give the model a test target per branch, which is why control-flow methods show the largest P1-to-P3 improvement. For accessors the specification adds little the sig-

nature did not already imply, which is why their improvement is smallest. The qualitative inspection thus explains the category-level quantitative result of Section 4.4: specifications help most where the gap between what a method does and what its signature reveals is largest, and least where the signature already determines the behaviour.

4.9 Threats to Validity

Internal validity. Sampling non-determinism is mitigated by five independent runs per method per condition and by statistical aggregation at low temperature. Prompt confounding is addressed by minimal prompts differing only in the controlled variation and by the P4 control. Subjectivity in manual validation is reduced by two independent raters with explicit criteria and high inter-rater reliability.

External validity. The corpus is a single, mature, well-structured library, which may favour the approach, and it lacks the *other* category of event handlers and callbacks. Results may not transfer to enterprise applications or domain-specific libraries, and confirmation across heterogeneous codebases is future work. The model has likely encountered Commons Lang in training, but the consistent P1-to-P3 gain is inconsistent with simple training-set recall: were the model merely recalling memorised tests, P1 would already approach P4, which it does not (27.5% against 94.8%). A single model family was used for budget reasons; the defended contribution is the ordering $P1 < P2 < P3 < P4$, which multi-model studies in the literature suggest is not model-specific, but a full multi-model replication at this scale is left to future work.

Heuristic soundness. Inference is heuristic and could in principle emit malformed or false clauses even after manual review. This is mitigated by discharging every clause through OpenJML and by the regression suite that gates engine changes against the verifier; flagged clauses are reviewed rather than retained. The manual-validation precision and the formal-discharge rate are reported separately precisely because they are independent oracles catching different errors.

Construct validity. Mutation testing has known limitations, principally equivalent mutants and operator representativeness. The PIT default operator set is used, equivalent mutants are filtered, and test count is reported as a complementary metric. The inversion of the count and mutation-score ordering between P2 and P3 is itself evidence that the mutation score measures something the count does not.

Conclusion validity. The principal contrast is well powered: with 312 paired observations and the within-pair variance estimated from a pilot, the design detects 2.2 points

at 80% power under the Bonferroni-corrected threshold, and the observed 40.7-point effect yields post-hoc power exceeding 0.999. All reported significant differences survive the correction across the family of twenty-four contrasts.

4.10 The Generalisation to Service-Boundary Code

The corpus of this study is pure library code, where the oracle problem is real but not at its most severe, and it is worth setting out explicitly why the setting where the problem bites hardest — service-boundary code — is both the natural next target and one for which the present result is a lower bound rather than an upper one. A method at a service boundary calls a remote endpoint or an RPC stub: its body delegates to an external service, its signature conveys nothing about the service’s status codes, error semantics, or retry behaviour, and its source, even when available, reveals only the call, not the contract the remote service honours. For such a method, the source-code condition P4 — the upper bound in this study — loses its advantage, because the source does not contain the answer; the answer lives in the remote service’s behaviour, which the source does not describe.

This changes the structure of the argument in a way that favours specifications. In the present study, P4 dominates P3 because the source contains the test answer outright; at a service boundary, the source does not, so the gap between P3 and P4 should narrow or close, and the inferred or documented specification of the service’s contract — carried, as Chapter 5 shows, in the service’s OpenAPI description — becomes the only source of oracle information that signature, source, and documentation do not already encode. The hypothesis for the planned follow-up is therefore that the P1-to-P3 gain reported here will *compound* at the service boundary: where the source is uninformative about intended behaviour, the specification is not merely a more compact alternative to the source but the sole carrier of the oracle, and signature-only generation collapses into mocking the call with no behavioural check at all. The present study establishes the effect in the setting where the source competes with the specification; the follow-up will test it in the setting where the source cannot.

Chapter 5

Distributing Inferred Specifications

The two preceding chapters established that specifications can be inferred with acceptable fidelity and that they carry measurable value to a downstream consumer. Both rest on an assumption that is rarely satisfied in practice: that the consumer has the source tree the inferrer ran over. This chapter removes the assumption. It develops and evaluates a mechanism for carrying inferred JML contracts across the boundary that separates source code from compiled artefact — into Java class files and into REST service descriptions — so that a consumer with only a JAR or only an HTTP interface can recover the specification the inferrer produced. The chapter answers RQ3.

5.1 The Distribution Gap

Recent work, including the study of Chapter 4, has shown that inferred JML specifications materially improve downstream tooling. That work shares an unstated assumption: the consumer of the specification — the test generator, the integrated development environment, the verifier, the language model reading an API at inference time — has access to the same source the inferrer processed. The assumption is rarely true. Java libraries are distributed as JAR files containing compiled bytecode, not source. REST endpoints expose only their HTTP signatures. gRPC services expose only their protocol-buffer descriptors. Source code is the exception in a consumer’s world, not the norm. A specification that lives only in the source tree is a specification available only to the team that owns the source — which is precisely the team least in need of it, since they can read the code.

The conceptual question of whether specifications *should* travel with compiled artefacts has a long history in the Design by Contract and JML traditions [35,36,40]. The practical question this chapter addresses is narrower and, in a sense, more demanding: can the *standard, unmodified* Java and REST toolchain be made to carry specifications, so that

enrichment requires no change to how artefacts are built, signed, published, or consumed? A mechanism that required a custom class loader, a patched compiler, or a bespoke service framework would not be adopted; a mechanism that rides inside the existing artefact formats might be. The research questions are accordingly framed around losslessness, overhead, throughput, and toolchain compatibility.

RQ3a (Losslessness). Can inferred JML contracts be embedded in compiled Java artefacts and reproduced byte-for-byte from the embedded bytes, with no source dependency?

RQ3b (Overhead). What is the bytecode-size cost of the embedding, and how does it scale across libraries of different shapes?

RQ3c (Throughput). Is embedding and reading fast enough to support per-pull-request continuous-integration workflows on production-scale libraries?

RQ3d (Compatibility). Does an OpenAPI extension family integrate with existing OpenAPI tooling without modification?

5.1.1 Why Existing Channels Fall Short

Section 2.10 reviewed the existing channels; the gap each leaves is worth restating concretely here. JML specification files (`.jml` stubs) are the verifier’s native format but travel as separate sidecar files, absent from the compiled JAR and invisible to any tool that does not look for them. Runtime assertion checking compiles specifications into executable checks inside the bytecode, carrying them into the artefact but only in executable form, restricted to runtime-evaluable properties, and at the cost of altering the program’s behaviour. The documentation field of an OpenAPI description carries prose for a human, not a structured contract for a tool. None of these carries a structured, behaviour-preserving, toolchain-transparent specification inside the standard artefact that any reader can recover without the source. That is the gap this chapter fills, on both the bytecode side and the service- interface side.

5.2 Format Design

The format is the contract between the embedder and any future consumer; it must be precise enough that an alternative embedder or reader could be written from the specification alone. The bytecode side uses a runtime-retained Java annotation; the service side uses an OpenAPI extension family; a sidecar JAR covers the cases neither can.

5.2.1 The Annotation Type

The embedded specification is carried by a single annotation per program element, retained at runtime and targetable at methods, constructors, fields, and types:

```
package com.jml.spec;

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.METHOD, ElementType.CONSTRUCTOR,
        ElementType.FIELD, ElementType.TYPE})
public @interface JmlSpecs {
    String[] requires()      default {};
    String[] ensures()       default {};
    String[] assignable()   default {};
    String[] loopInvariant() default {};
    /** Each entry is "ExceptionType/condition". */
    String[] signals()       default {};
    String version()         default "2";
}
```

A single annotation carries every clause kind as a separate string array. Within a member, array order is significant for **requires**: preconditions are emitted in source order, which is load-bearing for well-definedness chains, since a precondition asserting an array element's value depends on earlier preconditions establishing the array's non-nullness and the index's in-boundsness. Unrecognised members are skipped on read, which preserves forward compatibility: a future clause kind added as a new member is ignored by an old reader rather than breaking it.

5.2.2 Three Size Optimisations

A naive encoding — one annotation per clause, every clause spelled out in full — is wasteful, and the format applies three optimisations whose justification is empirical, grounded in the shape distribution of real inferred specifications.

The first is the *single packed annotation*. An earlier format version used Java's repeatable-annotation mechanism, emitting one annotation per clause with per-clause kind, text, order, and version members wrapped in a container. This is the maximally verbose shape, paying the per-clause annotation framing — a type reference, an element-value-pair structure, an end marker — for every clause. The current format collapses this to one annotation per element with the clause kinds as array members, eliminating the per-clause framing entirely.

The second is *default omission for the frame condition*. Every inferred specification in the corpus carries **assignable \nothing**, the engine's default for methods that appear pure. The format omits the **assignable** member when its only entry is **\nothing** and interprets an absent member as **\nothing** on read, saving one array entry per specification. A specification with genuine frame clauses lists them explicitly.

The third is a *token dictionary*. A twelve-entry dictionary substitutes the most common JML tokens — `\result`, `\old`, the quantifiers, the implication arrow, the null comparisons, the frame keywords — with single Unicode control bytes that cannot appear in legitimate JML, so the substitution is unambiguous and invertible. Because UTF-8 encodes each control byte as a single byte, each token occurrence saves the token’s length minus one. The reader decodes on read; a consumer ignorant of the dictionary sees the control characters and can either decode using the published table or fall back to the sidecar.

The choice of which tokens to encode and which clause to default is not arbitrary; each rests on the shape distribution measured on the real-inference corpus (Section 5.6). A format designed without the measurement would either include every JML keyword in the dictionary, paying constant-pool cost for tokens that never fire, or omit tokens that occur frequently in real specifications. The divergence between the optimisation’s effect on synthetic and on real specifications, reported below, is itself evidence that the saving depends on which clause shapes the consumer’s inferer produces.

5.2.3 Canonical Form

To support byte-level roundtrip equality, clause text is canonicalised before emission. Whitespace runs collapse to single spaces; there is no space around member-access or method-reference operators; binary operators take a single surrounding space; there is no space inside brackets; and the short implication and equivalence arrows are normalised to their long forms. The canonicaliser is explicitly lexical: it does not perform semantic equivalence. Two clauses that differ in operand order remain distinct strings, and roundtrip equality is preserved on the wire form rather than on a normalised theory level. This is a deliberate choice. Semantic equivalence would silently rewrite valid specifications in transit, which is unacceptable when the embedder’s job is to carry data faithfully; a consumer that wants semantic normalisation can apply it after recovery, but the transport layer must not.

5.2.4 The OpenAPI Extension and the Sidecar

For service boundaries, a parallel extension family carries the same content in an OpenAPI 3.x document: `x-jml-requires`, `x-jml-ensures`, `x-jml-assignable`, `x-jml-signals`, and `x-jml-invariant`, each an array of canonical-form strings, carried per operation and per schema. The `x-` prefix is the OpenAPI mechanism for vendor extensions, which standard parsers preserve without understanding; the extension therefore rides through the existing OpenAPI toolchain unmodified. A JSON Schema document accompanies the artefact for independent validation, and where the OpenAPI document is generated by a framework and not editable, a sidecar JSON file carries the same content keyed by route and verb.

For specifications that exceed the per-class bytecode budget — rare, but possible for a method with many quantified loop invariants — and for signed JARs that cannot be re-signed, the embedder emits a parallel artefact carrying JML stub files keyed by class internal name, with a manifest entry recording the source JAR’s hash so consumers can detect drift. The stub files are the format OpenJML reads natively, so the sidecar is consumable by the verifier without further transformation. The sidecar is the universal fallback: where in-bytecode embedding is unsuitable, the same content travels alongside.

5.2.5 The OpenAPI Extension in Depth

The service-side extension warrants a fuller treatment, because a service boundary differs from a library boundary in ways that shape the design. A library’s contract attaches to a method, identified by its name and bytecode descriptor; a service’s contract attaches to an *operation*, identified by a route and an HTTP verb, and to the *schemas* of the data it exchanges. The extension therefore carries contracts at two levels: per-operation, where **x-jml-requires** states the preconditions on a request and **x-jml-ensures** the postconditions on a response, and per-schema, where **x-jml-invariant** states the invariants a data type maintains. The two levels mirror the method-and-class structure of the bytecode side, adapted to the operation-and-schema structure of a service interface.

The **x-jml-signals** key handles exceptional behaviour, which at a service boundary means error responses. It carries an array of exception-condition pairs, the exception named by its JVM internal name so that a cross-language consumer can resolve it to whatever its own language calls the corresponding error. This cross-language resolvability matters because a service’s clients are often written in languages other than the service’s, and a contract that named exceptions in the service’s language alone would be opaque to them; the internal name is a language-neutral identifier the client can map.

The extension’s compatibility rests on the OpenAPI specification’s own provision for vendor extensions: any key prefixed **x-** is reserved for extension data, and conforming parsers must preserve it without interpreting it. The extension therefore rides through the standard OpenAPI toolchain — the parsers, the client generators, the documentation renderers — untouched, recovered by a consumer that knows the **x-jml-*** family and ignored by one that does not. A JSON Schema document accompanies the extension so that a consumer can validate an OpenAPI document’s contracts independently of the inferrer, and where the OpenAPI document is generated by a framework and not hand-editable, the same content travels in a sidecar JSON file keyed by route and verb, the service-side analogue of the bytecode sidecar. The design thus extends the toolchain-transparency property from the bytecode boundary to the service boundary by the same strategy: ride the format’s own extension mechanism, and fall back to a sidecar where the artefact cannot be modified.

5.3 Deployment Scenarios

The format design admits three deployment paths, and choosing among them is a matter of matching the carrier to the constraints of the artefact. The default path is in-bytecode embedding: the inferrer runs over a library’s source, the embedder writes the specifications into the compiled JAR’s class files, and the enriched JAR is published in place of the original. This is the path for the common case — an unsigned library whose specifications fit the per-class budget — and it is the path the empirical validation of Section 5.6 exercises. The consumer receives an ordinary-looking JAR that happens to carry recoverable contracts, and tooling that knows to look for them finds them while tooling that does not is unaffected.

The second path is the OpenAPI extension, for service boundaries. A service that documents its interface with an OpenAPI document — the dominant convention for REST APIs — can carry per-operation and per-schema contracts in the `x-jml-*` extension family, so that a client generator or a contract-test tool reading the document recovers the same preconditions and postconditions a library consumer would recover from bytecode. The path matters because the service boundary is where the oracle problem is most acute: a client of a remote service has neither its source nor its bytecode, only its interface description, and a contract carried in that description is the only specification the client can obtain. The OpenAPI path extends the reach of inferred specifications from the library-integration scenario of Chapter 1 to the service-integration scenario, and it is the mechanism the future-work discussion of service-boundary testing relies on.

The third path is the sidecar JAR, the universal fallback. Two constraints force it. A signed JAR cannot be modified without invalidating its signature, and re-signing requires the signer’s key, which the party enriching the JAR may not hold; embedding into such a JAR is impossible, but a parallel sidecar artefact carrying the same specifications as JML stub files travels alongside it without touching the signed bytes. And a specification too large for the per-class bytecode budget — rare, but possible for a method bearing many quantified loop invariants — can overflow the annotation, in which case the sidecar carries the overflow. The sidecar’s stub files are the format OpenJML reads natively, so the fallback is directly consumable by the verifier, and the manifest records the source JAR’s hash so a consumer can detect drift between the sidecar and the artefact it describes. The three paths together ensure that no artefact is un-enrichable: where in-bytecode embedding is unsuitable, the same content travels as a sidecar, and where the artefact is a service rather than a library, the same content travels in the interface description.

5.4 Implementation

The embedder is built on the ASM bytecode-engineering library, which permits class-file attributes to be added to an existing JAR in place, without recompilation. The

writer walks each class with an `ASM ClassVisitor`, and for each method, constructor, field, and type carrying a specification, it emits the packed annotation into the element's `RuntimeVisibleAnnotations` attribute. The reader is the inverse: it walks the embedded class files and recovers the specification from the annotation, decoding the token dictionary and synthesising the omitted frame default.

A subtlety in the ASM visitor lifecycle proved to be the source of the most consequential bug in the implementation, and it is instructive. The writer's lazy annotation emission was originally triggered by the visitor callbacks that fire for methods with bodies — the callbacks for code, parameters, and existing annotations. None of these fires for abstract or interface methods, which have no body and invoke only the end-of-method callback. The result was that interface methods silently received no embedded annotation, and an early prototype reported a roundtrip rate of 96.9% on one library and 97.6% on another, the shortfall concentrated entirely in interfaces. Adding the end-of-method callback to the trigger set closed the gap to 100%. The episode is a reminder that the cases a mechanism handles silently — here, methods without bodies — are exactly the cases where a measurement that does not seek them out will miss the failure.

The bug is worth dwelling on because of what it reveals about measurement design. The roundtrip rate of 96.9% looked like success — a small residual on a hard problem — and a study satisfied with a high-but-imperfect number would have reported it and moved on, attributing the gap to some intractable corner. The gap was not intractable; it was a single missing callback, concentrated in a structural feature of the code (interfaces) that the aggregate rate obscured. Disaggregating the roundtrip failures by program element revealed that every failure was an interface method, which pointed directly at the lifecycle callback that interfaces alone exercise. The lesson the thesis draws is that an imperfect aggregate should be disaggregated before it is accepted, because a residual concentrated in one structural category is usually a bug with a specific cause rather than an inherent limit. The 100% rate that the disaggregation made possible is a stronger result than the 96.9% would have been, and it was available only because the measurement sought out where the failures fell rather than tolerating their rate.

Multi-release JARs — archives carrying version-specific class files for different JDK versions in parallel directory trees — are the other structural feature that stressed the embedder, because a naive walk would embed into only one version's classes or would mismatch versions. The embedder handles the multi-release layout by embedding into each version's class files independently, so a consumer running under any supported JDK recovers the specification from the class file that JDK loads. Guava, a multi-release JAR, exercises this path, and its 100% roundtrip rate confirms the handling; the threats section notes that ahead-of-time compilation, which transforms the class files before they reach a JVM, is the remaining packaging variant the evaluation does not cover.

5.5 What Lossless Roundtrip Means, Precisely

The central correctness property of the embedder is *lossless roundtrip*: the specification recovered by the reader equals the specification the writer was given. Stating the property precisely matters, because there is a weaker property the embedder deliberately does not claim and a stronger one it deliberately does not need. The embedder claims that for every method, the clause list the reader recovers is equal, as a list of canonical-form strings, to the clause list the writer embedded. It does not claim *semantic* equality — that the recovered clauses are logically equivalent to the originals — because semantic equivalence is both unnecessary for faithful transport and dangerous to attempt.

The danger is worth dwelling on. A roundtrip that normalised semantics — that recognised $a > b$ and $b < a$ as equal and stored a canonical representative — would silently rewrite a consumer’s specification in transit. A consumer that wrote $a > b$ and recovered $b < a$ would have its data altered, however harmlessly, by a layer whose only job is to carry it faithfully. Worse, semantic normalisation is undecidable in general and unsound in the corners where it is decidable, so an embedder attempting it would introduce a class of bugs — specifications subtly changed in transit — that a purely lexical embedder cannot have. The embedder therefore canonicalises only *lexically* (Section 5.2): it fixes whitespace, operator spacing, and arrow forms, producing a single well-defined wire string for any given clause, but it never rewrites the logical content. Two logically equivalent clauses that differ lexically remain two distinct strings, and the roundtrip preserves each exactly.

This lexical discipline is what makes the 100% roundtrip claim both achievable and meaningful. Achievable, because lexical equality is decidable and the canonical form makes it stable across the write-read cycle; meaningful, because a consumer receives exactly the bytes the producer’s inferer emitted, with no silent transformation. The property is verified empirically by embedding and re-reading every specification and comparing clause lists for exact string equality, which is the measurement Section 5.6 reports. The distinction between this lexical roundtrip and a semantic one is not pedantry: it is the difference between a transport layer that can be trusted to carry data unchanged and one that cannot, and the choice of the lexical property is what earns the embedder the right to be called lossless.

5.6 Empirical Validation

The embedder is evaluated on real-world Java libraries under two regimes. The first uses *synthetic* specifications — one precondition per non-primitive parameter, one postcondition for reference returns, and the default frame — to isolate the embedding mechanism from the inferer’s behaviour and stress it uniformly. The second uses *real inferred* specifications produced by running JML-Inferer over the libraries’ sources, exercising the

full inference-to-embedding pipeline including quantifiers, history references, and branch-conditional implications. The headline results appear in Table 5.1.

Table 5.1: Embedder validation on real-world libraries. Method counts, lossless-roundtrip rate, and overhead are deterministic and reported once; throughput is the median across 39 runs. The fourth-row group reports the full inferer-to-embedder pipeline on real inferred specifications.

Library	Methods	Lossless	Byte ovhd.	Embed (m/s)	Read (m/s)
<i>Synthetic specs (isolating the embedding mechanism)</i>					
Apache Commons Lang 3.14.0	4,029	100.0%	+5.85%	16,225	221,617
Apache Commons IO 2.13.0	2,677	100.0%	+6.97%	21,976	327,618
Guava 33.3.0-jre	13,840	100.0%	+5.27%	12,618	127,924
<i>Real inferred specs (full inferer→embedder pipeline)</i>					
Apache Commons Lang 3.14.0	2,332	100.0%	+10.07%	8,648	103,779
Apache Commons IO 2.13.0	1,680	100.0%	+10.20%	10,215	107,318
Guava 33.3.0-jre	6,320	100.0%	+4.63%	6,052	74,144
JML-Inferer self-test	73	100.0%	—	—	—

5.6.1 Lossless Roundtrip (RQ3a)

Every corpus achieves a 100.0% lossless-roundtrip rate: every specification the harness embeds is recovered with byte-for-byte equality on its clause list, with zero mismatches anywhere. The real-inference regime is the more demanding test, because it exercises every JML construct the engine produces rather than the synthetic worst case. Across the three libraries the real-inference harness processes 1,118 source files containing 16,507 source method declarations, resolves 10,332 of them to distinct bytecode keys, and roundtrips every one losslessly. The constructs exercised include nested quantifiers, history references via `\old`, conditional postcondition implications, and loop invariants — the structurally complex output, not merely the simple null-check preconditions of the synthetic case. The distribution of constructs is reported in Table 5.2; every category roundtrips losslessly on every library.

5.6.2 Byte Overhead (RQ3b)

Synthetic-spec overhead sits between 5.27% and 6.97%; the differences across libraries are explained by structural shape rather than total size, with Commons IO running highest because its higher proportion of interfaces means more methods each carrying a clause set. In absolute terms the synthetic overhead is 38 KB for Commons Lang, 34 KB for Commons IO, and 162 KB for Guava — well below typical distribution sizes.

The synthetic figures are not an upper bound on real overhead, a point worth emphasising because a deployer planning a storage budget needs the realistic figure. The real-inference run on the same Commons Lang JAR measures 10.07%, more than four

Table 5.2: Spec-shape coverage on the 2,332 Commons Lang specifications embedded through the real-inference harness (representative; the other libraries give similar distributions). Every category roundtrips losslessly.

Construct	Specs	% of corpus
requires clauses	1,116	47.9%
ensures clauses	2,109	90.4%
assignable clauses	2,332	100.0%
loop_invariant clauses	269	11.5%
Quantifiers (\forall / \exists / \sum)	16	0.7%
\old references	85	3.6%
\result references	1,725	74.0%
Branch-conditional (=>)	345	14.8%

percentage points above the synthetic 5.85%, because real inferred specifications are richer than the synthetic worst case: 90.4% carry an **ensures** clause against the synthetic one-per-reference-return policy, 11.5% carry loop invariants the synthetic harness never emits, 14.8% include branch-conditional implications, and the average clause string is longer because the engine emits parameter bounds, result relations, and history references the synthetic policy never produces. Guava inverts the relationship — its real-inference overhead, 4.63%, is *lower* than its synthetic figure — because its large generic-heavy classes have dense bodies, so the same per-spec byte cost amortises against a larger denominator. The lesson for a deployer is to provision for the real-inference figure, around ten percent on a typical utility library, not the synthetic-uniform figure.

The Guava inversion is worth dwelling on because it corrects an intuition that would otherwise mislead a deployer. The natural expectation is that real specifications, being richer than the synthetic worst case, always cost more, and on the smaller utility libraries they do — four to five points more. Guava violates the expectation not because its specifications are leaner but because its denominator is larger: overhead is the embedded bytes divided by the original bytes, and Guava’s original bytes are inflated by large generic-heavy classes with dense method bodies, so the same per-method specification cost is a smaller fraction of a larger whole. The general principle the inversion reveals is that overhead depends on two quantities a deployer should consider separately — the absolute specification cost per method, which is a property of the inferrer’s output, and the original code size per method, which is a property of the library’s density — and that a dense library can carry richer specifications at lower proportional overhead than a sparse one. The synthetic-versus-real comparison isolates the first quantity by holding the specification policy fixed, and the cross-library comparison isolates the second by holding the policy fixed and varying the library, and the Guava inversion is what happens when the second effect dominates the first.

5.6.3 Format Optimisation

Table 5.3 reports the impact of the three size optimisations against the verbose predecessor format. The optimisations reduce overhead by 48 to 55 percent on the synthetic corpora and by 31 percent on real inference, while preserving the 100% lossless property in every case.

Table 5.3: Format-optimisation impact on bytecode-size overhead. The lossless-roundtrip rate is 100% under both formats.

Corpus	Verbose ovhd.	Optimised ovhd.	Reduction
Synthetic — Commons Lang	11.95%	5.85%	−51%
Synthetic — Commons IO	13.49%	6.97%	−48%
Synthetic — Guava	11.76%	5.27%	−55%
Real inference — Commons Lang	14.70%	10.07%	−31%

The synthetic reduction is larger than the real-inference reduction because synthetic specifications are dominated by exactly the patterns the optimisation targets — every specification carries the default-omittable frame, every precondition is a null check, every postcondition references the result — all of which the optimisation captures. Real specifications contain a wider variety of clause shapes, many of which the twelve-token dictionary does not cover, so the real-inference reduction is the more realistic estimate for a production deployment. A configuration sweep confirms the optimisations’ contribution: an always-explicit mode with no codec runs 1.19 percentage points above the default, and an automatically learned twenty-eight-token dictionary gains a further 0.59 points over the hand-picked twelve, establishing that the hand-picked dictionary already captures most of the available saving. A per-spec DEFLATE compression mode is a clear negative result, running 20 points *worse* than the default, because DEFLATE cannot exploit redundancy in the fifty-to-hundred-byte payloads typical after token encoding and the per-spec compressed payloads forfeit the constant-pool deduplication the uncompressed format enjoys. The negative result is reported because it is informative: it shows that the right granularity for compression, if any, is the class file rather than the individual specification.

5.6.4 Throughput (RQ3c)

Embedding throughput ranges across runs from roughly six thousand to thirty-one thousand methods per second, and reading throughput an order of magnitude higher, from roughly seventy-four thousand to over three hundred thousand. At the medians, embedding a fourteen-thousand-method JAR completes in well under a second end to end, and reading completes faster still because the reader skips full class rewriting. These figures comfortably support per-pull-request continuous-integration workflows: even a pessimistic re-run on every push of a Guava-sized library is sub-second, far below the Maven build time the inference pipeline already rides on. The throughput question is therefore answered

affirmatively with room to spare.

5.6.5 Throughput in Context (RQ3c)

The throughput question is answered affirmatively, but the figures repay interpretation against the workflows they are meant to support. Embedding a fourteen-thousand-method library completes in well under a second at the median embedding rate, and reading completes an order of magnitude faster because the reader does not rewrite the class files. The relevant comparison is not against some absolute standard but against the cost of the surrounding workflow. The embedding rides on a Maven build that already takes tens of seconds to minutes; the inference that produces the specifications takes longer still, at tens of milliseconds per file over thousands of files. Against this backdrop the sub-second embedding is negligible — it disappears into the noise of the build it augments.

This matters for the continuous-integration scenario the question targets. A plausible deployment runs the inferer and the embedder on every pull request, so that the enriched artefact reflects the current source. For this to be acceptable the embedding must fit within the per-pull-request budget, which is dominated by compilation and testing and measured in minutes. A sub-second embedding step fits trivially, even re-run from scratch on every push of the largest library measured. The reading throughput matters for the consumer side: a tool that recovers specifications from a dependency does so at hundreds of thousands of methods per second, fast enough that recovering an entire library's contracts is imperceptible. The throughput figures thus clear their bar with orders of magnitude to spare, which is the right outcome to report — not a tight fit that might fail under load, but a comfortable one that will not.

The variation in throughput across runs — the reason the figures are reported as medians over many runs rather than as single measurements — reflects the ordinary sources of variance in JVM workloads: just-in-time compilation warming up, garbage collection pausing, the operating system scheduling other work. These sources affect the absolute rate but not the conclusion, because even the slowest observed run completes the embedding within the per-pull-request budget. Reporting the median over many runs, with the observed range, is the honest way to characterise a throughput that varies for reasons unrelated to the embedder's logic, and it is the same discipline the lossless-roundtrip and overhead measurements do not need because those are deterministic.

5.6.6 Toolchain Compatibility (RQ3d)

The compatibility question is answered on two fronts. On the bytecode side, a negative test loads an embedded class through a class loader that does not see the annotation type, and confirms that the class loads successfully: reflective annotation access either returns the resolvable annotations or raises the documented exception, but class loading itself is

unaffected. A JVM with no awareness of the annotation type runs the embedded class exactly as it would the unembedded one — the embedding is invisible to the runtime. On the service side, the OpenAPI extension is preserved by the standard parsers without modification, because the `x-` prefix is the specification’s own mechanism for vendor extensions. The mechanism therefore meets its design goal: enrichment requires no change to how artefacts are built, loaded, published, or parsed.

5.7 The Class-File Format and Why the Mechanism Is Transparent

The transparency of the embedding rests on a feature of the Java class-file format that is worth making explicit, because it is what allows enrichment without toolchain modification. A Java class file is a structured binary container: a magic number, version, constant pool, and a sequence of structures describing the class’s fields, methods, and attributes. The format is deliberately extensible through *attributes* — named, length-prefixed blobs attached to the class, its fields, and its methods — and the specification mandates that a reader encountering an attribute it does not recognise must skip it, using the length prefix, rather than fail. This skip-the-unknown rule is the foundation of forward compatibility in the JVM, and it is what guarantees that a class file carrying an unfamiliar attribute loads correctly in a JVM that has never heard of it.

Runtime-visible annotations are carried in exactly such an attribute, the `RuntimeVisibleAnnotations` attribute, which the format defines and every conforming JVM reads. An annotation written into this attribute is recoverable by reflection if the annotation’s type is on the classpath, and is harmlessly skipped if it is not. The embedder exploits this: it writes the packed specification annotation into the `RuntimeVisibleAnnotations` attribute of each method, field, and type. A consumer that has the annotation type can recover the specification by reflection; a class-file reader that lacks it (such as the ASM-based reader the extractor uses) can recover it by reading the attribute directly; and a JVM that knows nothing of it loads and runs the class exactly as before. The negative-load test of Section 5.6 verifies the last case empirically, and the class-file format’s skip-the-unknown rule is the reason it passes.

The constant pool is the other relevant feature. Strings in a class file are stored once in the constant pool and referenced by index, so two methods whose specifications share a clause string — and inferred specifications share clause strings heavily, since `requires arr != null` and `assignable \nothing` recur across a library — pay for that string only once. This deduplication is what makes the per-method annotation overhead modest in aggregate, and it is precisely the saving that the per-spec DEFLATE mode of Section 5.6 forfeits by making each method’s payload unique, which is the mechanism behind that mode’s negative result. The format’s existing machinery for sharing strings does, for free,

what per-spec compression cannot.

5.7.1 A Worked Embedding

Consider the `max` method of Chapter 2, with its inferred contract of one trivial precondition, two postconditions, and the default frame. The embedder writes a single `@JmlSpecs` annotation onto the method, with `ensures` carrying the two postcondition strings and the other members defaulting — `requires true` is the trivial precondition the format omits, and `assignable \nothing` is the default the format synthesises on read. The two postcondition strings are token-encoded: the `\result` occurrences collapse to a single control byte each, and the strings are interned in the constant pool, so the second postcondition’s shared substructure with the first costs nothing beyond its distinct portion. On read, the extractor recovers the annotation, decodes the token bytes back to `\result`, synthesises the omitted frame and trivial precondition, and reconstructs the contract byte-for-byte. The roundtrip is lossless because the canonical form fixed the wire representation before emission, so there is a single, well-defined string for the reader to reproduce. This worked case is the unit that the 100% lossless-roundtrip rate of Section 5.6 aggregates over twenty thousand times.

5.8 Consuming Embedded Specifications

The embedding is only useful if consumers can act on it, and the format is designed so that several classes of consumer can, each by a mechanism natural to it. A reflective consumer — a test generator or a runtime contract checker running inside a JVM that has the annotation type on its classpath — recovers the specification by ordinary reflection, reading the `@JmlSpecs` annotation off the method and decoding its members. This is the path for a tool that wants to enforce contracts at runtime or to drive test generation from them, and it requires nothing of the consumer beyond depending on the small annotation-type library.

A static consumer — a verifier, an IDE, an analysis tool — recovers the specification by reading the class file directly, without loading it into a JVM. The ASM-based reader the extractor uses is one such consumer, and any class-file reader can be taught the format from the specification of Section 5.2. This is the path for OpenJML itself: a verifier checking client code against a dependency can read the dependency’s embedded contracts and use them as the assumed specifications of the called methods, exactly as it would use hand-written stubs, closing the modular-verification loop across the library boundary without the library’s source.

A language-model consumer — the case that motivates much of the thesis — recovers the specification however its tooling extracts class-file metadata, and incorporates it into the model’s context as the inferred contract of the API under discussion. This is the mech-

anism by which the test-generation benefit of Chapter 4 reaches a model working against a compiled dependency: the model reads the embedded contract rather than guessing the dependency’s behaviour, and the same improvement the controlled experiment measured on source-available code becomes available on compiled code. The service-side OpenAPI extension serves the same role for a model reasoning about a remote service from its interface description.

The robustness of the consumer story rests on the property the negative-load test establishes: an embedded artefact is safe to ship to consumers who know nothing of the embedding. Were the embedding to break class loading in a JVM without the annotation type, every consumer would have to add the annotation library to its classpath before using the enriched artefact at all — precisely the toolchain coupling the design exists to avoid. The test confirms the coupling does not arise: the class-file format’s skip-the-unknown rule tolerates a runtime-visible annotation whose type cannot be resolved, so the worst that happens is that reflective access to the annotation raises the documented type-not-present exception, which a consumer that does not ask for the annotation never triggers. The class loads, the methods run, and a consumer unaware of the embedding is unaffected — the precondition for enriching an artefact that will be consumed by parties the enricher does not control. The JSON Schema document accompanying the OpenAPI extension serves the analogous role on the service side, letting a consumer validate that the contracts it receives are well-formed before acting on them, the way it would validate any external input.

The forward-compatibility of the format matters for all three consumer classes. A consumer built against the current clause kinds skips members it does not recognise, so a future clause kind added to the format does not break existing consumers; and a consumer that does not know the token dictionary sees the control bytes and can either decode them from the published table or fall back to the sidecar’s uncompressed stub files. The format degrades gracefully: a consumer recovers as much of the specification as it understands and ignores the rest, which is the property that lets the format evolve without coordinating every consumer’s upgrade.

5.9 Discussion and Threats

5.9.1 What the Result Establishes

The contribution is a path from an inferred specification to a consumer-readable contract that requires no change to the build toolchain: any existing JAR can be enriched in place, and the enrichment survives ordinary distribution and is recoverable by any class-file reader without the source. The 100% lossless rate across four corpora and every JML construct the engine produces establishes that the transport is faithful; the sub-eleven-percent overhead and sub-second throughput establish that it is affordable; the

negative-load test and the OpenAPI parser compatibility establish that it is transparent. Together these close the distribution gap that the test-generation result of Chapter 4 would otherwise leave open: a specification that improves downstream tooling is now available to downstream consumers who lack the source, which is most of them.

The result also completes the integration scenario of Chapter 1. A developer adopting a compiled library, or a model reasoning about one, encounters it as bytecode without a contract, and learns its behaviour by trial, by reading whatever source is available, or by the exceptions a violated precondition throws. With the embedding, the library’s inferred contract travels inside the JAR, and tooling at the call site — an IDE, a verifier, a contract-checking test generator, a model reading the API — recovers it without the source. The integration scenario’s central difficulty, that the contract is present in the library’s design but absent from the form the integrator encounters, is resolved by carrying the contract into that form. The distribution study is, in this sense, the bridge between the inference study, which produces a contract from source, and the consumer scenarios, which need the contract without source: it is the stage that makes the produced contract reach the consumer that the value study showed would benefit from it.

5.9.2 Format Evolution and Versioning

A distribution format must evolve, and the design anticipates evolution in a way that the empirical results validate. The format carries a version member, and the reader accepts both the current packed format and its verbose predecessor, so that artefacts embedded against the earlier format continue to load against the current reader. This backward compatibility is not merely good practice; it is a correctness property the thesis tests, because a format that broke its predecessor on every revision would force every consumer to upgrade in lockstep, which is exactly the toolchain coupling the design aims to avoid.

The forward direction is handled by the skip-the-unknown discipline of the class-file format (Section 5.7) and the skip-unrecognised- members discipline of the annotation. A future clause kind, added as a new annotation member, is ignored by a reader that predates it rather than breaking that reader; a future format version is distinguished by its version member, and a reader can dispatch on it. The format can therefore add clause kinds and revise its encoding without invalidating either the artefacts embedded against earlier versions or the consumers built against them, which is the property that lets the inference engine’s output grow — new clause kinds as new patterns are added — without a coordinated upgrade of every consumer in the ecosystem.

The configuration sweep of Section 5.6 bears on evolution in a second way. By exposing the default-omission and token-dictionary conventions as configurable, the format lets a deployer whose inferrer has a different dominant clause shape opt out of conventions tuned for this thesis’s engine and supply its own. An inferrer that emitted a conservative `assignable \everything` default rather than `\nothing` would select a different default-

omission policy; an inferrer with a different token frequency profile would supply a different dictionary, or learn one from its own corpus with the dictionary learner the sweep evaluates. The format is thus not hard-coded to one inferrer’s output distribution but parameterised by it, which is what allows the same transport mechanism to serve inferrers other than the one this thesis built.

5.9.3 When to Embed and When Not To

The mechanism’s availability does not make embedding always the right choice, and a deployer should weigh the considerations the evaluation surfaces. Embedding is the right default when the consumer benefits from structured contracts and the artefact can be modified — an unsigned library published to a repository, enriched once at build time and consumed by tooling that recovers the contracts. The overhead, under eleven percent on a typical library, is negligible against distribution sizes, and the throughput fits any continuous-integration budget, so the cost side of the ledger is light.

Embedding is the wrong choice in three situations the design anticipates. A signed artefact that cannot be re-signed forecloses in-bytecode embedding, and the sidecar is the only option. An artefact whose consumers are uniformly unaware of specifications gains nothing from embedding and pays the overhead for nothing, though the overhead is small enough that the cost is more theoretical than real. And an artefact where even the small overhead is unacceptable — a tightly size-constrained deployment — should use the sidecar, which moves the bytes out of the artefact entirely. The configuration sweep gives a deployer the further choice of trading overhead against codec complexity: the always-explicit mode pays a little more for the simplicity of no token dictionary, and the learned-dictionary mode pays a little less at the cost of carrying a corpus-specific dictionary.

The decision, in short, is not whether the mechanism works — the evaluation establishes that it does, losslessly and cheaply — but whether the consumer will use the contracts and whether the artefact can carry them. The thesis provides the mechanism and the measurements; the deployment decision rests on the consumer landscape and the artefact constraints, which vary by context. The design’s provision of three paths — in-bytecode, OpenAPI, sidecar — is what ensures the decision is always “which path” rather than “whether possible”.

5.9.4 Threats to Validity

The evaluation covers three libraries plus a self-test rather than the full protocol of ten; the lossless analysis identifies the residual risks — interface methods and multi-release JARs — and the embedder reaches 100% on libraries exercising both, so the remaining libraries are mechanical to validate. The evaluation targets JDK 21; older JDKs and

ahead-of-time compilers such as GraalVM native-image are deferred, though the class files emitted by Commons Lang target Java 8, so the runtime-side validation should generalise. The source-to-bytecode key resolution in the real-inference harness matches on name and arity only, so a residual of methods is lost to overload collisions where two source overloads map to the same erased descriptor; the 100% lossless rate is on every specification the harness successfully embeds, so the residual is a measurement limitation rather than an embedder limitation, and a full type-resolving harness would recover it without changing the embedder’s behaviour. These threats bound the generality of the measurement but not the central claim, which is that the embedding mechanism transports every specification it is given, losslessly, at acceptable cost, through an unmodified toolchain.

A reflection on the nature of this study closes the chapter. Unlike the test-generation study, whose claim is causal and whose validation is statistical, the distribution study’s claims are about a deterministic mechanism, and its validation is exact measurement rather than inference. There is no sampling uncertainty in a losslessness rate: a specification either roundtrips or it does not, and the rate is a count, not an estimate. This gives the distribution study a different epistemic character — its claims are not “probably true within a confidence interval” but “observed to be true on every case measured” — and the threats that bound it are about coverage (which cases were measured) rather than about uncertainty (how confident the estimate is). The two studies’ different characters reflect the different kinds of claim they make, and matching the validation to the claim’s kind, as Chapter 1 argued, is what lets each study assert exactly the certainty its claim warrants: the value study a confident causal effect, the distribution study an exact mechanical property, and neither overreaching into the other’s kind of assertion.

Chapter 6

Compositional Propagation and Library Compatibility

A caller’s specification is not independent of the libraries it calls. When a method invokes another, the obligations the callee imposes become, after substitution, obligations on the caller; the caller’s contract is in part a function of its callees’ contracts. This dependence is the subject of the present chapter, and it is the mechanism on which the thesis’s adoption argument most directly turns. If a caller’s specification depends on its callees’, then a change to a library — a new version that tightens a precondition or weakens a guarantee — propagates into the specifications of the code that uses it, and the behavioural compatibility of an upgrade becomes a question answerable at the specification level. The chapter measures how strong this dependence is on real code, by implementing a top-down compositional analysis that propagates callee specifications into callers, measuring what it recovers across five open-source libraries totalling 21,052 methods, and characterising the structural shapes of the recovered obligations — which turn out to be exactly the shapes through which a library change reaches its callers. The chapter answers RQ4, and connects the result both backward to the test-generation value of Chapter 4 and forward to the compatibility application the dependence enables.

6.1 Why the Dependence of Caller on Callee Matters

The dependence of a caller’s specification on its callees’ is consequential for two reasons, one bearing on the thesis’s other results and one bearing on its central adoption argument.

The first is that it sets a ceiling on those other results. The specifications the earlier chapters validated, distributed, and supplied to a model were computed by the inference engine’s single-pass propagation, which captures only part of the caller-on-callee dependence (Section 6.2). If a more thorough analysis recovers substantially more of the dependence, then the specifications the earlier chapters used were weaker than they could have

been, and the value they measured was a lower bound. How much the single pass leaves uncaptured therefore determines whether the thesis’s other results sit near the achievable ceiling or well below it.

The second, and more important, reason is the compatibility application. The behavioural compatibility of a library upgrade is, at bottom, a question about how the library’s changed contracts affect the contracts of the code that calls it — and that effect exists only to the extent that caller specifications depend on callee specifications. If the dependence were weak, a library change would barely register in its callers’ contracts and specification-level compatibility analysis would have little to work with; if it is strong, a library change propagates visibly into caller contracts and the analysis has a rich signal. Measuring the strength of the dependence is therefore the prerequisite for the compatibility application: it establishes whether the signal that application would rely on is substantial. The chapter finds that it is — caller specifications depend on callee specifications heavily, through cross-method obligations that a single-pass analysis cannot even express — which is what makes Section 6.7’s account of compatibility analysis a mechanism with real content rather than a hopeful extrapolation.

6.2 Where Single-Pass Propagation Is Blind

The single-pass propagation of Section 3.7 is the right strategy for the simple case, and it is sound for that case: a precondition that a callee requires *unconditionally* must hold at the call site, and conjoining the substituted precondition into the caller’s set is correct. But the strategy is structurally blind to two classes of cross-method obligation, and the blindness is not a bug to be fixed within the single pass but a consequence of the pass’s architecture.

The first class is the *branch-conditional* obligation. Consider a caller whose body contains a call inside a conditional:

```
if (g) {
  n(args);    // n requires p
}
```

The caller’s precondition is not $p[args/params]$ but $g \Rightarrow p[args/params]$ — the obligation holds only on the path where the call is reached. Single-pass propagation has two options, both wrong. It can append the substituted precondition unconditionally, which over-constrains the caller on every path where the call does not fire; or it can skip the substitution to be safe, which misses the obligation entirely on the path where the call does fire. Neither is correct, because producing the correct clause requires the caller’s control-flow context — the guard g enclosing the call — and a single substitution at the call site does not carry that context.

The second class is the *polymorphic-dispatch* obligation. Consider a call through an

interface or overridable method whose declared type has several implementations. The runtime dispatch may select any of them, so a sound caller precondition must hold for *at least one* candidate: it is the disjunction $p_1 \vee p_2 \vee \dots \vee p_k$ of the candidates' substituted preconditions. Single-pass propagation resolves each call to a single nominal callee and cannot produce this disjunction, because it never enumerates the candidate set.

It is worth being precise about why these two shapes, and not others, are the ones the single pass misses. The single pass misses them because both require information that lives in the *caller*, not in the callee whose contract is being propagated. The branch-conditional shape requires the guard enclosing the call, which is a fact about the caller's control flow; the polymorphic shape requires the set of types the receiver might hold, which is a fact about the caller's context and the class hierarchy. Single-pass propagation, structured as a substitution of the callee's contract at the call site, has the callee's contract in hand but does not consult the caller's surrounding structure — it reads the callee and writes at the call site, and the caller's guards and the receiver's possible types are exactly the surrounding structure it does not read. The shapes the single pass *does* capture — unconditional preconditions on a single-candidate call — are precisely those that need nothing from the caller beyond the call's arguments, which the substitution already has. The boundary between what the single pass captures and what it misses is therefore not arbitrary: it is the boundary between obligations determined by the callee and the call's arguments, which a substitution handles, and obligations determined by the caller's control-flow and type context, which a substitution cannot reach.

Both shapes are recoverable, but only by an analysis that does what the single pass does not: walk the caller's body in a weakest-precondition fashion, accumulating the guard context at each program point and enumerating the dispatch candidates at each call site. The classical weakest-precondition calculus [14] addresses both structurally — a call site reached only on g contributes $g \Rightarrow p$ by the conditional rule, and a polymorphic call site contributes the disjunction over candidates by construction. A complete WP transformer for arbitrary Java is out of reach, for the reasons of Section 2.1, but a *scaffold* that walks the body in WP-flavoured order, lifts each call site's substituted callee specification through its enclosing guards, and recognises polymorphic candidates is implementable as a bounded pass over the AST. That scaffold is what this chapter builds.

6.3 The Refinement Pass

The pass takes a populated specification cache, a call graph over the same methods, and a map from method signature to AST declaration, and it mutates the cache in place by adding clauses no single pass could produce.

6.3.1 Driver

The driver iterates the strongly connected components of the call graph in reverse topological order, so that callees are refined before callers. For a singleton component — a method not involved in recursion — it refines the method once. For a cyclic component — a set of mutually recursive methods — it refines each method in the component and repeats until no method gains a new clause or a fixpoint cap is reached, the cap bounding runtime on pathological graphs and logging a warning when it fires. In practice the cap never fires on the libraries measured: their call graphs are acyclic under the analyser’s signature-matching scheme, so each method is refined exactly once.

6.3.2 Per-Method Refinement

For each method, the refiner walks every call expression in the body and, at each call site, performs five steps. First, it *resolves the callee* by matching the call’s name against the cache. Second, it *substitutes arguments*, replacing each formal parameter in the callee’s preconditions with the actual argument expression; the substitution is regex-based with word boundaries so that a formal name does not clobber a substring of an unrelated identifier, and a side-effecting actual argument rejects the substitution as unsound. Third, it *lifts through enclosing guards*: it walks the AST upward from the call, accumulating the conjunction of any conditional or ternary guard whose body contains the call, and if guards are present the substituted precondition becomes $guard \Rightarrow substituted$, with a call in an else-branch contributing the negated guard. Fourth, it *detects polymorphic dispatch*: if the resolved callee has overriding implementations in subtypes, found via the call graph’s class-hierarchy index, it collects every candidate’s substituted preconditions and emits their disjunction, which is sound because at least one candidate’s contract must hold for the dispatch to be safe. Fifth, it *conjoins* the lifted clause into the caller’s set if it is not already present. The pass also propagates callee *postconditions* into the caller, gated on a termination flag: a postcondition of a possibly- non-terminating callee describes a state established only if the callee returns, so propagating it unconditionally would be unsound, and the pass propagates only when the callee is known to terminate.

6.3.3 Why Postcondition Propagation Needs a Termination Gate

The postcondition propagation just described carries a soundness subtlety that the precondition propagation does not, and it is worth isolating because it is the kind of detail that distinguishes a sound compositional pass from a plausible-looking but broken one. A precondition is an obligation the caller must discharge before the call; propagating it is sound regardless of whether the callee terminates, because the obligation is on the entry to the call. A postcondition is a guarantee that holds *after* the call returns; propagating it into the caller’s contract asserts that the caller’s state, after the call, satisfies the callee’s

postcondition — which is true only if the call actually returned.

For a callee that always terminates this is unproblematic. For a callee that may not terminate — one with an unbounded loop or unbounded recursion — the postcondition describes a state reached only on the terminating executions, and propagating it unconditionally would assert it on executions where the callee diverges, which is unsound. The termination gate is conservative: when termination is unknown the pass declines to propagate, missing a postcondition rather than asserting an unsound one, consistent with the lower-bound character of the whole measurement. The polymorphic case compounds the subtlety, the sound clause being a disjunction over the terminating candidates' postconditions, handled identically to the precondition disjunction. The gate is thus a discipline applied at every postcondition propagation, and it is part of the reason the postcondition additions of Table 6.1 are smaller, on most libraries, than the precondition additions.

6.3.4 The Driver in Detail

The driver's reverse-topological traversal of strongly connected components is what makes the refinement well-founded, and the reasoning is worth setting out. Refining a method means lifting its callees' contracts into it, which is only sound if those callees' contracts are themselves complete; refining a caller before its callee would lift an under-refined contract and miss clauses the callee would later gain. Reverse-topological order over the condensation of the call graph — the directed acyclic graph of strongly connected components — guarantees that every callee outside a method's own component is fully refined before the method is visited. Within a component, where methods are mutually recursive and no acyclic order exists, the driver iterates: it refines every method in the component, and if any gained a clause it refines them all again, because a clause gained by one member may, on the next pass, lift into another. The iteration is monotone — clauses are only added, never removed — and the clause set per method is finite, so the iteration reaches a fixpoint; the configurable cap exists only to bound runtime on pathological graphs and logs a warning if it fires.

On the five libraries measured, the cap never fires, for an interesting reason: under the analyser's signature-matching resolution, the call graphs are acyclic, so every component is a singleton and every method is refined exactly once. The fixpoint machinery is therefore dormant on the measured corpora, present for soundness on graphs that would exercise it but never invoked. This is itself a finding about real code: the mutual recursion that would force fixpoint iteration is rare enough in the five libraries that the simpler singleton case dominates, which is why the runtime cost (Section 6.5) is so modest — the pass does one refinement per method and stops.

6.3.5 Class-Hierarchy Analysis and Dispatch Precision

The polymorphic-dispatch detection rests on a class-hierarchy index, and its precision is governed by the same trade-off that has long characterised call-graph construction for object-oriented languages. The simplest sound over-approximation of a virtual call’s targets is class-hierarchy analysis, which takes the targets to be every override of the called method in any subtype of the receiver’s declared type. It is sound — the runtime target is always among them — but imprecise, because it includes subtypes that the receiver could never actually be at the call site. A more precise analysis, rapid-type analysis, prunes the candidate set to subtypes that are actually instantiated somewhere in the program, and flow-sensitive analyses prune further by tracking which types reach the receiver.

The refinement pass uses the class-hierarchy index its call graph provides, which is a class-hierarchy-analysis-level approximation: it enumerates the overrides of the called method in the receiver type’s subtypes and emits the disjunction over their preconditions. The imprecision has a benign direction for the pass’s soundness: including a candidate the receiver could never be makes the disjunction weaker — easier to satisfy — which cannot make the emitted precondition unsound, only less informative. A disjunction over too many candidates is a precondition that demands too little, which is safe; a disjunction that omitted a real candidate would be a precondition that demands too much, which would be unsound, and the hierarchy index avoids omission by including every override. The pass thus inherits the standard class-hierarchy-analysis trade-off in its benign form: it may emit a disjunction weaker than a more precise analysis would, but never one that is unsound, and the disjunctive-shape counts of the measurement are therefore a sound if conservative account of the polymorphic obligations the code imposes.

The prevalence of the disjunctive shape on Vavr, the library with the deepest interface hierarchies, is in part a consequence of this approximation: a deep hierarchy means many overrides, and class-hierarchy analysis enumerates them all, so a call through a Vavr interface produces a disjunction over many candidates. A more precise analysis would prune some, narrowing the disjunctions, but at the cost of the flow-sensitive machinery the pass deliberately forgoes. The measurement reports the class-hierarchy-analysis-level result, which is the sound conservative one, and notes that a more precise dispatch analysis is a direction that would sharpen the disjunctions without changing the qualitative finding that they are present and substantial.

6.3.6 What the Pass Is and Is Not

The pass is a *lifting* step over an existing single-pass result, not a complete WP transformer. It does not compute the weakest precondition through arithmetic or array statements — it accepts the engine’s clauses verbatim — it does not reason about aliasing, it does not lift through loops (the engine’s loop-invariant heuristics handle those in the single pass), and it does not discharge clauses against a solver (the downstream OpenJML pipeline does that).

It does exactly enough — branch-guard lifting and polymorphic-dispatch disjunction — to expose the two structural classes of additions that single-pass propagation cannot produce. The measurement that follows asks whether those additions are empirically substantial on real code, and the answer is the chapter’s contribution.

A question the structural measurement raises but does not by itself answer is whether the refined specifications remain *sound* — whether the clauses the pass adds discharge against the implementation the way the engine’s other clauses do. The pass is designed so that they should: the lifted clauses are the callees’ already-validated preconditions, substituted and guarded, and a sound callee precondition lifted through a sound guard is itself sound. The substitution disciplines of Section 6.4 preserve this — they reject exactly the substitutions that would break soundness — so a clause the pass adds is, in principle, dischargeable whenever the callee’s original clause was. The refined specifications therefore feed the same OpenJML validation layer the engine’s other output feeds, and a clause that fails to discharge is flagged like any other, which means the second pass does not relax the soundness discipline of the first. The measurement reports the clauses the pass adds, not their discharge rate, because discharging the full refined corpus — twenty thousand methods with their refined contracts — is a substantial verification effort deferred with the downstream mutation evaluation as a completion of the study. The architectural point stands regardless: the refined clauses are not a new class of unvalidated output but lifted instances of already-validated callee clauses, passing through the same validation gate.

6.4 The Soundness of Substitution

The refinement pass lifts a callee’s precondition into the caller by substituting actual arguments for formal parameters, and the soundness of that substitution is not automatic — it is a property the pass must actively preserve, and the cases where it does not hold are the cases the pass must reject. The textbook WP rule for procedure call substitutes actuals for formals, but the rule assumes the actuals are pure expressions whose evaluation has no side effect and whose value does not change between the precondition’s evaluation point and the call. Real Java arguments violate both assumptions, and the pass handles the violations by refusing the unsound substitutions rather than performing them incorrectly.

The first hazard is a side-effecting actual. If a call passes `n(i++)` and the callee requires its parameter be non-negative, naively substituting yields `i++ >= 0`, a clause whose evaluation increments `i` and whose meaning is therefore ill-defined as a precondition — a precondition must be a predicate the caller can evaluate without changing the state it constrains. The pass detects increment, decrement, and assignment expressions in actual arguments and rejects the substitution for those call sites, contributing no clause rather than an unsound one. The conservatism costs recall — a genuine obligation is missed — but it preserves the invariant that every clause the pass adds is one the caller could in principle satisfy.

The second hazard is name capture in the substitution itself. Replacing a formal parameter named `s` by a textual search-and-replace would corrupt an unrelated identifier that contains `s` as a substring — turning `this.size` into `this.<arg>ize` — which is a class of bug the implementation history of the engine records as a real and recurring one. The pass substitutes with word-boundary matching, so that only whole-identifier occurrences of the formal are replaced, and a formal that happens to be a substring of another identifier leaves that identifier untouched. The word-boundary discipline is the textual analogue of capture-avoiding substitution in the lambda calculus: it ensures that substituting for `s` affects exactly the occurrences of the variable `s` and nothing else.

The third hazard concerns the guard lifting itself. A guard that is itself side-effecting — a condition that calls a mutating method — cannot soundly be lifted into a precondition, for the same reason a side-effecting actual cannot: the precondition would change the state it describes. The pass rejects side-effecting guards, lifting only guards that are pure boolean expressions over the caller’s state. Together these three disciplines — reject side-effecting actuals, substitute with word boundaries, reject side-effecting guards — are what make the lifted clauses sound additions to the caller’s contract rather than syntactically plausible but semantically broken strings, and they are the reason the measurement of Section 6.5 reports a lower bound on the compositional gain: every rejection is a clause the pass could have added under a less careful policy but declines to add under a sound one.

6.5 Empirical Measurement

6.5.1 Subjects and Method

The pass is measured on the source jars of five open-source Java libraries spanning four idiom families: Apache Commons Lang (utility/string code, 3,494 methods), Apache Commons IO (I/O abstractions, 2,111 methods), Apache Commons Math (numerical algorithms, 6,645 methods), jOOL (functional-style combinators, 3,691 methods), and Vavr (persistent collections, 5,111 methods). Combined, the corpus is 1,712 source files and 21,052 methods with a non-empty isolated baseline specification. The breadth is deliberate: it spans the imperative utility-class idiom that the earlier chapters emphasise and the higher-order generic style of the functional libraries, so that the result cannot be attributed to a single coding style. Commons Lang and IO are the same libraries used in Chapters 4 and 5, which keeps cross-chapter comparisons free of corpus-difference confounds.

For each library, the harness parses every source file, builds the call graph, runs the engine’s standard inference (including its single-pass `InterproceduralAnalyzer`) to produce the isolated baseline, snapshots the per-method clause counts, runs the refinement pass, and diffs the post-refinement specifications against the baseline, categorising each newly added precondition by syntactic shape. All measurements are deterministic given

the inputs, and each is reported once.

6.5.2 The Call Graph and the Measurement Harness

The pass depends on a call graph, and the harness that takes the measurement depends on resolving source methods to the cache, so both deserve description — not least because their limitations bound the measurement, as the threats section records. The call graph is built by walking every method declaration and recording, for each call expression in its body, the callee it resolves to, together with the class-hierarchy relationships — which classes extend which, which implement which interfaces — that the polymorphic-dispatch detection consults. The graph’s nodes are method signatures and its edges are calls; its condensation into strongly connected components is what the driver iterates.

The resolution of a call to a callee is by signature matching, and here the harness makes a deliberate simplification whose consequence the measurement must account for. Two source methods of the same name and parameter count can, under erased generics, share a bytecode descriptor, and the harness’s name-and-arity matching cannot always distinguish them. The consequence is that a residual of call sites resolve to the wrong callee or fail to resolve, and the pass adds no clause for those sites. Because a missed or misresolved call site can only cause the pass to add *fewer* clauses — it never invents a clause from a wrong resolution, the substitution being rejected when the arities disagree — the effect is to *under*-count the compositional gain. Every number the measurement reports is therefore a lower bound on the true gain a fully type-resolving harness would find, which is the conservative direction for a claim that the gain is substantial: the true gain is at least what is reported.

The measurement is otherwise deterministic. Given the source jars, the call graph, the inference, and the refinement are all deterministic functions of the input, so re-running the harness produces byte-identical output, which the threats section records as verified across all five libraries. The harness snapshots the per-method clause counts before refinement, runs the refinement, and diffs — a procedure that isolates exactly the clauses the second pass adds, attributing nothing to the single pass that the single pass did not produce. This isolation is what licenses the headline claim’s form: not that the refined specifications are large, but that they are larger than the single-pass baseline by a specific, measured amount, on the same methods, under the same inference, with only the second pass varied.

6.5.3 Headline Result (RQ4)

Table 6.1 reports the headline measurements across the five libraries. The second pass strictly extends the isolated set on every library: it adds at least one clause to between 24.33% and 43.85% of the methods in each library’s cache, a median of 36.33%, and adds 42,370 new precondition clauses and 25,203 new postcondition clauses in total. The single-

pass propagation is not subsuming on any library studied, and the gap is consistent across idiom families.

Table 6.1: Effect of compositional refinement over the isolated single-pass baseline across five libraries. *Refined* counts methods whose spec gained at least one new clause; percentages are of methods in cache.

	Lang	IO	Math	jOOL	Vavr
Methods in cache	3,494	2,111	6,645	3,691	5,111
Methods refined	1,532	767	2,492	898	1,799
<i>% of cache</i>	43.85%	36.33%	37.50%	24.33%	35.20%
With new precondition	1,160	522	1,929	802	1,225
With new postcondition	1,270	666	1,969	467	1,387
New precondition clauses	5,763	2,001	25,796	5,388	3,422
<i>avg/refined</i>	4.97	3.83	13.37	6.72	2.79
<i>max on one</i>	132	41	665	240	177
New postcondition clauses	5,206	2,015	11,745	3,223	3,014

The fraction of methods refined varies with idiom in an interpretable way. The utility library Commons Lang shows the highest fraction, because its delegating helpers expose many guarded call sites; the functional library Vavr shows fewer refined methods but concentrates its additions in disjunctive shapes from polymorphic dispatch, a consequence of its deep interface inheritance. The per-method dispersion is heavy-tailed — a small number of large façade methods account for the maxima of 132 to 665 new clauses — but the mean new preconditions per refined method (2.79 to 13.37) sits well below those maxima on every library, establishing that the refinement is a population-level effect rather than an artefact of a handful of large methods.

6.5.4 Shape Distribution

Table 6.2 categorises the newly added preconditions by syntactic shape. Branch-conditional implications — the shape the single pass structurally cannot produce, because it lacks the control-flow context the WP walk supplies — are the largest single shape on four of the five libraries, peaking at 64.1% on Commons Math, where heavy algorithmic code paths gate calls on numerical preconditions. The exception is jOOL, where null checks dominate because the higher-order combinators delegate to user-supplied lambdas through a small set of pinch-point validation sites; even there, branch-conditional shapes account for a substantial third of the additions. Disjunctive clauses — the polymorphic-dispatch signal, the other shape the single pass cannot produce — are present on every library and peak at 9.7% on Vavr, the library with the deepest interface inheritance chains.

Null and range/bound clauses make up most of the remainder. These are obligations the single pass *would* produce when the call site is unguarded; the second pass adds them when the call site is guarded by a condition the single pass would not lift. The dominance of branch-conditional and disjunctive shapes is the chapter’s central evidence: between

Table 6.2: Syntactic shape of the newly added preconditions across the five libraries. Branch-conditional implications dominate on four of five; disjunctive clauses are present on every library.

Shape (% of new precondition.)	Lang	IO	Math	jOOL	Vavr
Branch-conditional ($\Rightarrow$)	52.6%	42.8%	64.1%	34.1%	54.2%
Null check	21.8%	34.7%	24.1%	60.2%	30.9%
Range / bound	19.8%	18.9%	7.0%	0.5%	4.5%
Disjunctive ($\vee$)	4.3%	1.8%	3.7%	5.1%	9.7%
Other	1.5%	1.7%	1.2%	0.2%	0.6%

38% and 74% of the new clauses on each library are of shapes that the single pass cannot express by construction, which is direct confirmation that the architectural distinction between single-pass propagation and compositional refinement is real and consequential, not merely a difference in implementation thoroughness.

6.5.5 Interpreting the Magnitude

The headline figure — 42,370 new precondition clauses — is large enough to warrant interpretation, because a raw count can mislead in either direction. On one hand, the count aggregates over twenty-one thousand methods, so the per-method average is modest, and a reader should not imagine every method gaining hundreds of clauses; the median refined method gains a handful, and the large counts are concentrated in the façade methods that fan out to many callees. On the other hand, the count is a lower bound, for the reasons the substitution-soundness and callee-resolution discussions establish: every rejected substitution and every unresolved call site is a clause the pass declined to add under a sound policy, so the true compositional gain exceeds the reported one.

The figure that matters more than the raw count is the fraction of methods refined — between a quarter and nearly a half of each library’s cache — because it speaks to how pervasive the phenomenon is. That single-pass propagation leaves something to recover on a quarter to a half of all methods, across five libraries of different idioms, establishes that the gap is not a corner case affecting a few unusual methods but a routine consequence of how cross-method obligations arise. A gap affecting one method in a hundred would be a curiosity; a gap affecting one in three is a structural property of the single-pass architecture, and it is the pervasiveness, not the raw clause count, that makes the case for the second pass.

The shape distribution sharpens the interpretation once more. If the additions were dominated by null checks and range bounds — the shapes the single pass produces when call sites are unguarded — one could argue the second pass merely recovers clauses the single pass missed by oversight, and a more careful single pass would close the gap. But the additions are dominated by branch-conditional implications and include disjunctions on every library, and these are shapes the single pass cannot produce by construction,

not by oversight. The magnitude is therefore significant not merely for its size but for its composition: it is large, it is pervasive, and it is concentrated in exactly the shapes that demonstrate the architectural distinction is real.

6.5.6 Runtime Cost

The pass completes in 44 seconds total across all five libraries and 21,052 methods, an average of 2.1 milliseconds per method, ranging from 0.7 seconds on Commons IO to 20.3 seconds on the largest library, Commons Math. No strongly connected component hit the fixpoint cap on any library, because the call graphs are acyclic under the analyser’s signature-matching scheme and each method is refined once. The cost is dominated by the per-method AST walk for call sites, which is linear in body size; the lifting and substitution steps add constant factors per call site. The refinement is therefore an addition of seconds to tens of seconds on top of an inference pass that already takes minutes on libraries of this scale — a clearly affordable enhancement, which answers the cost half of RQ4.

The scaling behaviour across the five libraries confirms the linear character. The twenty-second figure on Commons Math, the largest library, against the sub-second figure on Commons IO, the smallest, tracks the ratio of their method counts and call-site densities rather than any superlinear effect: Math has both more methods and, as a numerical library, more call sites per method, and its runtime reflects the product. No library exhibits the runtime explosion that a fixpoint iteration on a deeply cyclic call graph would produce, because no library’s call graph is cyclic under the analyser’s resolution, and the dormant fixpoint machinery contributes nothing to the measured times. The practical consequence is that a deployer can predict the refinement cost from the corpus size — roughly two milliseconds per method — without worrying about pathological cases, because the pathological case (deep mutual recursion forcing many fixpoint iterations) does not arise in the libraries measured and would, if it did, be bounded by the configurable cap. The cost is thus not only affordable but predictable, which is the property a deployer integrating the pass into a build needs.

6.6 The Character of Each Library

The five libraries were chosen to span idiom families, and the shape distributions of Table 6.2 reflect their characters in ways that illuminate what drives compositional refinement. Reading the libraries one at a time turns the aggregate result into an account of *why* the second pass finds what it finds.

Apache Commons Lang, the utility library, shows the highest fraction of methods refined (43.85%) and a branch-conditional-dominated shape distribution. This follows from its design: it is a collection of small static helpers that delegate to one another and to validation utilities, and the delegation happens inside guards — a helper checks a condition

and, on one branch, calls a more primitive helper that imposes its own requirement. The guarded delegation is exactly the structure the branch-conditional lifting recovers, and the library's density of it is why nearly half its methods gain a clause. The library is, in a sense, the ideal case for the second pass: small methods, heavy delegation, guarded calls.

Commons IO, the I/O abstraction library, shows a lower refined fraction (36.33%) and a higher proportion of null checks among its additions. Its methods are larger and its delegation shallower, but its file and stream abstractions impose many non-null requirements — a stream must be open, a path must be non-null — that propagate through call chains. The branch-conditional fraction is still substantial, reflecting the defensive guarding common in I/O code, but the null check is more prominent than in Lang because the domain's central obligations are non-nullness obligations.

Commons Math, the numerical library, is the outlier in magnitude: it adds 25,796 new precondition clauses, more than the other four libraries combined, and its branch-conditional fraction peaks at 64.1%. The explanation is the structure of numerical code, which is dense with guarded calls into validation and computation helpers — a method computing a statistic checks the data's validity and, on the valid branch, calls a helper that requires it, and the helper in turn guards further calls. The deep, guarded call chains characteristic of numerical algorithms produce branch-conditional obligations at every level, and the second pass lifts them all, which is why a single large façade method can gain hundreds of clauses. Commons Math is the case that most stresses the pass and most rewards it.

jOOL, the functional-combinator library, inverts the pattern: null checks dominate its additions at 60.2%, and branch-conditional clauses, while still a substantial third, are less prevalent than elsewhere. The reason is the higher-order style. jOOL's combinators take user-supplied functions and delegate to them through a small set of validation pinch-points — a combinator checks its function arguments are non-null and then applies them — so the dominant propagated obligation is the non-nullness of the supplied function, not a guarded numerical precondition. The functional idiom concentrates obligations at the boundary where user code enters, and that boundary is a null check.

Vavr, the persistent-collection library, shows the highest disjunctive fraction at 9.7%, the polymorphic-dispatch signal. Its deep interface inheritance — a value type implementing a traversable interface implementing an iterable interface, and so on — means that a call through one of its interfaces may dispatch to any of many implementations, and the second pass emits the disjunction over candidates at each such site. Vavr is the case that most exercises the polymorphic-dispatch half of the pass, and its disjunctive fraction is direct evidence that the half is doing real work: a library that uses polymorphism heavily produces the disjunctive obligations a library that does not would lack.

The five libraries thus form a spectrum, and the second pass responds to each according to its idiom: guarded delegation in Lang and Math produces branch-conditional clauses,

boundary validation in IO and jOOL produces null checks, and deep polymorphism in Vavr produces disjunctions. The aggregate result — that the single pass is not subsuming on any of them — holds across the spectrum, but the *reason* it holds differs by library, and the per-library reading is what shows that the result is not an artefact of one coding style but a general consequence of how cross-method obligations arise in code.

6.7 From Compositional Dependence to Compatibility Analysis

The measurement establishes the fact the compatibility application rests on: a caller’s specification depends substantially on its callees’, through cross-method obligations a single-pass analysis cannot express. This section develops what that fact makes possible, which is the thesis’s most direct answer to the adoption question — a capability manual specification never made routine.

Consider a consumer whose code calls a library, and a new release of that library. Three kinds of change can occur at the library boundary. The library may change a method’s *signature*, which the compiler detects; this is the only kind of incompatibility today’s tooling reliably catches. It may preserve the signature but change the method’s *behaviour* — tightening a precondition so that inputs the old version accepted now violate the contract, or weakening a postcondition so that guarantees the caller relied on no longer hold. This second kind, behavioural incompatibility under signature compatibility, is the dangerous one: it compiles, it may even pass shallow tests, and it breaks in the cases the tests do not cover. The third kind is no change at all, and a consumer wants to distinguish a genuinely compatible upgrade from a dangerous one without running their entire test suite against every release and hoping the suite covers the changed behaviour.

Inferred, library-boundary specifications make the second kind detectable. The inferer, run over the old and the new library versions, produces a specification for each method in each version; comparing the two reveals which methods’ contracts changed and how — which preconditions strengthened, which postconditions weakened. A strengthened callee precondition is a candidate incompatibility: the caller may pass arguments the new version rejects. A weakened callee postcondition is another: the caller may rely on a guarantee the new version no longer makes. And because the compositional analysis of this chapter propagates callee contracts into caller contracts, the analysis can go further than flagging the library-side change: it can recompute the *caller’s* specification against the new library and compare it to the caller’s specification against the old, surfacing precisely the caller obligations that the library change altered. A caller obligation that appears only against the new version is a new demand the upgrade places on the caller; one that disappears is a guarantee the caller has lost. The cross-method obligations this chapter measured — the guard-conditional clauses lifted from guarded calls, the disjunctions from polymorphic

dispatch — are exactly the channels through which a library change reaches the caller, which is why the shape distribution matters: it shows that the propagation carries the kinds of obligation that behavioural change travels through.

The thesis establishes the mechanism — the substantial, measured dependence of caller specifications on callee specifications — but does not run the version-to-version experiment that would demonstrate compatibility detection end to end. That experiment, recomputing caller specifications against successive library versions and reporting the incompatibilities the comparison surfaces, is the foremost item of future work (Chapter 8), and the present chapter’s contribution is to show that the signal it would rely on is real and substantial. What the chapter demonstrates is the prerequisite: that a library’s specifications propagate into its callers’ strongly enough, and through the right structural channels, for their version-to-version change to carry the behavioural- compatibility information a consumer needs. The compatibility application is the capability this dependence enables, and naming it precisely — mechanism demonstrated here, end-to-end experiment deferred — is the honest statement of how far the thesis takes it.

6.8 Preliminary Downstream Signal

The structural result establishes that the second pass adds clauses; it does not by itself establish that the added clauses help a downstream consumer. A preliminary probe connects the refinement back to the test-generation result of Chapter 4. The experiment re-runs the test-generation comparison with two configurations of the specification context: the isolated baseline specification (the P3 condition of Chapter 4) and the compositionally refined specification, on the eleven-class Commons Lang façade, five generation runs per class.

On the eight of eleven classes that returned extractable tests in both configurations, the refined specification yields a 9.1% higher mean test count, with six of the eight showing an increase. The largest gains are on validation-heavy classes — **Validate** (+54.8%) and **BooleanUtils** (+27.5%) — where the refined specification’s branch-conditional clauses give the model additional case structure to test. The single substantial regression is **CharUtils** (−23%), where the disjunctive obligations from the class’s many overloaded predicate methods appear to be interpreted by the model as one combined assertion rather than several distinct cases. The aggregate is a weak positive signal with high per-class variance, and the small sample precludes a meaningful significance test.

The **CharUtils** regression repays closer attention, because a negative result in a downstream probe is as informative as a positive one. **CharUtils** is dense with overloaded predicate methods — variants of **isAscii** and its relatives — and the refinement pass, encountering calls through these overloads, emits disjunctions over the candidates’ preconditions. The disjunction is sound and informative to a verifier, but to a language model

generating tests it appears to read as a single combined assertion rather than as an enumeration of cases, so the model produces one test where the baseline specification's simpler clauses prompted several. The richer specification, in this instance, gives the model a clause whose structure it handles less well than the simpler clause it replaces. This is a genuine finding about the interaction between specification shape and model consumption: not every structurally richer specification is a better prompt, and a disjunction that helps a verifier may hinder a generator that does not decompose it. The observation does not undermine the aggregate positive signal — six of eight classes improve — but it identifies a specific mechanism by which refinement can backfire for a particular consumer, which is exactly the kind of nuance a downstream evaluation should surface and a structural measurement alone would miss.

The signal is reported with its limitations stated plainly. It measures the number of tests the model emits, not the mutation-killing rate, because the generated tests did not all compile cleanly in the measurement environment — a tooling issue in the test-generation harness and in the model's occasional emission of non-Java tokens, not a property of the specifications. A full mutation-coverage comparison between baseline and refined specifications is the natural next step and is identified as the top-priority follow-up. What the probe establishes is narrower but real: the delta between baseline and refined specifications is non-zero at the test-generation step, which is consistent with the hypothesis that the additional clauses carry downstream value and motivates the fuller evaluation.

The fuller evaluation, when it is done, should measure mutation score rather than test count, on a corpus large enough to support a significance test, with the generated tests compiling and running so that their fault-detection power can be assessed directly. The hypothesis it would test is specific: that the branch-conditional and disjunctive clauses the refinement adds equip a model to generate tests that kill mutants the baseline specification's tests miss — mutants on the guarded paths the branch-conditional clauses describe and the dispatch targets the disjunctions enumerate. The probe's positive-but-noisy test-count signal is consistent with this hypothesis but does not establish it, because a test count says nothing about whether the additional tests kill additional mutants. The **CharUtils** regression is a caution that the relationship may not be uniform — a richer specification can, for a particular consumer and a particular clause shape, produce fewer or weaker tests — so the fuller evaluation should report the per-class distribution, not only the aggregate. The probe's role in the thesis is to establish that the question is worth the fuller study's cost: the delta is non-zero, the mechanism is plausible, and the regression shows the answer is not trivially positive, which is exactly the situation that warrants a careful measurement rather than an assumption.

6.9 Worked Examples of the Two Shapes

The two recoverable shapes are best seen on concrete code. Consider first a caller that reaches a guarded call:

```
// callee: requires divisor != 0
static int safeDivide(int a, int divisor) { return a / divisor; }

static int scale(int value, int factor) {
    if (factor != 0) {
        return safeDivide(value, factor);
    }
    return value;
}
```

The callee `safeDivide` carries the inferred precondition `divisor != 0`. Single-pass propagation, encountering the call `safeDivide(value, factor)`, substitutes the actual argument to obtain `factor != 0` and faces the dilemma of Section 6.2: adding it unconditionally would forbid calling `scale` with `factor == 0`, which the method explicitly handles by returning `value`; skipping it would lose the obligation. The refinement pass walks the body, finds the call enclosed in the guard `factor != 0`, and lifts the substituted precondition through the guard to obtain `factor != 0 ==> factor != 0` — which simplifies to a tautology here, correctly reflecting that the guard already discharges the callee’s requirement. The example is deliberately degenerate to show the mechanism; in the common case the guard and the callee’s requirement differ, as when a call to a method requiring a non-empty list is guarded by a size check, and the lifted clause `size > 0 ==> list.size() > 0` carries genuine, non-tautological information that the single pass cannot express.

Now consider polymorphic dispatch:

```
interface Shape { double area(); } // no precondition
class Disk implements Shape {
    // requires radius >= 0
    public double area() { return Math.PI * radius * radius; }
}
class Square implements Shape {
    // requires side >= 0
    public double area() { return side * side; }
}

static double totalArea(Shape s) { return s.area(); }
```

The call `s.area()` in `totalArea` dispatches at runtime to whichever implementation `s` actually is. Single-pass propagation resolves the call to the nominal type `Shape`, whose `area` declares no precondition, and so adds nothing. The refinement pass, consulting the call graph’s class-hierarchy index, enumerates the implementing candidates `Disk` and `Square`, collects their substituted preconditions, and emits the disjunction `radius >= 0`

`|| side >= 0`. The disjunction is the sound caller obligation: for the dispatch to be safe, at least one candidate’s precondition must hold, namely the one corresponding to the runtime type. This is the polymorphic-dispatch shape, and it is structurally invisible to a single pass that resolves each call to a single nominal callee. The shape’s prevalence on Vavr (Table 6.2), the library with the deepest interface inheritance, is a direct consequence of how much polymorphic dispatch its design employs.

6.10 Positioning

The unified review of compositional and interprocedural analysis (Section 2.11) located the position this propagation occupies along three axes — whether a technique produces JML, reasons compositionally across procedure boundaries, and checks its reasoning against a solver — and found the cell it fills (produces JML, reasons compositionally by weakest-precondition lifting, feeds an existing JML verifier) unoccupied by prior work. The measurement of this chapter substantiates that positioning: it demonstrates the compositional reasoning concretely, on real code at scale, through the guard-lifting and dispatch-disjunction steps that distinguish the approach from the genuinely-compositional memory-safety analysers (which do not produce JML), the deductive verifiers (which consume rather than produce specifications), and the template-and-prune inferers (which do not lift callee contracts into callers). The novelty is the combination, and the value the chapter adds to the positioning is the evidence that the combination produces a substantial, structurally characterisable body of obligations — the evidence on which the compatibility application of Section 6.7 rests.

6.11 Implications for Inferer Architecture

The measurement settles a design question that the single-pass-versus-second-pass choice poses for any specification inferer, and the settlement has consequences beyond the particular engine of this thesis. The question is whether the additional architectural complexity of a compositional pass is justified by the information it recovers, and the answer the measurement gives is conditional on what the inferer is for.

For an inferer whose output feeds a consumer that benefits from richer specifications — a verifier, a test generator, a language model — the second pass is clearly worth integrating: it adds tens of thousands of clauses across the corpus at a cost of seconds, and the clauses are of shapes the single pass cannot produce. The cost-benefit is lopsided in the second pass’s favour, and the preliminary downstream signal suggests the added clauses carry value to exactly the kind of consumer that motivates the thesis. For an inferer whose output is consumed eagerly and superficially — an IDE showing hover text, a tool that displays the first precondition — the additional clauses may not justify their runtime, and the second pass is reasonably left off. The thesis’s design makes the

pass opt-in for precisely this reason: the right choice depends on the consumer, and the measurement gives a deployer the information to choose.

A subtler implication concerns inferers that wish to remain single-pass for engineering simplicity. The measurement shows that the branch-conditional shape, which dominates the gap on four of five libraries, requires only the caller’s local control-flow context — the guard enclosing each call — which a single-pass analyser already traverses during its AST walk. Such an analyser could lift its substituted clauses through enclosing guards before adding them, capturing the dominant shape of the gap without the full machinery of a second pass over the call graph. The polymorphic-dispatch shape is harder to capture in a single pass, because it requires consulting the class hierarchy at the point of propagation, which is the kind of cross-cutting lookup a second pass is naturally organised to perform. The measurement thus decomposes the gap into an easy half a single pass could capture with a local extension and a hard half that wants a second pass, and it quantifies each, which is the information an inferer designer needs to decide how far to go.

6.12 Threats to Validity

Internal validity. The clause-count diffs are deterministic; rerunning the harness produces byte-identical output, verified across all five libraries. The shape categorisation uses regex-based bucketing with a documented order of precedence, so the buckets are reproducible; a small residual of arithmetic comparisons the regex does not match falls into the *other* bucket and does not affect the headline result.

External validity. The five libraries deliberately span four idiom families, including generics-heavy and numerical code, to address the concern that the result is utility-class-specific; it holds across all five. Application and framework code (web frameworks, middleware) have different call-graph shapes and are not represented; generalisation to them is open. Google Guava, included in the earlier chapters, is excluded here after a stack-overflow fix in the call-graph builder’s interface-inheritance walk, because its source-jar size exceeded the measurement budget; the qualitative conclusion would not change but the run was not completed.

Construct validity. “Refined” is operationalised as gaining at least one clause not in the baseline, using exact string equality, so two semantically equivalent but syntactically different clauses count as distinct — which makes the new-clause count an upper bound on genuinely new logical obligations, though the shape categorisation is unaffected. The callee-resolution step matches on name and arity, so a residual of call sites is lost to overload collisions; a missed call site can only under-count additions, so the reported numbers are a lower bound on the true compositional gain on every library. The downstream signal measures test count, not mutation kills, and rests on a sample of eight classes; it is reported

as preliminary for exactly these reasons.

Conclusion validity. The structural claims are exact measurements, not statistical inferences, so no significance test applies to them; the conclusions hold for the corpora and the specific refinement implementation reported. The qualitative finding — that single-pass propagation cannot lift through guards or dispatch, and that a second pass recovers substantial clauses of exactly those shapes — has a structural explanation that suggests it will transfer, but quantitative transfer to other libraries is an open question.

6.13 Summary

This chapter asked whether the single-pass interprocedural propagation that makes inference practical captures the cross-method specification information a genuinely compositional analysis would, and answered by building and measuring a top-down weakest-precondition refinement pass. The answer is that the single pass is not subsuming: across five libraries spanning four idiom families and 21,052 methods, the second pass adds 42,370 precondition clauses, refining a quarter to nearly a half of each library’s methods, at a cost of seconds to tens of seconds. The additions are dominated by branch-conditional implications and include polymorphic-dispatch disjunctions on every library — the two shapes the single pass cannot produce, because both require the caller’s control-flow or class-hierarchy context that a substitution at the call site does not carry. The per-library analysis showed the result holds across idioms for idiom-specific reasons, and a preliminary downstream probe connected the additions back to the test-generation value of Chapter 4, finding a positive but noisy signal that motivates a fuller mutation-coverage evaluation.

The chapter’s contribution is twofold: a measurement, at a scale and across a breadth the literature has not previously reported, of what single-pass inference leaves uncaptured; and a characterisation of that residual as two structurally identifiable shapes rather than an undifferentiated gap. The measurement gives an inferrer designer the evidence to decide whether a second pass is worth its complexity, and the characterisation shows that the easy half of the gap — the branch-conditional shape — could be captured by a local extension to a single pass, while the harder half — the disjunctive shape — wants the second pass’s cross-cutting view of the class hierarchy. Beyond the design guidance, the measurement supplies the prerequisite for the thesis’s compatibility argument: it establishes that a caller’s specification depends on its callees’ substantially and through the structural channels — guard-conditional and dispatch-dependent obligations — through which a library change propagates to the code that uses it. This dependence is what makes specifications at the library boundary a basis for behavioural-compatibility analysis, the capability that Section 6.7 develops and that the thesis offers as a direct answer to the adoption question: a thing practitioners could not previously do, made possible by inferred, propagating, library-level specifications. The end-to-end version-comparison ex-

periment that would demonstrate it remains future work, but its mechanism is measured and real.

Chapter 7

Discussion and Synthesis

The studies of this thesis were presented in sequence, each answering a research question. This chapter steps back to consider them as a whole, and the whole is an argument about the adoption of formal methods. It argues that the studies together remove the principal barrier to that adoption and open a capability that did not previously exist (Section 7.1); it draws out the methodological thread that runs through all of them, the disciplined separation of heuristic inference from formal validation (Section 7.3); it examines what the studies jointly establish about the value of inferred specifications (Section 7.4); it confronts the tensions and limitations that cut across the chapters (Section 7.6); and it considers the combined threats to validity that no single chapter’s threats section captures (Section 7.9).

7.1 The Adoption Argument

The thesis began from a puzzle: formal methods are mature and effective, yet used only in a few niche domains. Its answer is that the binding constraint is the manual cost of the specifications verification requires, and the four studies form a single argument that this constraint is substantially removable — and that removing it opens a capability beyond cheaper verification.

The argument has two movements. The first establishes that inference removes the cost: the engine produces accurate specifications (Chapter 3, the fidelity result), and those specifications carry measurable value to a demanding consumer (Chapter 4, the test-generation result). Together these answer the obvious objection to automatic inference — that a cheap specification is a worthless one — by showing the inferred specifications are both accurate and useful. If the argument stopped here it would establish a worthwhile but incremental result: verification made cheaper, which lowers the entry cost to the niche without necessarily widening it.

The second movement is what bears on adoption most directly, and it is the thesis’s

distinctive claim. Specifications inferred and carried at the *library boundary* (Chapter 5) make a dependency’s contract available to a consumer who lacks its source — and because a caller’s specification is *derived from* its callees’ (Chapter 6), a change to a library propagates into the contracts of the code that uses it. This is the basis for behavioural-compatibility analysis across library versions: a capability that manual specification, confined to whole systems specified in isolation, never made routine, and one that answers a question — is this upgrade safe? — that every practitioner who manages dependencies faces. The adoption argument is therefore not merely “verification is now cheaper” but “inferred, propagating, library-boundary specifications offer something practitioners want and could not previously have,” which is the kind of new value that moves a technology out of a niche rather than merely lowering the cost of entering it.

The studies depend on one another in ways the individual chapters make concrete. The distribution mechanism is useful only because the inference produces something worth distributing, and its validation deliberately exercises the full inference-to-embedding path rather than synthetic specifications, to show it transports what the engine actually emits. The compositional analysis operates on the specifications the inference produces, and the compatibility capability it enables depends on the distribution that puts those specifications at the library boundary. The test-generation value is what makes the whole worth pursuing: without a demonstrated downstream benefit, the machinery would serve no purpose. The chapters can be read independently, but they describe one argument — that the specification-cost barrier to formal methods is removable, and that removing it at the library boundary opens compatibility analysis as a new capability.

A consequence worth drawing out is that the pipeline closes a loop the literature has left open. The verification community has mature tools to *check* specifications and the testing community has mature tools to *consume* them, but both have always required the specifications as input, and the manual cost of supplying that input is the bottleneck the introduction identified. The pipeline supplies the input automatically and gets it to the tools that need it, including across the source-to-binary boundary that separates a library’s author from its users. The contribution of the thesis is not any single stage — each has precursors — but the demonstration that the stages compose into a working whole whose output carries measurable value.

7.2 The Coherence of the Whole

Before examining the methodological thread and the cross-cutting threats, it is worth making explicit what binds the studies into a single argument rather than a sequence of separable results. The answer is that they share not only a system — though they do, every study using or extending JML-Inferer — but a single underlying question, pressed at each stage of the argument: *can the manual cost of specifications be removed, and removed in a way that opens new capability rather than merely cheapening an old one?* Inference

removes the cost of producing specifications; validation ensures the cost reduction does not sacrifice soundness; distribution puts the specifications at the library boundary where verification across dependencies happens; and the compositional measurement establishes the caller-on-callee dependence on which behavioural-compatibility analysis rests. Each study is the same question pressed against a different obstacle to the proposition that automatic, library-boundary specifications can move formal methods beyond the niche.

The coherence shows in how the studies' results depend on one another's framing. The distribution study's insistence on exercising the real inference pipeline, not just synthetic specifications, is motivated by the need to show it transports what the inference study actually produces. The compositional study's downstream probe is framed as a re-run of the test-generation study's experiment, so that its preliminary signal is measured on the same metric and is interpretable as a continuation of the value finding. The validation layer described in the inference chapter is the same layer whose discharge results the test-generation chapter reports and whose soundness guarantee the distribution chapter relies on when it claims to transport verified contracts. Remove any one study and the others lose not their individual results but part of their motivation: inference without value is unmotivated, value without reach is academic, reach without completeness is premature, and completeness without the prior three is a refinement of nothing. The thesis is the argument that the four together establish a proposition none establishes alone, and the coherence is the dependency among their framings, not merely their shared use of one system.

7.3 Heuristic Inference, Formal Validation

The deepest methodological commitment of the thesis, recurring in every chapter, is the disciplined separation of heuristic plausibility from formal soundness. The inference engine is heuristic: it matches syntactic patterns, and a pattern can fire where it should not. Rather than claim a soundness the engine cannot deliver, the thesis treats inference and verification as two independent layers in which the first proposes and the second disposes. This stance — infer aggressively, verify conservatively — is, in a sense, the methodological thesis underlying the technical one, and it is worth examining why it works.

The alternative stances each fail in an instructive way. A purely heuristic engine with no verification would emit unsound clauses with no recourse, and its output could not be trusted by a verifier or a safety-critical consumer. A purely sound engine — one that emits only clauses it can prove by construction — would be a verification-condition generator in disguise, and would inherit the scalability problems that make full WP computation impractical: it would need the loop invariants that are the hardest thing to infer, and it would emit nothing for the methods where it could not complete the proof. The two-layer architecture escapes both failure modes. The heuristic layer is free to be aggressive, emitting any clause a pattern suggests, because the validation layer will catch the unsound ones; the

validation layer is free to be conservative, rejecting any clause it cannot prove, because the heuristic layer has already done the creative work of proposing candidates. The division of labour is between a creative-but-fallible proposer and a rigorous-but-uncreative checker, and it is the same division that underlies Houdini’s guess-and-prune architecture [20], here grounded in code structure rather than in templates.

The independence of the two oracles — manual validation against documented intent, formal discharge against the implementation — is a refinement of the same theme that Chapter 4 exploits. The two oracles catch different errors: manual validation catches clauses that are formally consistent but miss the documented point, and formal discharge catches clauses that look right to a human reviewer but conflict with the code’s actual arithmetic or aliasing. Reporting them separately, rather than collapsing them into a single accuracy figure, is what allows the thesis to claim that its specifications are accurate under *both* a human and a machine oracle — a stronger claim than either alone, and one that the discharge rate exceeding the manual-validation precision (Chapter 4) makes vivid: the verifier accepts clauses the humans called incomplete, because completeness against documentation and consistency with implementation are genuinely different properties.

7.4 What the Studies Jointly Establish About Value

Each study contributes a piece of evidence about the value of inferred specifications, and the pieces reinforce one another.

Chapter 4 establishes the most direct value claim: supplying inferred specifications to a language model improves its test generation by a large and statistically robust margin — 272% more tests and 40.7 percentage points of mutation score over a signature-only baseline — and the improvement is not replicated by generic prompt enrichment, which isolates the specification as the active ingredient. This is value to a consumer, measured by the consumer’s own metric.

Chapter 5 establishes that this value is *reachable*. A specification that improves test generation is useless to a consumer who cannot obtain it, and most consumers — developers integrating a compiled dependency, models reading an API at inference time — cannot obtain a specification that lives only in the source tree. The 100% lossless embedding at sub-eleven-percent overhead establishes that the value demonstrated in Chapter 4 can be delivered to consumers who lack the source, which is the precondition for the value mattering at scale.

Chapter 6 establishes that there is *more* value available than the single-pass inference captures. The 42,370 clauses the second pass adds, dominated by shapes the single pass cannot express, are not hypothetical: they are produced by a working pass on real libraries, and the preliminary downstream signal — a positive though noisy effect on test count when the refined specifications are supplied to the model — connects them back to the value

metric of Chapter 4. The chapter establishes a ceiling that the earlier chapters were operating below, and a means of raising toward it.

Read together, the three value claims form an argument with a structure: inferred specifications are valuable (Chapter 4), the value is deliverable to those who need it (Chapter 5), and there is more value to be had by inferring more thoroughly (Chapter 6). No single chapter makes this argument; it emerges only from the conjunction, which is the sense in which the thesis is more than the sum of its three studies.

The argument's structure also clarifies what would weaken it. If the value claim failed — if specifications did not improve the consumer — the other two would be machinery serving no purpose, and the thesis would be a well-engineered solution to a non-problem. If the value held but the reach claim failed — if specifications could not be delivered without the source — the value would be confined to the teams that already have the source and need the specifications least, and the contribution would be academic. If value and reach held but the completeness claim showed the single pass already captured everything — if the second pass added nothing — the thesis would still stand on its first two claims but would lack the forward-looking element that the gap and its recovery supply. The three claims are thus not merely additive but mutually supporting: value justifies the machinery, reach makes the value matter, and completeness shows the value is not yet exhausted. The thesis's strength is that all three hold, and its argument is that their conjunction establishes the proposition — that automatic specifications can be as good as the practice needs — that motivates the whole.

7.5 The Economics of the Pipeline

The thesis's premise is economic — that the manual cost of specifications is the binding constraint — and the synthesis can make the economic argument the studies jointly support more explicit. The cost of a manually specified codebase scales with its size: every method needs a contract, each contract costs human effort, and the total is the per-contract cost times the method count, which for a large codebase is prohibitive. The pipeline replaces this with a different cost structure. Inference costs machine time proportional to code size but negligible human time; validation costs machine time for the solver and human time only for the flagged residual; distribution costs machine time and no human time; refinement costs machine time and no human time. The human cost, which dominated the manual approach and scaled with method count, is reduced to reviewing the flagged clauses — a fraction of the total — and scales with the failure rate rather than the method count.

This is the economic transformation the thesis demonstrates is achievable. The discharge results bound the human review cost: with over ninety percent of clauses discharged automatically and the flagged residual a few percent, the human reviews a small fraction

of the inferred clauses rather than authoring all of them. The throughput figures bound the machine cost: inference at milliseconds per file, embedding at thousands of methods per second, refinement at milliseconds per method, all of which disappear into a build that already takes longer. The net effect is that the cost of having specifications shifts from a human cost that scales with size to a machine cost that scales with size plus a human cost that scales with the much smaller failure rate — which is precisely the shift that makes specifications affordable for codebases where manual specification never was.

The economic argument has a limit the thesis is careful to respect. The inferred specifications are not the equal of expert hand-written ones in completeness — the recall gap is real, and a verification effort requiring strong, complete contracts would still need human authorship for the methods the engine specifies only partially. The economic transformation is not from manual to free but from manual to mostly-automatic-with-review, and its value is greatest where partial-but-abundant specifications suffice — test generation, lightweight verification, documentation, model context — which is exactly the set of consumers the thesis targets. For the heaviest verification, the pipeline lowers the entry cost without eliminating the expert effort the last mile requires, and the honest economic claim is that it shifts the cost structure favourably for most uses, not that it abolishes the cost for all.

7.6 Tensions and Design Trade-offs

Several tensions cut across the chapters, and acknowledging them is part of an honest account of the work.

The first is the *precision–recall tension* in inference. The engine favours precision: it emits a clause only when a pattern matches a structure it trusts, and it declines to guess where it is unsure, which is why a few methods receive only empty contracts. A more aggressive engine would emit more clauses and catch more of the documented intent, at the cost of more false positives for the validation layer to filter. The thesis sits deliberately on the precise side of this trade-off, because the downstream consumers — a verifier, a test generator — are better served by a smaller set of trustworthy clauses than by a larger set of doubtful ones, and because the comparison with language-model inference (Chapter 4) shows that the imprecise-but-complete strategy produces exactly the hallucinated clauses the thesis aims to avoid.

The second is the *heuristic-versus-formal tension* in coverage. The engine’s heuristic front end can emit clauses the formal back end cannot discharge — the 7.1% of clauses that hit the three tractability limits of Chapter 4. These clauses are not wrong; they are unverifiable at the current strictness with the current solver, and the thesis retains them as a separate class (useful as oracle information to a language model, outside the verified-sound surface) rather than discarding them. The tension is between the engine’s

reach and the verifier’s grasp, and it is resolved not by restricting the engine to what the verifier can prove — which would discard genuinely useful clauses — but by labelling each clause with its verification status and letting the consumer decide how much to trust the unverified ones.

The third is the *single-pass-versus-compositional tension* in interprocedural analysis, which Chapter 6 makes explicit. The single pass is fast, simple, and sound for the unconditional case; the compositional pass is richer but more complex. The thesis does not argue that the compositional pass should always run — it is an opt-in enhancement, and for a consumer that wants only the cheap unconditional obligations the single pass suffices — but it does establish that the choice is a real one with measurable consequences, and that an inferrer wishing to remain single-pass could capture the easy half of the gap (branch-guard lifting, which needs only the local control-flow context) without the harder half (polymorphic disjunction, which needs the class hierarchy). Naming the trade-off precisely, and quantifying what each side costs and yields, is itself a contribution.

7.7 The Language Model as Producer and Consumer

A theme that surfaces in two of the studies, and that the synthesis can draw out more sharply than either study alone, is the asymmetric role of the language model in the specification life cycle. The model appears twice: as a candidate *producer* of specifications (the comparison baseline of Chapter 4) and as a *consumer* of them (the central experiment of Chapter 4 and the downstream probe of Chapter 6). The two roles produce opposite verdicts, and the contrast is instructive.

As a producer, the model is weaker than the heuristic engine on the dimensions that matter for a downstream verifier: it attains lower precision, because it hallucinates clauses that are plausible in surface form but false against the code, and far lower run-to-run consistency, because its sampling is stochastic where the static analysis is deterministic. The model’s productions are not grounded in the code’s structure — it generates a specification the way it generates any text, by prediction — and the lack of grounding shows up as exactly the failure mode a grounded analysis avoids. As a consumer, by contrast, the model is excellent: given a grounded specification, it produces markedly better tests, because the specification supplies the statement of intent the model cannot reliably manufacture for itself. The model that hallucinates when asked to produce a contract uses a supplied contract faithfully.

The asymmetry has a clean explanation. Producing a sound specification requires grounding in the code’s actual semantics, which the heuristic engine has (it reads the AST) and the model lacks (it predicts from text). Consuming a specification requires fluency in turning intent into tests, which the model has in abundance and the static engine lacks entirely. The two are complementary in exactly the way the thesis’s architecture exploits:

a grounded producer feeds a fluent consumer, and the combination outperforms either alone. The methodological caution that the thesis maintains — use the static engine to produce, use the model to consume — is not a prejudice against language models but a consequence of where each kind of system is strong. A future system might close the producer gap by grounding a model’s specification generation in a verifier’s feedback, in the spirit of Houdini’s propose-and-prune loop with a model as the proposer; the thesis does not build that system, but its results delineate the niche such a system would have to fill.

7.8 Toward Automated Specification Engineering

Stepping back from the individual results, the thesis can be read as an argument for a broader proposition: that specifications, long treated as a manually authored input to the verification and testing pipeline, can become an automatically maintained *artefact* of it, computed from code, certified against it, distributed with it, and refined as analysis improves. The four stages of the pipeline correspond to the four operations one would want on such an artefact — create it, check it, ship it, improve it — and the thesis demonstrates each on real code.

The proposition matters because it changes the economics that have kept formal specifications marginal. If a specification is a manual artefact, every method that lacks one is a method whose specification will probably never be written, because the marginal cost falls on a human who has other work. If a specification is an automatically maintained artefact, every method has one by default, kept current by re-running the inferer in continuous integration, certified by re-running the verifier, and shipped by re-running the embedder. The blank page that the introduction identified as the barrier is filled not once, heroically, but continuously, mechanically, as a side effect of the build. The cost that Matichuk et al. [39] measured as scaling with specification volume is paid by a machine whose throughput (Chapter 3’s 88 milliseconds per file, Chapter 5’s thousands of methods per second) makes the volume irrelevant.

This is, of course, an aspiration that the thesis only partially realises. The specifications the engine produces are accurate but incomplete; the verifier discharges most but not all of them; the distribution is lossless but the inference behind it is heuristic; the refinement recovers compositional information but its downstream value is so far only a preliminary signal. The thesis demonstrates that each stage works on well-structured library code, not that the pipeline is ready to maintain the specifications of an arbitrary enterprise system. But the demonstration is enough to establish the proposition’s plausibility: the operations compose, they run at practical speed on real libraries, and their output carries measurable value. Whether the proposition scales to the full diversity of production software — the application code, the framework code, the concurrent and reflective and dynamically-loaded code that the controlled corpora exclude — is the question

that the thesis’s limitations mark out as the frontier, and that its future work proposes to approach.

7.9 Combined Threats to Validity

Each chapter states the threats specific to its study; this section addresses the threats that span them, which no single chapter’s threats section captures.

The most important cross-cutting threat is *corpus homogeneity*. Apache Commons Lang appears in all three studies, Commons IO in two, and the libraries are uniformly mature, well-structured, open-source utility and value-type code. This is a deliberate choice — using shared corpora keeps cross-chapter comparisons free of corpus-difference confounds, and mature libraries are the realistic target for specification inference — but it bounds external validity in a way that compounds across chapters. A result that holds on Commons Lang in all three studies is well-established on Commons Lang and correspondingly less established elsewhere. Chapter 6 mitigates this most directly, deliberately spanning four idiom families including generics-heavy and numerical code and finding the result holds across all five libraries; but application code, framework code, and proprietary enterprise code remain unrepresented in every study, and the *other* method category — event handlers and callbacks — is empty in the controlled corpus of Chapter 4. Generalisation beyond well-structured library code is the single largest open question the thesis leaves, and it is named as such in every chapter that touches it.

The second cross-cutting threat concerns the *language model*. The test-generation study (Chapter 4) and the downstream probe (Chapter 6) both use a single model family, for budget reasons, and both defend an *ordering* of conditions rather than absolute scores. The defence rests on the model-independence of the JML format and on multi-model studies in the literature reporting consistent qualitative findings across model families, but a full multi-model replication at the scale of the thesis’s experiments is not performed and is named as future work. The risk is that the magnitude of the effects is model-specific even if their direction is not; the thesis claims only the direction.

The third cross-cutting threat is the *heuristic-soundness* concern that the two-layer architecture is designed to address but cannot fully eliminate. Every clause is discharged through OpenJML, but discharge is itself bounded by the solver’s tractability, and the unsupported clauses (Chapter 4) are neither proved nor refuted — they are retained on the strength of manual review and their usefulness as oracle information, outside the verified surface. A consumer that requires every clause to be machine-verified would discard these, and the thesis’s claim that its specifications are accurate must be read with the distinction between verified, flagged, and unsupported clauses kept in view. The architecture reduces the risk that unsound clauses reach a consumer; it does not reduce it to zero, and the honest statement is that the residual risk is concentrated in exactly the

clauses the chapter labels as unsupported.

The fourth, specific to the most recent work, is that the downstream signal of Chapter 6 measures test count rather than mutation kills, because the generated tests did not all compile cleanly. This is the weakest empirical link in the thesis, and it is reported as a preliminary signal rather than a result for exactly that reason. The fuller mutation-coverage comparison between baseline and refined specifications is the natural completion of the argument and the top-priority follow-up; until it is done, the claim that the compositional additions carry downstream value rests on a non-zero but noisy test-count delta rather than on the fault-detection metric the rest of the thesis relies on.

7.10 Engineering Lessons

Beyond its empirical findings, the thesis accumulated engineering lessons that recur across the studies and that are worth stating as guidance, because they are the kind of knowledge that a thesis organised around results does not otherwise record.

The first is that a high-but-imperfect aggregate should be disaggregated before it is accepted. The interface-method embedding bug of Chapter 5 hid behind a 96.9% roundtrip rate that looked like success; disaggregating the failures by program element revealed that every one was an interface method, which pointed directly at the missing lifecycle callback. The same discipline applies to the discharge results of Chapter 4, where disaggregating the non-discharged clauses by structural pattern revealed the three tractability limits rather than an undifferentiated failure rate. The lesson is that an aggregate metric is a starting point for investigation, not an endpoint, and that the residual in any imperfect measurement usually has a specific, findable cause.

The second is that the cases a system handles silently are the cases most likely to harbour bugs, precisely because they generate no signal when they fail. Interface methods received no embedded annotation and threw no error; the failure was visible only in the aggregate roundtrip rate, not in any exception or log. A test suite that asserted the presence of an embedded annotation on every method, including those without bodies, would have caught the bug immediately. The lesson is to test the silent cases explicitly, because the loud cases test themselves.

The third is that a deliberately conservative policy — rejecting unsound substitutions, gating clauses by return type and termination, declining to infer where uncertain — costs recall but buys a guarantee worth more than the recall it costs. Across all three studies the engine favours the conservative choice, and the consequence is a lower-bound character to its claims: the inferred specifications are at least as accurate as reported, the compositional gain is at least as large, the embedding loses at most nothing. The lesson is that for a tool whose output feeds a verifier or a test generator, the conservative bias is the right one, because a missed clause is recoverable by a later, more aggressive analysis while an

unsound clause that reaches a consumer is not.

7.11 The Recurring Role of Negative Results

A feature of the thesis that the synthesis can highlight is the consistent value of its negative and surprising results, which arise directly from the commitment to measurement on real code. The studies are not a sequence of confirmations; they include findings that ran against the initial expectation, and these are among the most informative.

The clearest is the per-spec compression result of Chapter 5. The intuition that compressing each specification would reduce the embedding overhead is natural, and the measurement refutes it decisively: per-spec DEFLATE made the overhead dramatically worse, because the payloads are too small for the compressor to find redundancy and the compressed payloads forfeit the constant-pool deduplication the uncompressed format enjoys. The negative result is informative beyond its immediate context: it shows that the right granularity for any compression of embedded specifications is the class file, not the individual specification, because only at the class-file level is there enough shared structure to amortise. A study that reported only the successful optimisations would have left a deployer free to try per-spec compression and rediscover the failure.

A second is the P2-versus-P3 inversion of Chapter 4: the guidance condition produces more tests but a lower mutation score than the specification condition. The result was the point of including the guidance control, but its magnitude — that more tests can mean worse fault detection — is a finding with methodological reach, cautioning the entire test-generation field against test count as a quality proxy. A third is the precise characterisation of the three tractability limits: rather than reporting an undifferentiated discharge-failure rate, the study identifies the structural patterns that defeat the solver, turning a negative result (some clauses do not discharge) into positive knowledge (these specific shapes exceed these specific solver capabilities). A fourth is the Guava-overhead inversion, where real inferred specifications produced *lower* overhead than synthetic ones on the largest library, contrary to the expectation that real specifications are richer and therefore costlier — explained by the larger denominator of Guava’s dense generic classes.

These results share a provenance: they came from measuring rather than assuming, on real artefacts whose properties did not match the intuition. The thesis’s commitment to measurement on production code is what surfaced them, and their presence is evidence that the commitment paid off — a study that argued from construction would have asserted the intuitions and missed the corrections.

7.12 What Would Falsify the Thesis

A claim worth defending is a claim that could in principle be refuted, and stating what would falsify the thesis’s central claim sharpens what the evidence does and does not establish. The thesis claims that heuristic inference, validated against a formal oracle, produces specifications accurate enough to be useful, distributable, and valuable to consumers. Several findings would have refuted it, and the studies were designed so that they could have.

The fidelity claim would have been refuted by a low discharge rate or a low manual-validation precision — if the verifier had rejected most inferred clauses, or if the raters had judged most of them incorrect, the specifications would not be accurate enough to be useful. The observed rates (above 90% on both oracles) do not refute it, but they could have; an engine whose patterns fired indiscriminately would have produced exactly the low rates that would falsify the claim. The value claim would have been refuted by the guidance control: if the guidance condition had matched or exceeded the specification condition on mutation score, the effect would have been attributable to prompt enrichment in general rather than to specifications in particular, and the value of specifications specifically would be unestablished. The control did not refute the claim — guidance produced more tests but a lower mutation score — but it was included precisely because it could have, and a weaker experimental design that omitted it would have left the claim vulnerable to exactly that alternative explanation.

The reach claim would have been refuted by a roundtrip rate below 100% that resisted explanation, or by an overhead so large as to be impractical; the interface-method bug produced a sub-100% rate that, had it been accepted rather than disaggregated, would have left the lossless claim unproven. The completeness claim would have been refuted by a second pass that added few clauses, or clauses of shapes the single pass could already produce — if the refinement had merely duplicated the single pass’s output, it would have shown the single pass already compositional. The observed additions (tens of thousands of clauses, dominated by shapes the single pass cannot express) do not refute it, but a single pass that was already capturing the guarded and polymorphic obligations would have produced the small, redundant additions that would falsify the claim.

That each claim had an identifiable way to fail, built into the design of the study that tested it, is what distinguishes the thesis’s empirical assertions from mere descriptions of a system that works. The claims survive not because they could not have failed but because they were tested in ways that would have exposed them if they were false, and the surviving claims are correspondingly load-bearing.

7.13 Summary

The studies compose into a single argument about why formal methods stay niche and what would change it. The barrier is the manual cost of specifications; inference removes it, producing specifications that are accurate (Chapter 3) and valuable (Chapter 4); and inferred specifications carried at the library boundary (Chapter 5), because a caller's specification depends on its callees' (Chapter 6), make verification across dependencies and behavioural-compatibility analysis — the latter a capability manual specification never made routine — available to ordinary software. The argument rests throughout on the disciplined separation of heuristic inference from formal validation, the proposer-and-checker division that lets the engine be aggressive and the verifier conservative. The principal limitations are the homogeneity of the corpus, the use of a single model family, the residual unverified clauses, and that the compatibility capability is established as a mechanism rather than demonstrated end to end — each named in the chapter it most affects and each pointing toward the future work the next chapter sets out.

The synthesis has aimed to show that the thesis is a single argument rather than several. The argument is that the manual cost of specifications, long the binding constraint on the adoption of formal methods, can be lifted by automatic inference without sacrificing the soundness verification provides or the reach distribution provides — and that the resulting library-boundary specifications, propagating from callee to caller, open behavioural-compatibility analysis as a new capability. Each study removes one obstacle: inference removes the production cost, validation removes the soundness objection, distribution removes the reach objection, and the compositional measurement establishes the cross-library propagation the compatibility capability depends on. The obstacles are removed not in the abstract but by measurement on production code, and the residual limitations are the honest record of where the removal is partial. What remains, after the studies, is an argument that has survived the threats each was designed to expose it to, on the evidence each was designed to gather — which is the strongest form an empirical thesis can take, and the form this one aspires to.

Chapter 8

Conclusion and Future Work

8.1 Summary of Contributions

This thesis began from a puzzle: formal methods are mature and effective, yet used only in a few niche corners of software engineering. Its diagnosis is that the binding constraint is not the verification technology but the manual cost of the specifications that technology requires, and its programme was to ask whether automatic inference — and the capabilities inferred specifications enable — could move formal methods out of the niche. It pursued the question through four research questions, the first two asking whether inference removes the cost barrier and the second two establishing the mechanisms by which library-boundary specifications enable verification and compatibility analysis.

The answers, established empirically on production Java code, are as follows.

On inference accuracy (RQ1). Java Modeling Language preconditions, postconditions, and loop invariants can be inferred from source by pattern matching organised around the weakest-precondition and strongest-postcondition calculi, at a precision of 94.2% for preconditions and 87.6% for postconditions against human-judged intent, with 92.9% of all inferred clauses discharged by an SMT-backed verifier against the implementation. The inference is heuristic and not sound by construction; soundness is recovered *a posteriori* by discharging every clause and flagging the failures. The accuracy is high enough that automatic inference is a credible substitute for the manual authoring that constitutes the adoption barrier.

On downstream value (RQ2). Cheaply inferred specifications deliver real value: supplied as context to a large-language-model test generator, they improve its unit-test generation by a large and statistically robust margin — 272% more tests and a mutation score 40.7 percentage points higher than a signature-only baseline. A guidance-only control rules out the alternative explanation that any prompt enrichment would suffice, establishing that the specification is the active ingredient. The cost reduction is therefore worth having: the cheaply produced specifications are not cheap in value.

On library-boundary availability (RQ3). Inferred JML contracts can be carried with compiled Java libraries and service-interface descriptions, recovered losslessly by a consumer without the source, at a bytecode-size overhead below eleven percent and a throughput compatible with routine workflows. Across four corpora exceeding twenty thousand methods the embedder achieves a 100% lossless-roundtrip rate through a toolchain that requires no modification. This makes a dependency’s contract available where verification of client code against it must happen — at the boundary, without the dependency’s source.

On compositional propagation (RQ4). A caller’s specification depends substantially on its callees’: a top-down compositional analysis, measured across five libraries and 21,052 methods, recovers 42,370 cross-method obligations the single-pass propagation cannot express, dominated by the guard-conditional and dispatch-dependent shapes through which a library change reaches the code that calls it. This dependence is the mechanism that makes library-boundary specifications a basis for behavioural-compatibility analysis: when a library’s inferred contracts change between versions, the impact on its callers’ contracts is computable through exactly these obligations. The thesis establishes the mechanism and positions the end-to-end version-comparison experiment as its foremost future work.

The thesis statement asserted that the adoption of formal methods is constrained chiefly by the manual cost of specifications, that the constraint is substantially removable by inference validated against a formal oracle, and that library-boundary specifications — because caller specifications derive from callee specifications — make verification across dependencies and behavioural-compatibility analysis available to ordinary software. Each part is now backed by quantitative evidence on real-world code.

8.2 The Empirical Findings at a Glance

It is worth gathering the principal quantitative findings in one place, because their conjunction is the thesis’s evidentiary core and they are otherwise distributed across four chapters. On inference accuracy: the engine produces non-trivial specifications for 99.0% of the evaluation corpus, at 94.2% precondition precision and 87.6% postcondition precision against documented intent, with 92.9% of all inferred clauses discharged by the verifier against the implementation. On downstream value: specification-guided test generation produces 272% more tests and a mutation score 40.7 percentage points higher than signature-only generation, an effect size beyond conventional thresholds and not reproduced by a guidance-only control. On library-boundary availability: across four corpora exceeding twenty thousand methods, the embedder achieves a 100% lossless-roundtrip rate at a bytecode overhead below eleven percent and a throughput of thousands of methods per second. On compositional propagation: a caller’s specification depends on its callees’ through 42,370 cross-method obligations recovered across five libraries and 21,052 methods, refining a quarter to nearly a half of each library’s methods, with the guard-conditional and dispatch-dependent shapes — those through which a library change reaches its callers

— dominating.

These numbers are not independent assertions but a connected argument. The accuracy figures establish that inference removes the cost barrier; the value figures establish that the cheaply produced specifications are worth having; the availability figures establish that they can be delivered to the library boundary where verification of client code happens; and the propagation figures establish that a library’s specifications reach its callers’ contracts strongly enough to make behavioural-compatibility analysis a real prospect. Read together they answer the adoption question the thesis posed: the specification-cost barrier is removable, and removing it at the library boundary opens a capability — compatibility analysis across dependencies — that the manual-specification regime never made routine.

8.3 Reflections on the Methodology

Two methodological choices proved central enough to warrant restating as lessons. The first is the two-layer architecture: infer aggressively by heuristic pattern matching, validate conservatively by formal discharge. This division — a creative-but-fallible proposer and a rigorous-but-uncreative checker — is what allows the engine to be useful without being sound by construction, and it is, in retrospect, the single design decision that makes the rest of the thesis possible. An engine restricted to what it could prove would emit too little; an engine with no formal back-stop would emit clauses no consumer could trust. The separation is the resolution.

The second is the commitment to measurement on real code at scale. Every claim in the thesis that could be made by counting — clauses discharged, tests generated, bytes overhead, clauses added by a second pass — was made by counting, on production libraries rather than on constructed examples. The commitment is not merely a matter of rigour; it shaped what the thesis could discover. A study built on constructed examples discovers what its author thought to construct, and an author who believed per-spec compression would help, or that a high roundtrip rate was good enough, or that more tests meant better tests, would have constructed examples confirming those beliefs. Measurement on real artefacts refused those beliefs: the compression made things worse, the roundtrip gap hid a specific bug, the extra tests killed fewer mutants. The negative and surprising results that the synthesis chapter collects are the dividend of the commitment, and they are, in aggregate, as much a part of the thesis’s contribution as the confirmations, because they map the boundaries of the approach as precisely as the confirmations map its successes. The cost is that the conclusions are corpus-bound and quantitative rather than universal; the benefit is that they are defensible, and that the failures they surface — the three tractability limits, the interface-method embedding bug, the compile failures in the downstream probe — are real failures of real systems rather than artefacts of toy inputs. The thesis’s most useful negative results, including the counter-productive per-spec com-

pression of Chapter 5 and the precise characterisation of what the solver cannot discharge in Chapter 4, came directly from this commitment.

8.4 Limitations

The principal limitations have been stated in the chapters and synthesised in Chapter 7; they are collected here for completeness. The evaluation corpora are uniformly mature, well-structured, open-source library code, and the result that holds across them is correspondingly less established on application code, framework code, and proprietary systems, and on the event-handler and callback methods absent from the controlled corpus. The language-model experiments use a single model family and defend an ordering of conditions rather than absolute magnitudes. The formal validation is bounded by the solver’s tractability, leaving a residual class of clauses neither proved nor refuted. And the downstream value of the compositional refinement rests, as yet, on a test-count signal rather than on the mutation-kill metric the rest of the thesis relies on. None of these undermines the central claims, but each bounds their scope, and the honest summary is that the thesis establishes its conclusions firmly on well-structured library code with a single model family, and points beyond that scope rather than reaching it.

The limitations are also, read constructively, a map of where the most valuable next work lies, and they are not uniform in their cost to address. The single-model limitation is the cheapest to lift, requiring only the budget to re-run parameterised experiments, and lifting it would most directly strengthen the external validity of the value claim. The corpus-homogeneity limitation is more expensive, requiring new corpora of application and framework code and, for some method categories, new inference patterns, but it is the limitation whose lifting would most broaden the thesis’s reach. The residual-unverified-clauses limitation is the hardest, depending on advances in SMT reasoning and JML translation that the thesis does not itself deliver, and it bounds not the thesis’s claims — which concern the verified clauses — but the fraction of inferred clauses that can be moved into the verified class. The compositional-downstream limitation is the most immediately addressable of the empirical gaps, requiring only the repair of a test harness and a mutation run on existing artefacts. Ordering the limitations by the cost of addressing them is, in effect, the research roadmap the next section sets out, and the fact that the cheapest-to-address limitations are also among the most strengthening is the reason the thesis regards its conclusions as a foundation to build on rather than a ceiling reached.

8.5 Future Work

The thesis opens several directions, in rough order of immediacy.

The version-to-version compatibility experiment. The foremost direction, because it would convert the thesis’s central capability claim from a demonstrated mechanism into a demonstrated result, is the version-comparison experiment that Chapter 6 positions as future work. The thesis established that a caller’s specification depends substantially on its callees’, and argued that this dependence makes a library change’s impact on its callers’ contracts computable; the experiment that would close the argument runs the inferer over successive versions of a real library, recomputes the specifications of client code against each version, and reports the behavioural incompatibilities the comparison surfaces — strengthened callee preconditions the client may now violate, weakened callee postconditions the client relied on. Validated against known breaking changes from the library’s release history, such an experiment would show that specification-level comparison detects behavioural incompatibilities that signature compatibility misses, demonstrating the capability that is the thesis’s most direct answer to the adoption question. The mechanism is measured; the end-to-end demonstration is the work that remains.

Completing the compositional downstream evaluation. A second immediate gap is the one named at the end of Chapter 6: the downstream value of the more thoroughly propagated specifications is established only by a test-count signal, because the generated tests did not all compile cleanly. Completing the mutation-coverage comparison — repairing the test-generation harness so the generated suites compile and run, then measuring whether the propagated specifications yield a higher mutation score — would convert the preliminary signal into a result on the same metric the rest of the thesis uses, and would close the loop between Chapter 6 and the value metric of Chapter 4.

Service-boundary code. The test-generation result of Chapter 4 was obtained on pure library code, where the oracle problem is real but not at its most acute. The setting where it is most acute is service-boundary code: methods that call REST endpoints or RPC stubs, where the implementation delegates to an external service, the signature conveys nothing about status codes or retry semantics, and signature-only generation collapses into mocking the call. The distribution mechanism of Chapter 5 already extends to service interfaces via the OpenAPI extension, so the machinery is in place to ask whether inferred specifications retain or amplify the test-generation gain in that setting — the question is whether the effect reported here compounds where the oracle problem bites hardest.

Multi-model replication. The single-model limitation is mechanical to address and would strengthen every experiment that involves a language model. The replication packages are structured for it — the model client is parameterised and the prompts are model-independent — so the work is a matter of budget rather than of design. A multi-model study would establish whether the magnitudes of the effects, and not merely their direction, transfer across model families.

Stronger loop-invariant inference. The loop-invariant heuristics of Chapter 3 produce a non-trivial invariant for roughly eighty-five percent of loops, and the downstream consequence is measurable: the methods left with only a weak invariant attain materially lower test quality. Strengthening invariant inference — through machine-learning approaches, or through hybrid static-dynamic techniques in the spirit of dynamic invariant detection — would raise the floor on exactly the methods where the current engine is weakest, and would attack the loop-related cases among the three tractability limits.

Closing the verifier’s tractability gaps. The three tractability limits of Chapter 4 — recursive multiplicative growth, quantified invariants over mutable arrays, the for-each translation — are properties of the solver and the JML translation, not of the engine. Two of them are plausibly addressable: the for-each translation gap is a question of exposing the loop-variable-to-element relationship to user JML, and the mutable-array gap is a question of array-theory formulation that further work on the OpenJML fork might narrow. Closing them would move clauses from the unsupported class into the verified class, shrinking the residual risk that the synthesis chapter identified.

Single-pass capture of the easy compositional half. Chapter 6 established that the branch-conditional half of the compositional gap needs only local control-flow context, which a single-pass analyser already has during its AST walk. An inferer that wished to remain single-pass could in principle lift its substituted clauses through enclosing guards before adding them, capturing the dominant shape of the gap without the machinery of a second pass. Whether this hybrid is worth building — whether the engineering simplicity of a single pass is better preserved by extending it or by adding a clean second pass — is a design question the measurement makes precise but does not settle.

Beyond Java. The system and its evaluation are Java-specific. The WP/SP design discipline, the infer-then-verify architecture, and the distribution mechanism are not intrinsically Java-bound, and porting the approach to another language with a contract notation and a verifier — or to a language with neither, building both — would test how much of the thesis’s findings are about specification inference in general and how much about Java in particular.

Hybrid static-dynamic and ensembled inference. The thesis’s engine is purely static, and its complement — dynamic invariant detection — finds the numeric and relational invariants the static engine cannot see. A hybrid that ran both and took the union of their outputs, with the verifier filtering the combined set, would cover more of the specification space than either alone, and the infer-then-verify architecture extends naturally to it: both producers propose, the verifier disposes. The same architecture admits a learned producer — prompting a model to propose specifications and filtering its proposals

through the verifier, in the spirit of Houdini with a model as the proposer — which would test whether the model’s recall, combined with the verifier’s soundness filter, exceeds the static engine’s precision-favouring output. The producer-consumer asymmetry the thesis documents suggests where each producer is strong, and an ensemble that played to each producer’s strength is a direction the architecture invites.

8.6 A Research Roadmap

The future directions above can be ordered into a roadmap by their dependencies and their payoff. The highest-payoff step, requiring a new study but no new system, is the version-to-version compatibility experiment: run the inferer over successive releases of a real library, recompute client specifications against each, and report the behavioural incompatibilities the comparison surfaces, validated against the library’s known breaking changes. This is the step that would turn the thesis’s central capability claim — that library-boundary specifications support behavioural-compatibility analysis — from a measured mechanism into a demonstrated result, and it targets the adoption question most directly, because compatibility analysis across dependencies is the capability that distinguishes inferred library-boundary specifications from a merely cheaper version of what experts already write.

A second, more immediate step, requiring no new technique, is to complete the compositional downstream evaluation: repair the test-generation harness so the generated suites compile, and measure whether the more thoroughly propagated specifications raise the mutation score, converting the preliminary test-count signal into a result on the thesis’s primary metric. This closes the one empirical gap the thesis leaves within its own scope.

A third step, requiring modest engineering, is the multi-model replication of the test-generation experiments. The replication packages are structured for it, and it would establish whether the magnitudes of the effects transfer across model families or whether only their direction does — a consequential strengthening of the thesis’s external validity, achievable without a new technique.

A fourth step, requiring a new study but no new system, is the service-boundary evaluation. The distribution mechanism already reaches service interfaces through the OpenAPI extension, so the machinery is in place; what is needed is a corpus of service-boundary methods and a controlled experiment analogous to the library-code one, testing the hypothesis that the specification’s value compounds where the source cannot serve as an oracle. This targets the setting where the oracle problem is most acute.

A fifth step, requiring genuine new technique, is to attack the boundaries of heuristic inference and of solver discharge — the algorithmic postconditions the static engine cannot reach, the recursive and array-quantified clauses the solver cannot prove. Progress here is the hardest and least certain, depending on advances in invariant inference and in SMT

reasoning that the thesis does not itself deliver, but it is the direction that would raise the ceiling on what the whole approach can certify. The roadmap thus proceeds from demonstrating the thesis’s central capability, through completing its empirical gaps and strengthening its external validity, to extending its reach and finally raising its ceiling — a sequence of increasing difficulty whose first step carries the highest payoff because it makes the compatibility capability concrete.

8.7 Broader Implications

The findings of this thesis bear on three communities beyond the immediate one of specification inference, and naming the implications situates the work in the larger conversation it contributes to.

For the *verification* community, the thesis offers a partial answer to the specification-supply problem that has constrained the field’s reach. Deductive verifiers are powerful and mature, but they verify what they are given, and what they are given is written by hand. An inference engine that supplies accurate specifications automatically, even incomplete ones, changes the entry cost of verification: a project can begin with machine-inferred contracts and refine the ones that matter, rather than facing a blank slate. The discharge results of Chapter 4 suggest the inferred contracts are good enough that a large majority pass the verifier unaltered, which means the human effort can concentrate on the residual the machine cannot handle — the recursive, the quantified-over-arrays, the genuinely hard. The thesis does not claim to automate verification; it claims to automate the specification step that verification has always assumed, and to do so well enough that the assumption becomes realistic.

For the *testing* community, the thesis reframes the relationship between specifications and test generation. The established tools solve the stimulus problem and leave the oracle problem to the user; the thesis shows that the oracle can be supplied automatically, from inferred specifications, and that doing so improves a language-model generator’s output by a large margin. The implication is that the oracle problem and the specification-supply problem are the same problem viewed from two directions, and that solving the second — inferring specifications — solves the first — supplying oracles — as a consequence. The finding that test count is a poor proxy for oracle quality, established by the guidance control, is a methodological caution the testing community can apply independently of the inference machinery: a test-generation technique should be evaluated by mutation score or an equivalent fault-detection metric, not by the volume of tests it produces.

For the community working at the intersection of *language models and software engineering*, the thesis offers a concrete instance of a general principle: that learned models and grounded analyses are complementary, and that the productive architecture uses each where it is strong. The model is a fluent consumer of grounded information and an unre-

liable producer of it; the static analysis is a reliable producer and no consumer at all. The systems that will get the most from language models in software engineering, the thesis suggests, are those that ground the model's fluency in analysis the model cannot perform — feeding it inferred specifications, verified facts, structured context — rather than those that ask the model to manufacture the grounding itself. The producer-consumer asymmetry the thesis documents is a small, measured instance of a pattern that recurs wherever correctness matters.

8.8 Closing Remarks

The barrier to formal methods, the thesis argued at the outset, is not the verifier but the blank page — the manual cost of writing the specifications that the mature checking technology requires as input. This thesis has shown that the blank page can be filled automatically, with specifications accurate enough to be useful and compact enough to be distributed, and that the resulting specifications carry measurable value to the contemporary tools, both classical and learning-based, that consume them. It has also shown that the single-pass inference that makes this practical leaves recoverable information behind, and that a second pass recovers it.

The larger point is that inference, validation, distribution, and refinement compose. A specification inferred but unverified cannot be trusted; a specification verified but undistributed reaches no one; a specification distributed but inferred only shallowly carries less than it could. The contribution of this thesis is the demonstration that these stages can be made to work together on real code at real scale — that the path from unspecified Java to verified, distributable, refined contracts can be walked automatically, and that walking it is worth the effort because the contracts at the end of it measurably improve the tools that read them. The manual cost that has kept formal specifications on the margins of practice is, the evidence here suggests, no longer the obstacle it was.

If the thesis has a single message beyond its individual results, it is that the question worth asking about formal specifications has changed. For decades the question was whether their benefit could justify their cost, and the answer, for most software, was no — the cost was too high and was paid by humans who had other work. The thesis's evidence suggests the question is now different: not whether the benefit justifies the cost, but what becomes possible when the cost is largely removed. The studies here answer a first instalment of that question — better tests, reachable contracts, recoverable compositional information — but the question is larger than the answers, because a world in which every method has an inferred, verified, distributed contract is a world in which a great deal of tooling that has always assumed specifications as a scarce, manual input can assume them as an abundant, automatic one. What that tooling does with abundant specifications is a question the thesis opens rather than closes, and it is the question the author hopes the work that follows will take up.

Bibliography

- [1] Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt, and Mattias Ulbrich, editors. *Deductive Software Verification — The KeY Book: From Theory to Practice*, volume 10001 of *Lecture Notes in Computer Science*. Springer, 2016.
- [2] June Andronick, Ross Jeffery, Gerwin Klein, Rafal Kolanski, Mark Staples, He Zhang, and Liming Zhu. Large-scale formal verification in practice: A process perspective. In *34th International Conference on Software Engineering (ICSE)*, pages 1002–1011, 2012.
- [3] Angello Astorga, Siwakorn Srisakaokul, Xusheng Xiao, and Tao Xie. PreInfer: Automatic inference of preconditions via symbolic analysis. In *48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 678–689. IEEE, 2018.
- [4] Mike Barnett and K. Rustan M. Leino. Weakest-precondition of unstructured programs. In *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, pages 82–87, 2005.
- [5] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. Translating code comments to procedure specifications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA ’18*, pages 242–253. ACM, July 2018.
- [6] Chandrasekhar Boyapati, Sarfraz Khurshid, and Darko Marinov. Korat: Automated testing based on Java predicates. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 123–133, 2002.
- [7] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.
- [8] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma

- Rodriguez. Moving fast with software verification. In *NASA Formal Methods Symposium (NFM)*, pages 3–11, 2015.
- [9] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *International Conference on Learning Representations (ICLR)*, 2023.
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [11] David R. Cok. OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse. In *1st Workshop on Formal Integrated Development Environment (F-IDE)*, volume 149 of *EPTCS*, pages 79–92, 2014.
- [12] Henry Coles. PIT: Mutation testing system for Java. <https://pitest.org>, 2016. Accessed: 2025-01-15.
- [13] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *ACM Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.
- [14] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975.
- [15] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1976.
- [16] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. TOGA: A neural method for test oracle generation. In *44th International Conference on Software Engineering (ICSE)*, pages 2130–2141, 2022.
- [17] Natalia Dragan, Michael L. Collard, and Jonathan I. Maletic. Reverse engineering method stereotypes. In *22nd IEEE International Conference on Software Maintenance (ICSM)*, pages 24–34, 2006.
- [18] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering (PACMSE)*, 1(FSE):1889–1912, 2024.
- [19] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007.

- [20] Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *International Symposium of Formal Methods Europe (FME)*, volume 2021 of *LNCS*, pages 500–517. Springer, 2001.
- [21] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. Extended static checking for Java. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 234–245, 2002.
- [22] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. Mutation-guided LLM-based test generation at Meta. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Industry)*, 2025.
- [23] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *19th ACM SIGSOFT Symposium and 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011.
- [24] Alberto Goffi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. Automatic generation of oracles for exceptional behaviors. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 213–224, 2016.
- [25] Anthony Hall. Seven myths of formal methods. *IEEE Software*, 7(5):11–19, 1990.
- [26] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [27] Soneya Binta Hossain and Matthew Dwyer. TOGLL: Correct and strong test oracle generation with LLMs. *arXiv preprint arXiv:2405.03786*, 2024.
- [28] David Hovemeyer and William Pugh. Finding bugs is easy. In *ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 92–106, 2004.
- [29] JavaParser Team. JavaParser: Java parser and abstract syntax tree. <https://java-parser.org>, 2024. Accessed: 2025-01-15.
- [30] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.
- [31] Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2312–2323, 2023.
- [32] James C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.

- [33] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby C. Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems*, 32(1):2:1–2:70, 2014.
- [34] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. Do LLMs generate test oracles that capture the actual or the expected program behaviour? *arXiv preprint arXiv:2410.21136*, 2024.
- [35] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006.
- [36] Gary T. Leavens et al. JML reference manual. <http://www.jmlspecs.org>, 2013. Accessed: 2025-01-15.
- [37] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *45th International Conference on Software Engineering (ICSE)*, pages 919–931, 2023.
- [38] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. *arXiv preprint arXiv:2401.08807*, 2024.
- [39] Daniel Matichuk, Toby Murray, June Andronick, Ross Jeffery, Gerwin Klein, and Mark Staples. Empirical study towards a leading indicator for cost of formal software verification. In *37th IEEE International Conference on Software Engineering (ICSE)*, pages 722–732, 2015.
- [40] Bertrand Meyer. Applying “design by contract”. *Computer*, 25(10):40–51, 1992.
- [41] Davide Molinelli, Luca Di Grazia, Alberto Martin-Lopez, Michael D. Ernst, and Mauro Pezzè. Do LLMs generate useful test oracles? an empirical study with an unbiased dataset. In *40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.
- [42] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE)*, pages 75–84, 2007.
- [43] Rahul Pandita, Xusheng Xiao, Hao Zhong, Tao Xie, Stephen Oney, and Amit Paradkar. Inferring method specifications from natural language API descriptions. In *34th International Conference on Software Engineering (ICSE)*, pages 815–825, 2012.
- [44] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. *Advances in Computers*, 112:275–378, 2019.

- [45] Cedric Richter and Heike Wehrheim. Beyond postconditions: Can large language models infer formal contracts for automatic software verification? *arXiv preprint arXiv:2510.12702*, 2025.
- [46] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024.
- [47] Samuel Teuber and Bernhard Beckert. Next steps in LLM-supported Java verification. In *IEEE/ACM International Workshop on Neuro-Symbolic Software Engineering (NSE), ICSE Workshops*, 2025.
- [48] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.
- [49] Guancheng Wang, Qinghua Xu, Lionel Briand, and Kui Liu. Mutation-guided unit test generation with a large language model. *arXiv preprint arXiv:2506.02954*, 2025.
- [50] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. No more manual tests? evaluating and improving ChatGPT for unit test generation. *arXiv preprint arXiv:2305.04207*, 2024.
- [51] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. Large language models for unit testing: A systematic literature review. *arXiv preprint arXiv:2506.15227*, 2025.
- [52] Daniel Zimmermann and Heike Wehrheim. Increasing the practicality of formal methods for devops with lightweight specifications. In *Formal Methods for Industrial Critical Systems (FMICS)*, volume 11815 of *LNCS*, pages 3–20. Springer, 2019.